



Review article

Molecular property prediction: Input types and information processing in machine learning models

Muhammed Thameem^{a,b}, Obaid AlHmoudi^{a,b}, Ahmad Al Salloum^c , Naeema Al Darmaki^c, Ali Elkamel^{*,a,d,e}, Ali A. AlHammadi^{*,a,b}

^a Department of Chemical and Petroleum Engineering, Khalifa University of Science and Technology, P.O. Box 127788, Abu Dhabi, United Arab Emirates

^b Center for Catalysis and Separations, Khalifa University of Science and Technology, P.O. Box 127788, Abu Dhabi, United Arab Emirates

^c Department of Chemical and Petroleum Engineering, United Arab Emirates University, P.O. Box 15551, Al-Ain, United Arab Emirates

^d Research and Innovation Center on CO₂ and Hydrogen (RICH Center), Khalifa University of Science and Technology, P.O. Box 127788, Abu Dhabi, United Arab Emirates

^e Department of Chemical Engineering, University of Waterloo, Waterloo, Ontario, Canada

ARTICLE INFO

Keywords:

Descriptors

SMILES

SELFIES

Graphs

Transformers

LLM

Geometric graph neural networks

GNN

Deep learning

ABSTRACT

Molecular property prediction is at the core of machine learning (ML)-driven materials and drug discovery. Effectively navigating the ML workflow requires careful consideration of molecular representations, input preparation strategies, and model architectures. This review aims to provide an intuitive understanding of information processing and input construction in different machine learning models, offering guidance on how different ML approaches align with various prediction tasks and molecular representations. The methodologies are broadly categorized into descriptor-based, string-based, and graph-based models, with greater emphasis placed on graph-based approaches due to their superior ability to capture molecular structure and spatial geometry. Within the string-based category, focus is given to transformers and large language models (LLMs), which are gaining increasing attention owing to their success in natural language processing (NLP). Among graph-based models, the emerging class of geometric graph neural networks (Geometric GNNs) is discussed in detail, as these models represent the current state of the art across multiple benchmark datasets in molecular property prediction.

1. Introduction

Development of new materials and drugs involves rigorous trial-and-error laboratory experiments that are costly and time-consuming. Computational modelling reduces the burden of this approach by predicting potential candidates through virtual simulations of molecular or atomic systems. Computational methods such as Density Functional Theory [1] (DFT) simulations provide a relatively accurate means for predicting molecular properties and designing novel materials. However, DFT simulations require significant computing power and time, especially for large-scale systems [2]. Machine learning (ML) models are used to bypass this problem by learning from historical data without relying on the complex physics of atomic systems [3]. The introduction of machine learning (ML) has sparked significant attention in a wide range of areas in material sciences and chemistry, with applications in drug development [4], batteries [5], catalysis [6], and so on.

At the core of artificial intelligence (AI)-driven materials discovery is molecular property prediction [7]. As a data-driven approach, it requires input data containing existing information of molecules and output data of the properties to be predicted. AI models act as function approximators between these inputs and outputs [8]. The inputs to the model are machine-readable molecular representations – they are unique identifiers of the molecules that are more expressive compared to traditional representations such as molecular formula or IUPAC name [9]. The machine learning models extract information from the molecular representation to predict the target property. They have been used for a variety of prediction tasks, ranging from physical-level properties such as toxicity and permeability to molecular-level properties such as potential energy and atomic forces [10,11]. The selection of suitable representations and models is a multifaceted problem that requires considerations of the target application, available data, and computational resources. For example, predicting atomic forces – which are

* Corresponding authors.

E-mail addresses: ali.elkamel@ku.ac.ae (A. Elkamel), ali.aalhammadi@ku.ac.ae (A.A. AlHammadi).

<https://doi.org/10.1016/j.rineng.2026.109241>

Received 29 August 2025; Received in revised form 21 January 2026; Accepted 21 January 2026

Available online 23 January 2026

2590-1230/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

vector quantities in 3D space – requires representing the molecule in 3D and using models that can produce vector outputs that transform appropriately under rotation. In such cases, 3D molecular graphs and geometric graph neural networks [12] (Geometric GNNs) are the most suitable choices, as they inherently respect the geometric symmetries of the system.

Navigating the machine learning workflow can be challenging for researchers in chemistry-related fields. While the prediction objective and target properties may be well defined, selecting appropriate molecular representations, choosing suitable models, and preparing the input data often require careful considerations. Several comprehensive reviews exist for guiding researchers on machine learning applications in selected domains within molecular property prediction, such as catalysis, drug discovery, energy storage and materials design. Abraham et al. [13] presented a comprehensive overview of ML applications in catalysis, emphasizing descriptor selection strategies, commonly used algorithms, and their role in predicting catalytic properties. In Suresh et al. [14], a detailed review of classical and deep learning applications across various stages of drug discovery is discussed. Shen et al. [15] provided an overview of ML applications in energy storage, focusing on their impact on material design of dielectric and battery systems. Similarly, much of the recent reviews [4,16,17] within the ML-based molecular/material modelling domains follow a common structure – identifying key challenges within specific application areas, outlining relevant feature extraction and modeling approaches, and presenting comprehensive surveys that link these methods to practical implementations. While this approach is effective for guiding ML practitioners within the application domain, it can be challenging for newcomers in the field to discern the rationale behind the choice of input representations, model selection and architecture improvement strategies. To address this, several reviews have focused on the conceptual and methodological foundations of ML-based molecular modelling, aiming to provide a holistic understanding that can be extended to a wide range of applications. In this direction, Ramos et al. [18] provided a comprehensive review of large language models for general applications in chemistry, with detailed explanations of input representations, model architectures, and training strategies. Duval et al. [19] presented a comprehensive review of geometric graph neural networks for 3D atomistic systems, with a strong focus on information processing within the models. Similarly, Liyaqat et al. [20] surveyed common deep learning models and multi-modal approaches for molecular property prediction, offering key insights into input representations and information processing. Nonetheless, the survey did not include the emerging class of equivariant graph neural networks, which currently represent the state of the art across multiple benchmarks. Currently, a comprehensive review that unifies the classical, language, and advanced graph-based models for molecular property prediction, especially from the perspective of input representations and internal information processing, remains lacking. Moreover, several existing reviews that discuss information processing within machine learning models rely heavily on abstract mathematical formulations, which can be challenging to interpret for researchers in chemistry-related fields. To address these gaps, this study provides,

- A comprehensive review unifying classical-, language-based and advanced graph neural network models
- Explanations focused on information processing and its correlation with molecular input representations
- Information processing in advanced models is illustrated through simplified graphical representations, with vector- and tensor-level information pathways for clarifying how molecular information is propagated

Overall, the models discussed in this review are categorized into descriptor-based, string-based, and graph-based models, with increasing emphasis from descriptors to graphs. Descriptor-based machine learning

is included in order to provide a holistic picture, although detailed explanations of specific models are not covered. For String-based models, transformers and large language models (LLM) are focused as they are increasingly getting attention due to their success in natural language processing (NLP). Finally, a more detailed analysis on traditional graphs and geometric graph neural networks (GNNs) is discussed since they are considered state-of-the-art in several benchmark datasets used in molecular property prediction [10,21–24]. The overview of molecular representations and ML models discussed in this review is shown in Fig. 1. A summary of the class of representations, models, key advantages and limitations discussed in this review is given in Table 1. The sections in this review are summarized as follows,

Section 2: Molecular representations.

Section 3: Descriptor-based machine learning – Descriptors, input preparation and models.

Section 4: String-based machine learning – Input preparation, RNNs, transformers and LLM.

Section 5 & 6: Graph machine learning – 2D and 3D graphs, GNNs and Geometric GNNs.

2. Molecular representations

The first step in applying machine learning for molecules is to convert the molecule into a format that is recognizable by the machine learning model. In chemistry, molecules are commonly represented using their molecular formula, IUPAC name, or visually through structural diagrams such as Lewis structures or ball and stick models. However, to apply machine learning, we need more expressive representations [9] that can eventually be converted into numbers that carry information about the molecule, as computers only recognize numbers at its core. Several approaches have been explored for effective molecular representation for ML tasks, however, most of the literature [9,25–27] have focused mainly on three types: descriptor-based, string-based and graph-based representations (Fig. 2 a-d). Descriptor-based representations are unique vectors or matrices [28,29] that describe the molecule based on its qualitative and quantitative characteristics [30]. These representations can be hand-crafted, or they

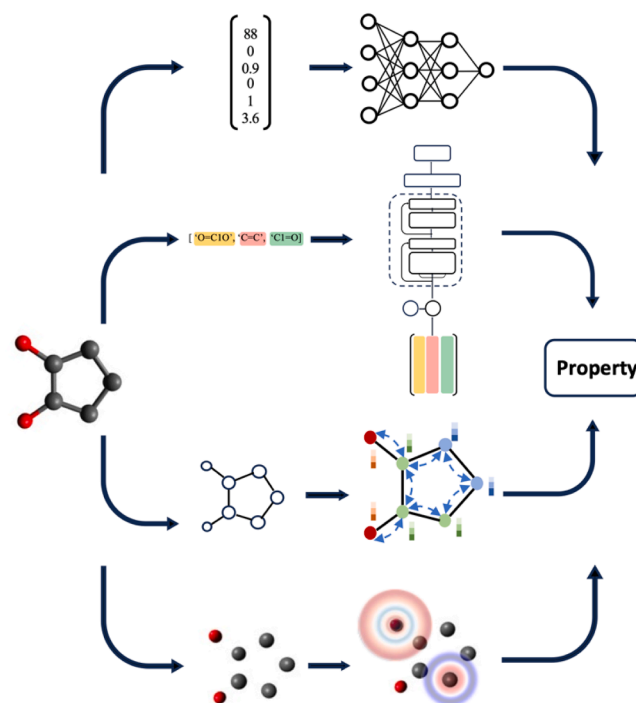
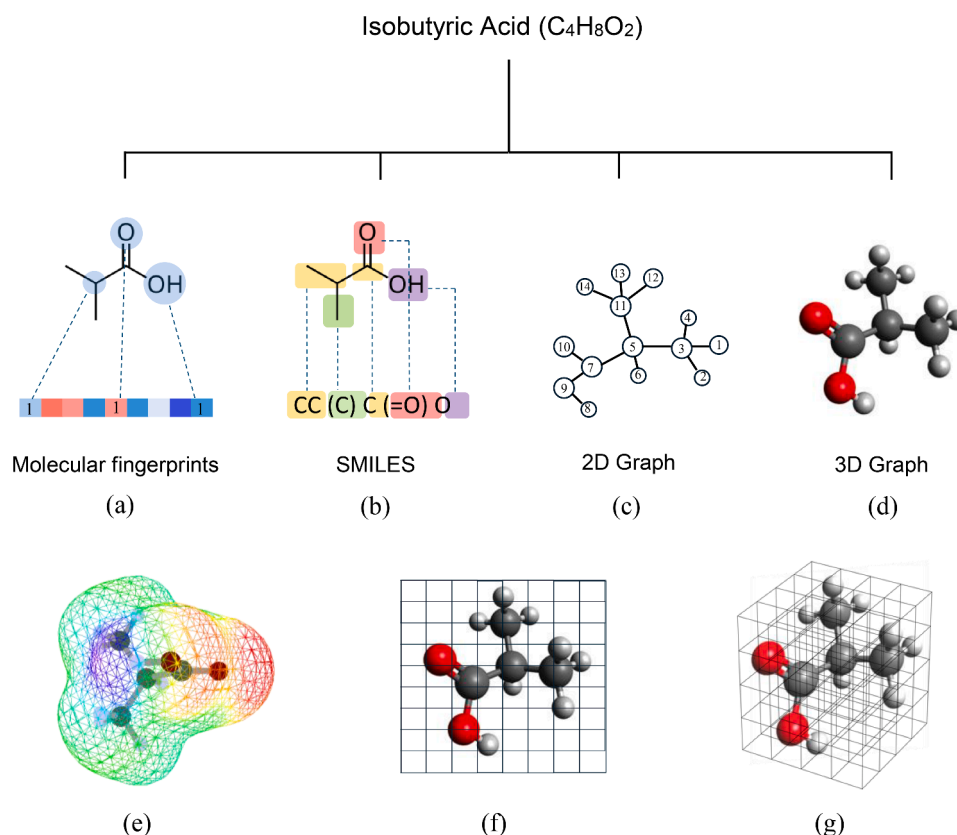


Fig. 1. Different molecular representations used as inputs to ML models for molecular property prediction.

Table 1

Summary of advantages and limitations of molecular representations and machine learning models.

Molecular representation	Model class	Information processing	Key advantage	Limitations
Descriptors	Classical ML models	Independent variable processing	Efficiently operate on low data regime	IID assumption ignores connectivity information; less suitable for inverse design
Strings	Recurrent neural networks Transformers	Sequential processing and memory gates Parallel processing and global sequence awareness with multi-head attention	Learns from simple text-based representation (e.g. SMILES) High generalizability with large data	Vanishing and exploding gradient problem in longer sequences High computational cost of models; large training data requirement; global attention may reduce sensitivity to highly localized information
2D graph	Graph neural networks	Message passing along graph connectivity	Easily captures topological and relational information	Excessive message passing in deep models can cause over-smoothing, leading to the loss of local information and discriminability
Geometric graph	Geometric graph neural networks	Symmetry-preserving message passing	Highly accurate for Interatomic potentials and atomic force prediction	Computationally expensive; advanced models require higher order representations such as spherical harmonics

**Fig. 2.** Molecular representations of Isobutyric acid (a) Descriptor-based fingerprints (b) string-based representations (SMILES) (c) 2D graph (d) 3D or geometric graph (e) surface-charge mesh (f) 2D image (g) 3D grid.

can be constructed by standardized methods based on 2D molecular structure [31–33] or 3D geometry [28,29]. The choice of descriptor representation to be used depends on the property to be predicted. For example, to predict quantum mechanical (QM) properties such as potential energy, descriptors based on 3D geometry [28,29] are preferred, since QM properties depend on 3D atomic positions. Depending on the domain of application [34–37], descriptor-based representations may also require additional property-based information about the molecule, which is often not available in large datasets [10,11,21,22]. Although the descriptor-based approach is still being used in molecular machine learning for different areas of applications (Table 2), the research is shifting towards more expressive, freely available and purely structural-dependent molecular representations [26,27]. Molecular graphs and string-based representations satisfy this requirement and are extensively used for developing generalized machine learning models

[26,38]. String-based representations of molecules rely on a standardized chemical “language”, in which each substructure is encoded as a sequence of characters or strings that collectively represent the overall molecular structure [26]. String representations are compact, widely available in large datasets [10,11], and they enable the application of advanced natural language processing (NLP) models such as Transformers [39] and pretrained Large Language Models [40–46] for molecular property prediction. Although string-based representations are based on 2D molecular structure, the connectivity information between atoms is not encoded explicitly; rather, they are encoded indirectly in the string sequence. Molecular graphs address this limitation [27] as they are based on the graph structure of the molecule with separate connectivity information, where atoms are represented as nodes and bonds as edges. In addition, the specific characteristics of atoms and bonds can be represented using node and edge feature vectors,

Table 2

Examples of descriptor-based ML models and datasets for molecular property prediction in different areas of chemistry.

Task	ML Models	Datasets	Article Reference
Catalyst property prediction	RF, SVM, MLP, Lasso, etc.	gitlab.com/SEDenmarkLab/ccheminfolib	Zahrt et al [36].
	GPR, XGBoost	github.com/SeoinBack/DSTAR-CO2RR	Mok et al [47].
	GPR, RF, MLP, Linear regression	Morgan et al [48].	Jacobs et al [49].
Polymer property prediction	RF, GPR, SVM, MLP, etc.	Li et al [50].	Li et al [50].
	GPR	Polymer genome dataset [51,52]	Kim et al [53].
	AdaBoost, XGBoost, Lasso, etc.	github.com/cataugie/polymersML	Malashin et al [34].
	SVM, Partial least square (PLS)	Wu et al [54].	Wu et al [54].
Thermodynamic property prediction	MLP, RF, GPR, SVM, etc.	PolyInfo [55]	Tao et al [56].
	RF, MLP, MPNN	Dortmund data bank [57]	Felton et al [35].
	RF, MLP, GB, SVM	Ferraz-caetano et al [58].	Ferraz-caetano et al [58].
	SVM, RF	Yaws' Handbook [59], Ghahremanpour et al [60].	Aldosari et al. [61]
Electrochemical property prediction for battery materials	SVM, MLP	Yalamanchi et al [92].	Yalamanchi et al [92].
	RF, GBM, SVM, DT	Oral et al [93].	Oral et al [93].
	RF, GBM, SVM, MLP, etc.	github.com/thePOWERligand/NCM-ML	Wang et al [94].
	SVM, DT, RF, MLP, etc.	NA	Min et al [95].
Chemical reaction property prediction	RF, SVM, MLP, Linear regression	NA	Shariq et al [96].
	MLP	github.com/apmtcc/NNrate	Lu et al [97].
	SVM, MLP, XGBoost	RMG-Py database [98]	Choi et al [37].
	RF, SVM, Lasso, etc.	github.com/doylelab/rxnpredict	Ahnehan et al [99].
	RF, XGBoost	QM7 [100], MoleculeNet [11], FreeSolv [101], etc.	Yang et al [102].

respectively. A unique characteristic of graph-based representation is that 3D spatial information of atoms can be incorporated into the graph, which is crucial for quantum-mechanical property prediction [10,23] and molecular dynamics.

In addition, several alternative molecular representations such as surface-charge meshes, 2D images, or 3D voxel-based shapes have been explored in the literature [20,26] (Fig. 2 e-g). These formats are typically processed using deep learning architectures including geodesic convolutional neural networks (geodesic CNNs), 2D CNNs, and 3D CNNs [20,26]. However, in this review, we focus only on the three most common representations: descriptors, strings, and graphs.

3. Descriptor-based machine learning

3.1. Descriptors & fingerprints

A molecular descriptor is any feature that can be used to characterize the molecule [30]. Descriptors can be based on a property, structure or 3D geometry of a molecule [47]. For example, molecular weight is a property-based descriptor represented by a numerical value (e.g., 44 for CO₂). The presence or absence of a specific substructure, such as -OH, is a structural descriptor that can be represented as a binary value: 1 if present, 0 if absent. Based on the structure and properties of a molecule, a unique vector can be constructed that can serve as the descriptor-based representation of the molecule (Fig. 3). Other than manually selecting the features to form vectors, standardized methods can be used to generate unique representations known as molecular fingerprints. Molecular fingerprints are unique vectors based on 2D structure, consisting of binary digits, in which each digit represents the presence or absence of a substructure [48]. Several standardized methods exist for creating such fingerprints, such as extended connectivity circular fingerprint [31] (ECFP), molecular access system keys (MACCS) [33], topological-torsion fingerprints [32], Morgan fingerprints [49], and atom-pair fingerprints. In contrast to vector representations, descriptors such as coulomb matrix [29] (CM) and bag of bonds [28] (BoB) are matrices based on interatomic distance in 3D space. Representations based on 3D geometry, such as CM and BoB, are particularly useful for quantum mechanical (QM) property prediction [28,29].

In addition, there are continuous descriptors based on 3D geometry that are high-dimensional and suitable for encoding more fine-grained information compared to the discrete descriptors mentioned above. These descriptors encode local environment of atoms in 3D space based on interatomic distances [50,51], angular properties [52] and quantum mechanical interactions [53,54]. Such continuous descriptors are mainly used in describing 3D atomic systems such as bulk crystals, while the discrete descriptors are commonly used for representing individual molecules [47]. Besides descriptors derived from explicit molecular properties, there also exist neural descriptors and fingerprints – vector representations learned by ML models during training. In this review, however, we use ‘descriptors’ and ‘fingerprints’ exclusively to refer to features directly extracted from the molecule, not to learned neural embeddings. Different types of molecular descriptors are illustrated in Fig. 3.

3.2. Input preparation

As mentioned in the previous section, there are several types of descriptor-based representations that can serve as the input to ML model. If manually selected features are to be used as the input, the types of descriptors to be included in the input vector must first be defined [47]. Since there are thousands of descriptors to choose from, the selection of the proper ones is highly important for efficient machine learning. This selection process can be guided by domain knowledge, where relevant descriptors are manually chosen based on their significance to the target property. However, in many cases, the right set of descriptors is chosen based on trial-and-error approach, in which the descriptor vector is adjusted based on the prediction accuracy of the model [47]. Alternative to manual selection of descriptors, standardized fingerprints can also be employed to systematically encode molecular features based on its structure. Molecular fingerprints can be easily constructed using RDKit [55] library in Python. Since many of the available datasets represent the molecule input as SMILES [56] (simplified molecular input line entry system), it needs to be first converted from SMILES to RDKit mol object to use RDKit fingerprinting functions. The functions for conversion from SMILES to mol object and creating fingerprint vectors are provided in RDKit documentation [57]. The prepared fingerprint vectors can be used as input to basic ML models and feedforward neural networks. If matrix representations such as bag

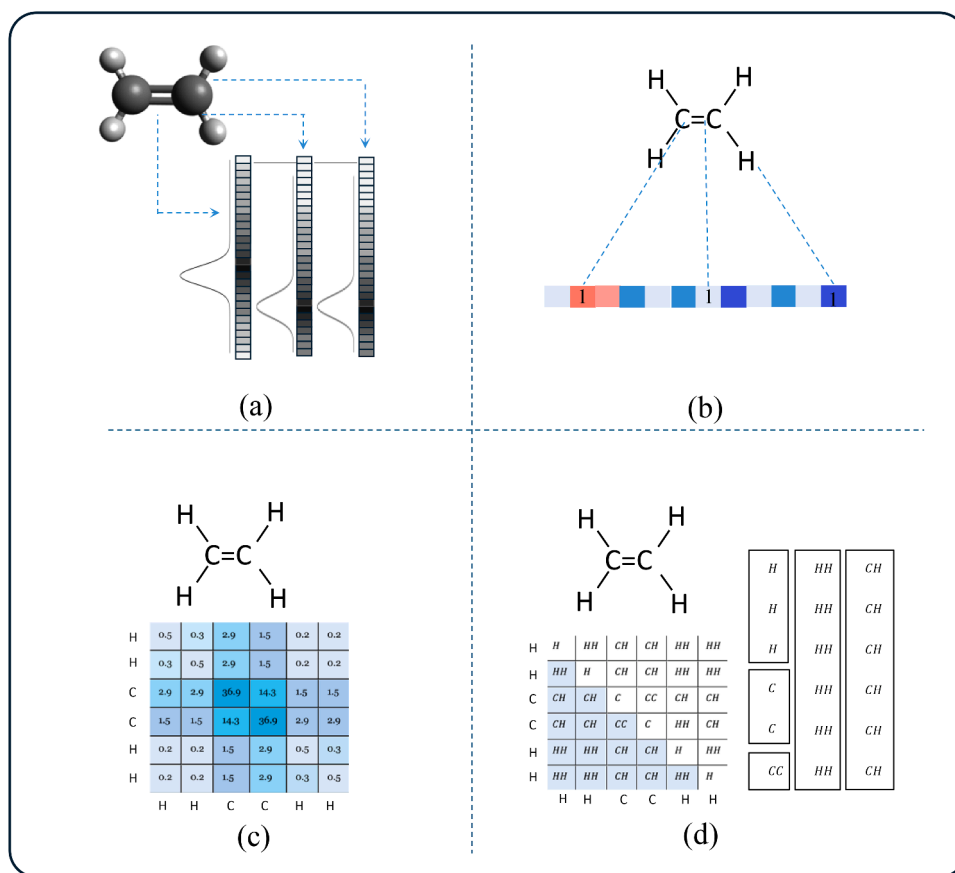


Fig. 3. Molecular descriptors (a) Continuous descriptors (b) molecular fingerprints (c) coulomb matrix (d) bag of bonds.

of bonds or coulomb matrices are chosen as molecular representations, the matrix needs to be flattened into a one-dimensional vector before using it as input to ML model.

The methods described above may produce high-dimensional input vectors containing hundreds of feature descriptors. In practice, only a subset of these descriptors could be carrying the necessary information for building a reliable model. While it is possible to build the model with all available descriptors, removing redundant descriptors can improve model accuracy, increase speed, reduce overfitting and enhance interpretability. Descriptor selection methods are commonly classified as filter-based, wrapper-based and embedded methods [58]. Filter-based methods utilize statistical metrics such as correlation, variance and mutual information, without involving any predictive model. Common techniques in this category include correlation analysis, analysis of variance (ANOVA) F-value, chi-square test, Pearson correlation, and mutual information [58]. Relying only on statistical analysis is a simpler and faster feature selection approach; however, its less accurate compared to other techniques. In wrapper-based methods, the algorithms depend on a machine learning model [59]. A feature is considered relevant when its inclusion leads to better performance of the model. The feature subset resulting in the best model performance is then selected. Common wrapper methods include forward selection, backward elimination, genetic algorithm (GA), recursive feature elimination (RFE) and simulated annealing (SA) [58]. Although wrapper methods provide strong predictive performance, they are computationally expensive and can overfit due to repeated model training. Embedded methods address these limitations by performing feature selection as part of the model-building procedure, rather than as a separate optimization step. For example, LASSO regression inherently drives the coefficients of irrelevant features to zero, so only features with non-zero coefficients are considered important [58,59]. In tree-based

models, feature importance can be inferred from how frequently and how effectively features are used to split the feature space. Similarly, in traditional feedforward neural networks, the magnitudes of learned weights or activation patterns can be analyzed to estimate the contribution of each feature to the model's predictions. Embedded methods offer a balance between computational cost and risk of overfitting. They are generally more accurate than filter-based methods, while being less computationally expensive than wrapper-based approaches. In addition to feature-selection methods discussed above, feature extraction techniques such as principal component analysis (PCA) [60] and linear discriminant analysis (LDA) [61] are also widely used as dimensionality-reduction tools to enable efficient machine learning.

3.3. Descriptor-based models

The selection of a machine learning model depends on the nature and characteristics of the input data. Descriptor-based inputs, for example, are typically encoded as one-dimensional vectors. Moreover, the values in a descriptor-based vector are often treated as independent, meaning – the sequence in which the values are arranged does not usually provide additional information. For vector inputs where sequential information is not important, classical machine learning models are well-suited. Commonly used models for descriptor-based representations include, gaussian process regression [62] (GPR), decision tree [63] (DT), random forest [64] (RF), gradient boosted trees [65] (LightGBM), extreme gradient boosting [66] (XGBoost), lasso regression [67], partial least squares [68] (PLS), support vector machines [69] (SVM) and multi-layer perceptron (MLP). Several models need to be evaluated to determine the best-suited model for the dataset. The selection of models follows the standard workflow of training, hyperparameter optimization, and testing. In the descriptor-based category, the simplest model is the linear

regression model. The model is used for capturing linear relationships between inputs and outputs, but it is not suitable for non-linear data. Linear regression is characterized by Eq. (1).

$$\hat{y} = X\beta + \epsilon \quad (1)$$

Lasso regression is a linear model with regularization. In Lasso, the coefficient β is incorporated into the loss function as a penalty term to reduce unwanted coefficients to zero. Regularization prevents overfitting and enhances generalizability. Among the non-linear models, tree-based models are widely used for molecular ML. Decision tree (DT) models work by splitting the feature space into regions that group samples with similar target values. A decision tree makes predictions using a piecewise-constant function of the form,

$$f(x) = \sum_{m=1}^M c_m 1\{x \in R_m\} \quad (2)$$

where each region R_m corresponds to a leaf node and c_m is the prediction assigned to that region.

DTs become highly effective when combined in ensemble frameworks, either arranged in parallel as in random forests [64] or sequentially as in boosted methods such as XGBoost [66] and LightGBM [65]. Tree-based models perform well with small datasets but are inherently poor at extrapolation.

SVM and GPR are kernel-based models that operate based on measuring similarity between data points through kernel functions. Kernel-based models excel at capturing non-linear relationships by implicitly mapping the data into a high-dimensional space. A unique

feature of GPR is that it captures relationships between input features through its kernel-based covariance structure, unlike the other classical models that assume all features are independent. Kernel-based models are generally defined by the Eq. (3),

$$f(x) = \sum_{i=1}^n \alpha_i K(x_i, x) + b \quad (3)$$

where $K(x_i, x)$ is the kernel function that measures similarity between the training point x_i and a new input x , and α_i and b are learned model parameters. The above-mentioned models could be classified as non-neural ML models. Among the neural networks, the most basic one is the simple feedforward neural network or multi-layer perceptron (MLP). A three-layer neural network can be defined as,

$$f(x) = \sigma(W_3(\sigma(W_2(\sigma(W_1x + b_1)) + b_2)) + b_3) \quad (4)$$

where W and b represent the linear transformation matrix and the bias term, and σ represent the non-linear activation. Generally, among the descriptor-based models, classical (non-neural) ML models perform better in terms of speed and accuracy for medium-sized data ($\sim 10k$ samples) compared to multi-layer perceptron (MLP) networks [70]. A more detailed list of advantages and limitations of classical ML models is listed in Abraham et al. [13]

Fig. 4 a-g shows simplified illustrations of classical ML models and Fig. 4 h shows an example of manual construction of a descriptor-based vector of isobutyric acid and using it as the input to the ML models. Table 2 shows examples of descriptor-based ML models used for different applications in chemistry. It is important to note that

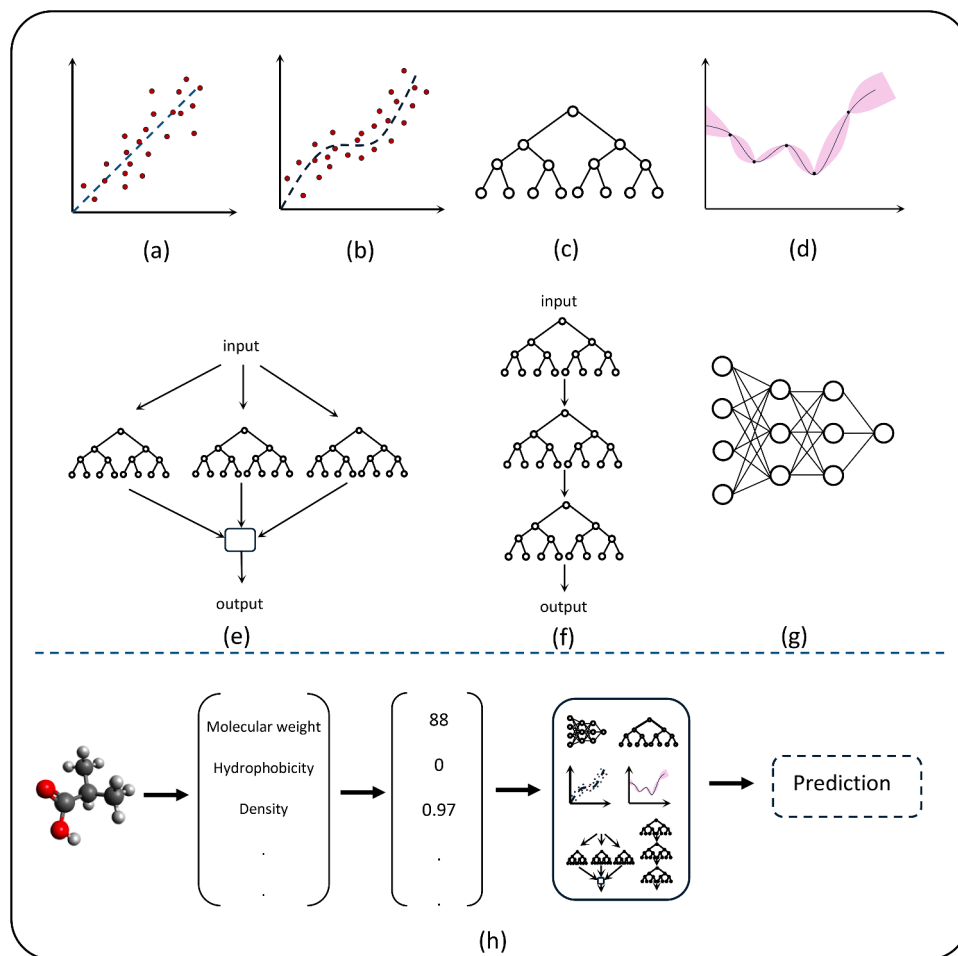


Fig. 4. Classical ML modelling (a) linear regression (b) kernel-based methods (c) decision tree (d) gaussian process regression (e) ensemble trees (f) boosted trees (g) multi-layer perceptron (h) manually constructed descriptor vector entering classical ML models.

descriptor-based vectors can also be incorporated into graph neural networks (GNNs) as node features [71], and continuous descriptors such as atom-centered symmetric functions are used to encode 3D spatial information in geometric GNNs [72–74]. However, in this review, descriptor-based models refer to classical ML models as they are solely based on descriptor representations, with no in-built mechanism to encode sequence or connectivity

3.4. Key insights and limitations

The key advantage of descriptor-based models is their ability to provide relatively accurate results with limited amount of data. Generating large-scale data from lab experiments can be costly and time-consuming, as obtaining a single data point often requires running an experiment from start to finish. On the other hand, large publicly available datasets typically come with simple molecular representations and may lack specific information on experimental conditions necessary for accurately predicting the given task. If the modelling goal can be achieved with smaller datasets, researchers generally prefer using small experimental data obtained under controlled environments rather than large datasets of unknown origin [75]. Here, classical ML models shine, as they are well-suited for applications where sufficient descriptors are available, but the number of data points is limited due to experimental cost. Moreover, even for large datasets, given that sufficient descriptors are available, classical models can still outperform deep neural networks, especially tree-based models like Random Forest and XGBoost [76].

The first challenge in descriptor-based machine learning lies in the selection of descriptors. The selection of descriptors is often subjective and it is typically determined through trial-and-error approach [47]. If a key descriptor is missing from the input vector, the model may fail to provide accurate results. Apart from the selection of descriptors, another major challenge is the generalizability of the models – how the model performs with unseen data. Generally, most ML models are interpolative, and their performance is expected to be suboptimal for out-of-distribution data. Efforts on improving generalizability have centered around making the structural information of the molecule part of the model architecture [103–105]. Classical models, however, are not inherently designed to process molecular structure. This stems from a key assumption behind most of these models, that the input variables are independent and identically distributed [106] (IID). While models like GPR do capture feature dependencies in the input, it is also not designed to capture graph connectivity. On the shift towards more generalizable models, new frontiers are being explored in molecular machine learning in search of molecular representations and model architectures that can better encode and ‘understand’ the nature of molecules [42,72–74, 103–105,107,108].

4. String-based machine learning

“Strings” in computer science refer to sequence of characters used to represent text-based data. A string can include numbers, letters, symbols, or blank spaces. String sequences that encode structural information of molecules and the models that detect patterns in sequential data are discussed in this section.

4.1. String representation

In string representations, molecules are represented as sequence of strings (set of characters), in which each string in the sequence corresponds to a substructure of the molecule. The Simplified Molecular Input Line Entry System (SMILES) [56] is the most widely used string-based representation of molecules. SMILES provides a relatively simple encoding method while capturing essential molecular structure and it is extensively used in molecular property prediction [9]. In 2020, Krenn et al. [109] introduced Self-Referencing Embedded Strings (SELFIES) as

a promising alternative for SMILES. While SMILES encoding is compact and efficient for machine learning, SELFIES employs a more comprehensive encoding approach [109] (Fig. 5 a). Despite its simplicity, SMILES has inherent limitations, including the possibility that a single SMILES string may correspond to multiple molecular structures [109]. SELFIES are designed to eliminate this problem by providing a unique representation for each molecule. Moreover, operations such as shuffling, inserting, or deleting substructure tokens always produce syntactically valid molecules in SELFIES; however, similar manipulations in SMILES can lead to invalid molecules [109]. In addition to SMILES and SELFIES, other string-based representations include InChI [110], DeepSMILES [111], SMARTS [112], and BigSMILES [111].

4.2. Input preparation

Input preparation for natural language processing (NLP) models is an integral part of its ML workflow and significantly affects the model performance. This section will explain how a typical dataset with SMILES encoding is used as input to the NLP models. The first step is the tokenization of the input SMILES. Tokenization refers to the process of splitting input strings into discrete units, known as tokens (Fig. 6 a). In NLP, tokenization splits a sentence into separate words or subword units. Similarly, for molecular strings, tokenization splits the molecular structure into substructures or character-level units. There are several ways for tokenizing SMILES input. A simple way is to split each character into separate tokens [113] (Fig. 6 a). However, care must be taken for multi-character atoms such as ‘Br’, to ensure they are not divided into separate tokens like ‘B’ and ‘r’. Another way is to split the commonly repeating substructures into separate tokens (Fig. 6 a). It can be done by methods such as Byte-Pair Encoding [114], Unigram [115], WordPiece [116] or by manually defining domain-specific tokens based on prior knowledge [117]. Once the tokenization strategy is fixed, a vocabulary of all unique tokens appearing in the dataset is constructed, and each token is assigned a unique numerical value or token ID. This mapping allows the model to convert tokens into numerical values for processing and decode them back when needed. In addition, certain special tokens can also be added to the vocabulary depending on the application [117,118], such as padding tokens (PAD), start tokens (SOS), end tokens (EOS), unknown tokens (UNK), or classification token (CLS). The usage of these special tokens is shown in the supporting information. The use of special tokens is based on the type of application; examples for its usage in molecular property prediction tasks can be found in Winter et al. [118] and Xu et al. [117] (Fig. 5 b-d). The set of tokens and their corresponding numbers are stored in a dictionary, where each token serves as the value, and its index serves as the corresponding token ID [113]. Once the vocabulary is built with all possible tokens, SMILES inputs of each molecule in the dataset need to be tokenized. In the tokenization step, the input strings are split into tokens; subsequently, each token is converted into its corresponding token ID defined in the dictionary. Substructure-level tokenization followed by conversion into token IDs is shown in Fig. 6 b. The substructure tokens and token IDs in Fig. 6 b are arbitrary and used solely for illustrative purposes; actual token assignments may vary depending on the chosen vocabulary and tokenization scheme. After tokenization, each molecule is represented as a vector of token IDs. Such a vector containing a list of numerical values, where each value corresponds to a specific string (substructure) in the input SMILES, serves as the input to the NLP model. Another approach to input representation is one-hot encoding [113]. In this method, instead of providing a vector of token IDs, each token is represented as a binary vector where the index corresponding to the token ID is set to 1, while all other elements remain 0. As a result, in one-hot encoding, the molecular representation takes the form of a matrix, each row corresponds to a token in the sequence, and each column represents a position in the vocabulary.

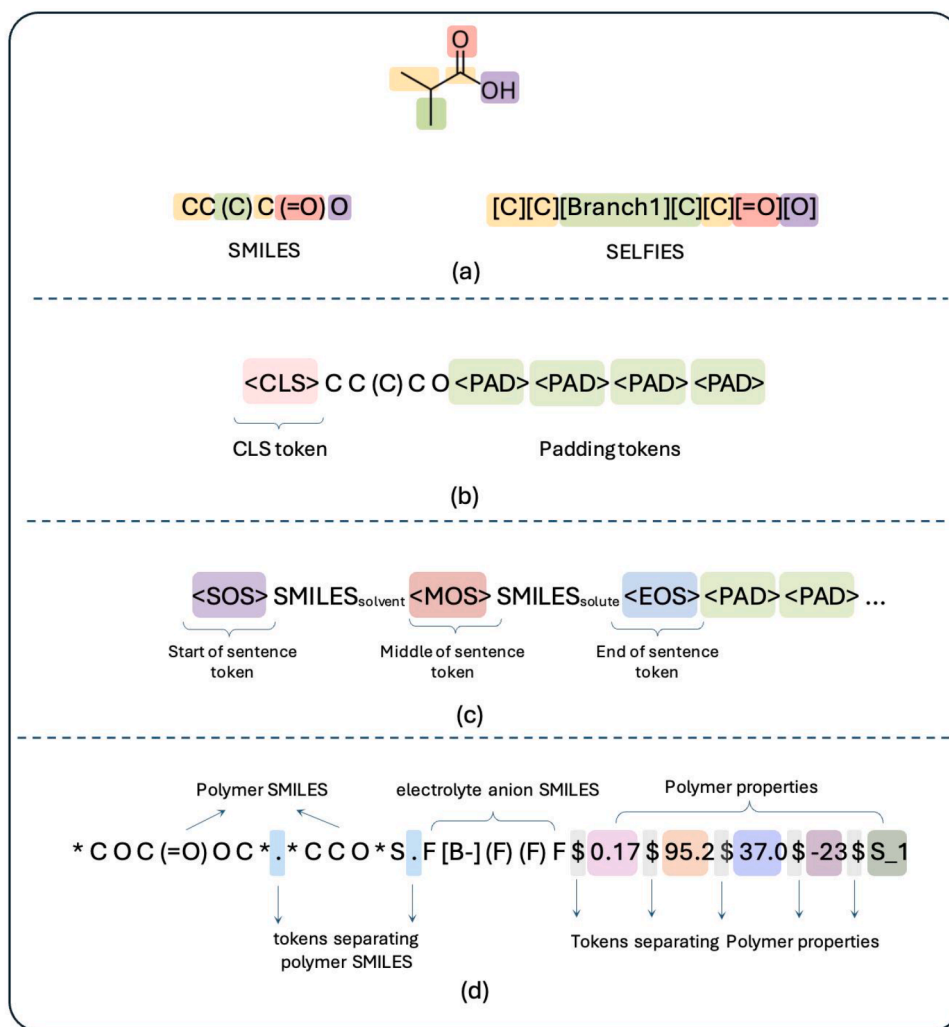


Fig. 5. String representations and special tokens (a) SMILES and SELFIES representations of isobutyric acid (b) CLS and padding tokens (c) special tokens used for representing binary mixture in Winter et al. [118] (d) special tokens used for describing polymer-electrolyte system in Xu et al. [117].

4.3. String-based models

A primary feature of string-based representations is their sequential nature. The connectivity information of the molecule is encoded based on the sequence, and any change in that sequence may result in a different molecule [109]. Therefore, in order to decode information from the string-based representation, we need models that can detect sequential patterns in the input. The class of AI models that address this type of input is called sequence models. Vanilla RNN [119], LSTM [120], GRU [121], and Transformers [39] are some of the commonly used sequence models. As these models find their application primarily in the natural language processing (NLP) domain, they are also called NLP models.

4.3.1. Recurrent neural networks (RNN)

Recurrent neural networks (RNN) are used for detecting patterns in sequential data. Although RNNs find their applications mainly in natural language processing and time-series analysis, it can also be utilized for molecular property prediction when string-based representations are used. LSTM [120] and GRU [121] are specialized versions of RNN, modified with additional memory channels for enhanced pattern detection in long sequences. This section will explain how a string-based input is processed in an RNN once the input preparation is done as explained in Section 4.2. Initially, the vector containing the token IDs or the matrix containing one-hot encodings of the input SMILES string is

passed into an embedding layer. The layer is a learnable linear transformation that converts each token IDs or one-hot encodings into a vector representation of specified dimensionality (Fig. 6 b), referred to as the embedding dimension. The output coming from the embedding layer for a given molecule is a matrix, in which each substructure of the molecule is represented by an embedding vector. The embedding matrix is fed into the RNN model, where each embedding vector is sequentially processed by individual RNN units (Fig. 6 c). The first embedding vector is passed to the initial RNN unit, generating a hidden state vector, which is then propagated along with the next embedding vector to the subsequent RNN unit. This process continues throughout the sequence, allowing the model to capture substructure dependencies. The hidden state vector from the final RNN unit carries the accumulated information from the entire sequence, and this vector can be used for predicting the target property. Typically, the last hidden state is fed to a feedforward neural network for final property prediction [122]. In the case of a multi-layer RNN, all hidden state vectors from the first RNN layer are passed as inputs to the second RNN layer [119]. The final hidden state from the last RNN layer is then used for predicting the target property. The final hidden state is supposed to contain information from the whole sequence; however, vanishing or exploding gradients during training can cause the model to forget information from earlier tokens [120], limiting what the final hidden state can represent. Models such as LSTM [120], GRU [121] and attention-based models [123] are proposed to create more ‘awareness’ to the whole sequence by preventing vanishing

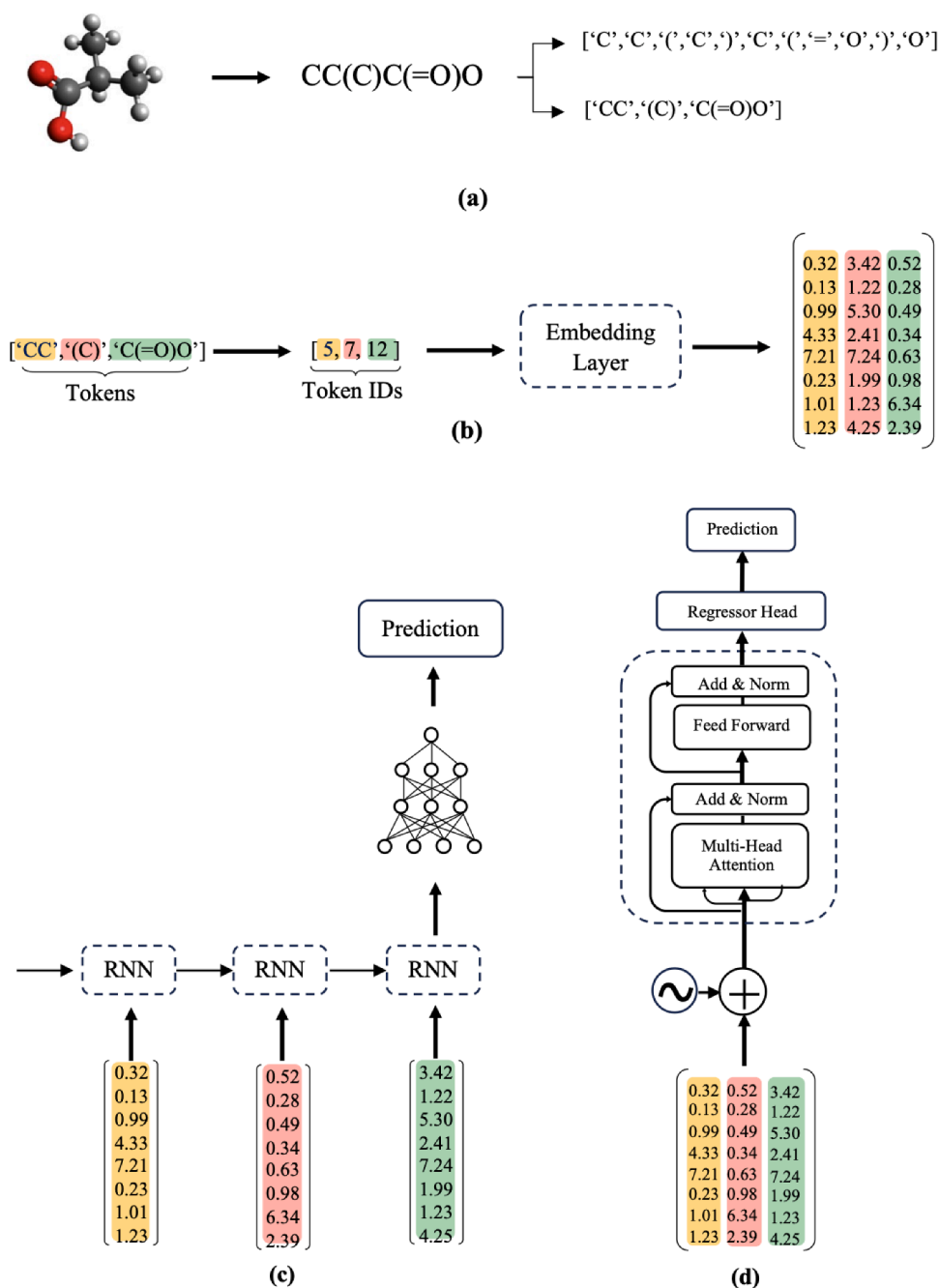


Fig. 6. (a) Character-level and substructure level tokenization of input sequence. Special tokens are not shown for simplicity (b) Tokenized input enters the embedding layer and embedding matrix is formed. Substructure-level tokens are used for simplified illustration. (c) Each embedding corresponding to molecular substructures enters the RNN units sequentially for property prediction (d) Embedding matrix enters the transformer encoder for property prediction. All embedding vectors are processed simultaneously in a transformer encoder. The values inside the embedding vector, dimensionality and token IDs are for illustration purpose only.

or exploding gradients. RNN-based models have been used in several areas [26] in molecular property prediction, including polymers [122], DNA sequence [124] and drug development [125]. In 2017, Vaswani et al. introduced transformer model solely based on attention [39], completely eliminating sequential processing used in RNN. Transformers revolutionized natural language processing, by extension, have brought powerful sequence modeling capabilities to molecular property prediction.

4.3.2. Transformer

Transformer is a natural language processing model based on the

attention mechanism [39]. The original transformer is a sequence-to-sequence model that consists of an encoder and a decoder. The encoder extracts the information from the input sequence and the decoder converts the encoded information to the output sequence. Apart from the original transformer architecture, encoder-only and decoder-only models are also used for different applications. In the context of molecular machine learning, the encoder-only models are excellent for property prediction [45], decoder-only models find its application primarily in de-novo molecule generation [126], and the original encoder-decoder models are applied in a variety of tasks, including reaction outcome prediction [127]. The following steps

outline how a SMILES input is processed in a typical encoder-only transformer for molecular property prediction (Fig. 6 d). Once the input is prepared and tokenized according to the methods specified in the input preparation section, each vector or one-hot encoding matrix corresponding to the molecules passes through an embedding layer (Fig. 6 b). Like the RNNs, the input SMILES is converted into an embedding matrix in the embedding layer. Unlike RNNs which process

embedding vectors sequentially, transformers process them simultaneously in parallel using the attention mechanism [39]. However, this parallel processing may cause the model to lose sequential information from the input, and positional or sequential information of substructures needs to be incorporated into the embedding matrix. To address this, a positional encoding matrix is added to the embedding matrix [39]. Each input embeddings are then linearly transformed into three separate

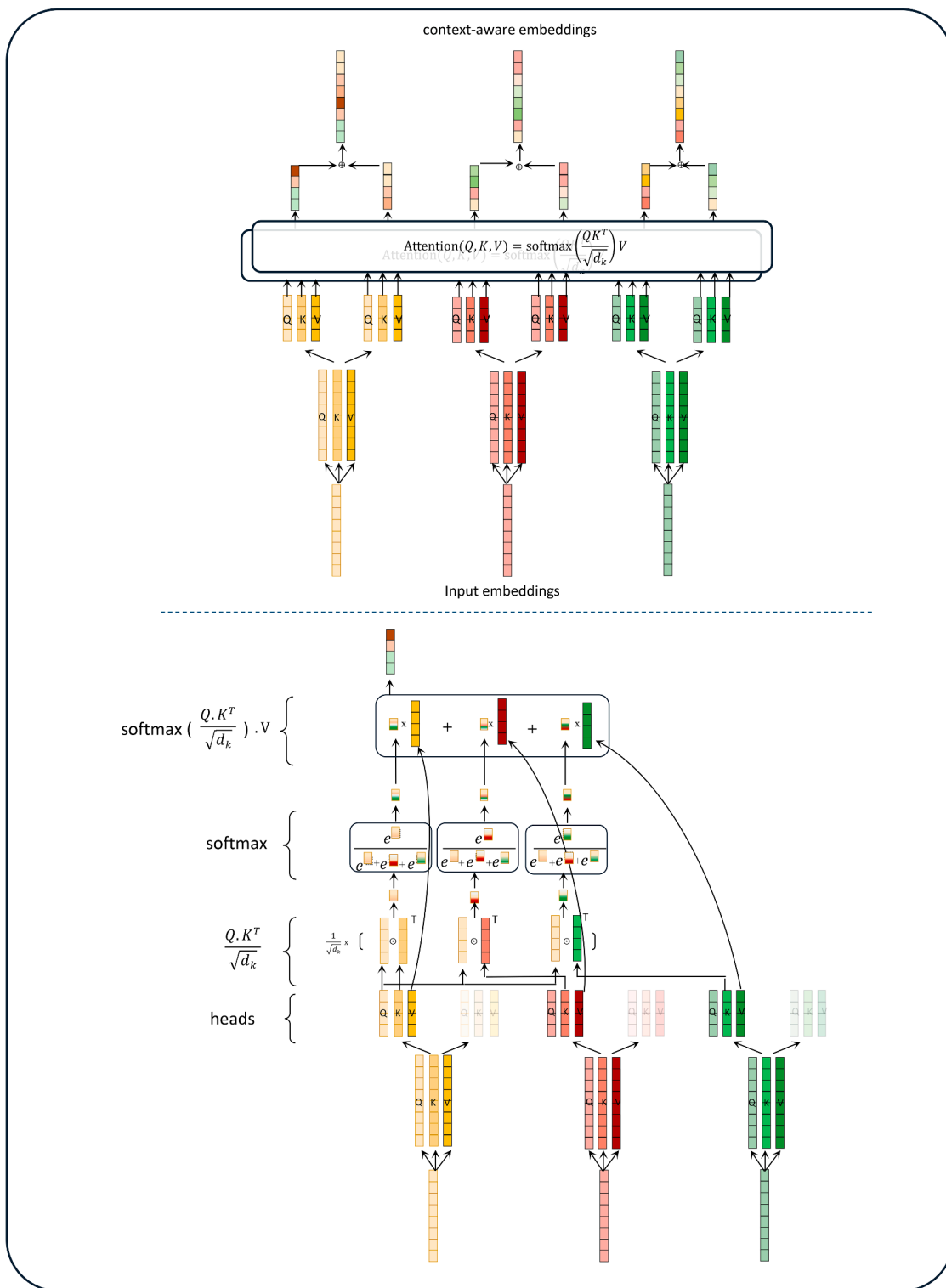


Fig. 7. Information processing in transformer encoder (a) 2-head attention layer applied to 3 input embeddings (b) Information processing for the first input token within a single attention head. 'e' refers to the exponential function.

embeddings - query (Q), key (K), and value (V) - using learned weight matrices. Before entering the multi-head attention layer, the Q, K and V vectors are split across the embedding dimension into multiple parts. The number of such splits is referred to as the number of heads of the model. The multi-head attention layer converts each head into contextual-rich embeddings of the same dimension using the self-attention mechanism described by Eq. (5), shown in Fig. 7.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (5)$$

Where d_k is the dimensionality of the key (K) embeddings. After the MHA layer, the heads are concatenated together and linearly

transformed to form the context-aware embeddings for each token, with the original embedding dimension. The MHA operation with 3 input embeddings and 2 heads is shown in Fig. 7. The output from the MHA layer, despite having the same dimension as the incoming embeddings, carries valuable information on the interdependencies between the substructures of the molecule that are crucial for predicting the target output (Fig. 7 a). A more detailed illustration of the information processing of a single head from the first input embedding is shown in Fig. 7 b

After the MHA layer, the add&norm layers (Fig. 6 d) use layer normalization [39] that normalizes the values inside the context-rich embeddings across their feature dimensions. The feedforward layer refers to a two-layer multi-layer perceptron (MLP) applied independently

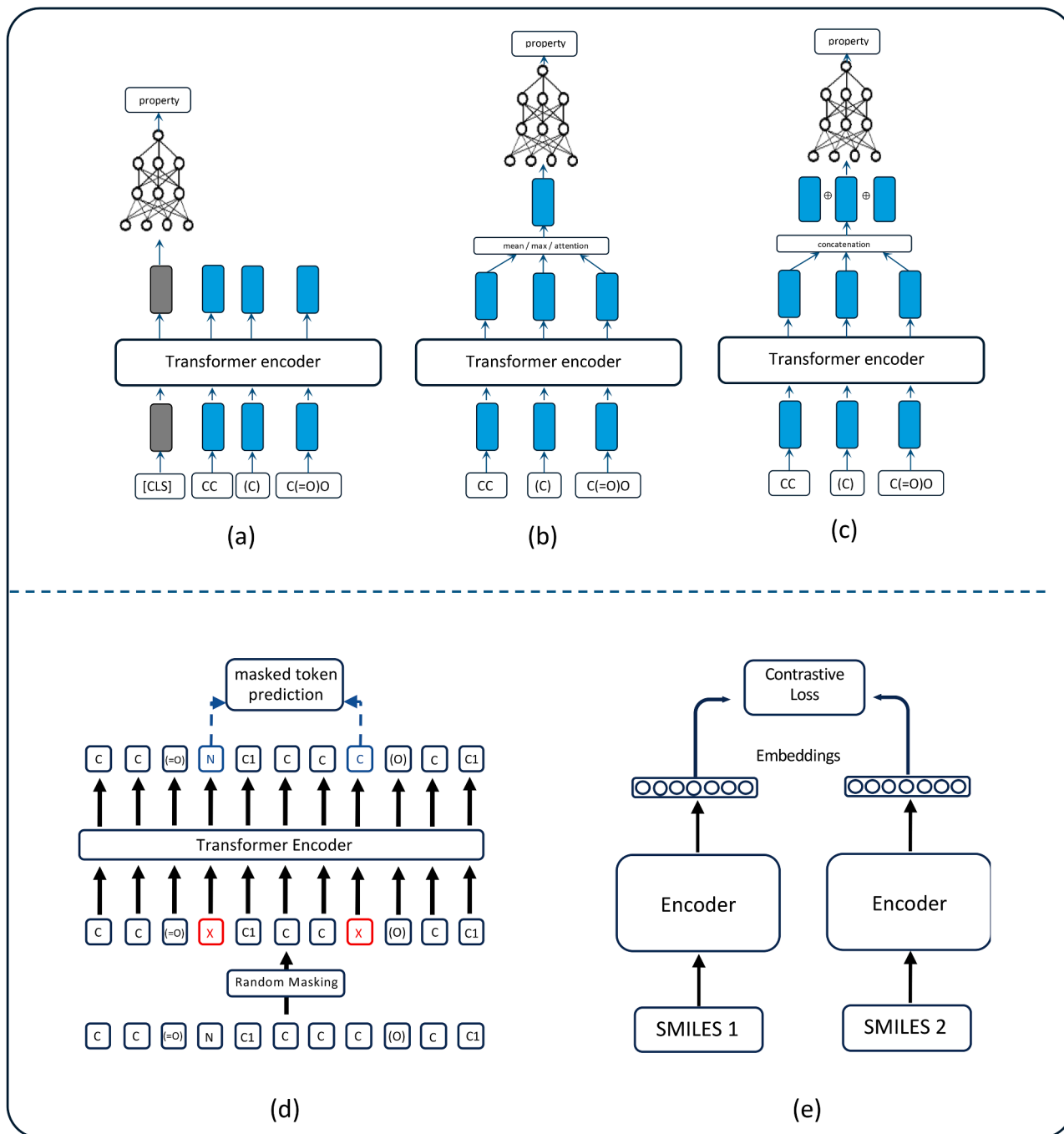


Fig. 8. Pooling operations and self-supervised pretraining methods (a) Using <CLS> token for downstream property prediction (b) Average, maximum or self-attention operation on the context-rich embeddings (c) concatenating all embeddings (d) Masked language modelling (MLM) (e) contrastive learning (CL).

to each token embedding output from the MHA layer. The regressor head in Fig. 6 d is not part of the original transformer encoder; however, it is used for regression and classification tasks using encoder-only models [117,118,128]. The regressor head typically consists of a series of feedforward multi-layer perceptron (MLP) layers for extracting the information from the contextual-rich output from the encoder to predict the target output. However, in order to apply regression or classification, the embedding matrix representing each molecule needs to be converted into a flat one-dimensional vector. There are several ways of achieving this, with the common ones being global average pooling, global max pooling, self-attention, concatenation or using the [CLS] token [129] (Fig. 8 a-c, Table S.1). After applying one of the above methods, the MLP layers can be stacked up to the final output prediction layer. The above-mentioned architecture is suitable for both regression and classification tasks [18]. The regressor head in Fig. 6 d is typically used for transfer learning or fine-tuning the model for specific applications based on the pretrained encoder part [117,118,128].

4.3.3. Large language models (LLM)

Large language models (LLMs) are transformer-based architectures pre-trained with large amount of text data [130]. Generally, LLMs follow a pre-training and fine-tuning pipeline before they are deployed for application. The pre-training stage is similar to teaching a certain language to the model to understand its syntax and contextual relationships, and fine-tuning refers to further adapting the model for a specific type of prediction by using the trained language as input. LLMs in chemical applications are pre-trained using string-based molecular representations, such as SMILES, SELFIES or any other type of text-based representation [18,41]. Once pretrained, the LLM can be fine-tuned for different applications such as molecular property prediction [45], chemical reaction outcome prediction [127] or de-novo molecule generation [126]. Encoder-only models are typically used for molecular property prediction tasks [18] such as regression and classification. For pretraining encoder-only models, the encoder part of the transformer before the regressor head is used (Fig. 6 d). During fine-tuning, the regressor head is trained on task-specific labeled data, while the encoder is either frozen or further optimized [117,118,128]. Pretraining encoder-only models can be done using labelled (supervised) or unlabelled data (unsupervised). In Winter et al. [118], synthetic labeled data from COSMO-RS was used for pretraining the model, and experimental data was used for fine-tuning the model. However, supervised pre-training tends to be more effective when the downstream property prediction task is closely related to the properties present in the labelled pretraining dataset. Supervised pretraining uses the same loss function as standard supervised learning tasks, given by

$$\mathcal{L}_{\text{sup}} = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i) \quad (6)$$

where $\ell(\cdot)$ denotes the task-specific loss function. x_i and y_i denotes the input and the corresponding label. Generally, LLMs applied to chemical tasks use unsupervised or self-supervised approach as they enable the models to learn meaningful representations of chemical language and structure without requiring labeled data. Masked language modelling (MLM) and contrastive learning (CL) are two of the commonly used self-supervised techniques for pre-training encoder-only LLMs in molecular property prediction [40,43–45,131–134]. In MLM, random tokens in the input sequence are masked, and the model is trained to predict the masked token by utilizing other tokens in the input [129] (Fig. 8 d). This forces the model to learn the syntax and contextual relationship of the language being trained, as the model can only predict a missing (masked) token accurately if it can capture the relationships between the tokens before and after the missing token. The loss function for MLM can be defined as,

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log p_{\theta}(\mathbf{x}_i | \mathbf{x}_{\setminus \mathcal{M}}) \quad (7)$$

where $\mathbf{x}_i$ and $\mathbf{x}_{\setminus \mathcal{M}}$ denote the target tokens at masked positions and the masked input sequence, respectively. $p_{\theta}(\mathbf{x}_i | \mathbf{x}_{\setminus \mathcal{M}})$ represents the model's predicted probability of the masked token $\mathbf{x}_i$ given the remaining unmasked context. In contrastive learning (CL), two or more molecular input representations are processed through encoder layers to generate their corresponding embedding vectors (Fig. 8 e). The relative similarity or dissimilarity of the molecules is compared through a contrastive loss function [135]. A commonly used contrastive learning (CL) loss was introduced by Chen et al. [136], defined as,

$$\mathcal{L}_{ij} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)} \quad (8)$$

where $\text{sim}(\cdot)$ denotes a similarity function (typically cosine similarity), and τ is a temperature scaling parameter. The contrastive loss function helps bring similar molecular embeddings closer to each other in the embedding space and ensures that dissimilar molecules are separated by a certain margin [137]. Fig. 8 e is a simple depiction of the CL method, in which only two SMILES inputs are compared using two encoders. CL method can also be applied with multiple SMILES, graphs or combinations of other representations using suitable encoders for each type of input [138–141]. CL forces similar molecules to be clustered separately in the embedding space, which is crucial for effectively fine-tuning the model for property prediction tasks.

4.3.3.1. LLM base models. LLM-based architectures for molecular property prediction often use variants of the original Transformer [39] model (Table 3). These variants differ in terms of pretraining methods, pretraining data, model configuration or the part of the transformer used (encoder, decoder or both) [18]. BERT [129], RoBERTa [142] and DeBERTa [143] are encoder-only models, LLaMA [144] is based on only the decoder, and T5 [145], BART [146] are encoder-decoder models. RoBERTa [142] base model is employed for several tasks in chemistry applications (Table 3) as it was originally optimized for text classification, which is similar to property prediction of string-based molecular representations. Table 3 shows different open-source models, training data and different transformer variants used for molecular property prediction. LLMs trained on one chemical language (e.g., SMILES) can also be efficiently domain-adapted to another chemical language (e.g., SELFIES), often with substantially lower computational cost and, in some cases, improved accuracy [147,148]. A detailed review of LLM architectures and their usage for a wide range of applications in chemistry is provided in Ramos et al. [18].

4.4. Key insights and limitations

String-based representation can harness the power of advanced sequence models for molecular property prediction. Transformers and large language models are increasingly being applied to chemical property datasets as shown in Table 3. Transformer-based variants are relatively easy to implement due to the well-established workflows originally developed for natural language processing. Most common architectures and tokenization schemes can be readily implemented using libraries such as Hugging Face in Python [172,173]. However, custom tokenization schemes and data augmentation techniques may be required depending on the specific application [117,118,155]. A key advantage of transformers is their ability to capture global relationships in input data. In a given transformer layer, each input token or sub-structure attends to all other tokens simultaneously (Fig. 7 b). This shows that the model has low inductive bias towards local chemical structure such as neighborhood patterns. This feature makes transformers particularly effective for modeling molecular-level properties that are less sensitive to small variations in local structure.

A major limitation in transformer-based models is that they require massive datasets and advanced computing resources for effectively

Table 3
Large language models for molecular property prediction.

Model	Training Dataset	Architecture	Task	Year
ChemFM [149]	UniChem database [150]	LLaMa-13B	Bioactivity, physicochemical, pharmaceutical property prediction	2025
SPT _{PC-SAFT} [151]	Synthetic data from COSMO-RS	Transformer Encoder	Thermodynamic property prediction of pure components	2024
MoleX [152]	ZINC [153]	RoBERTa	Bioactivity, toxicity predictions	2024
TransPolymer [117]	PI1M dataset [154]	RoBERTa	Polymer property prediction	2023
polyBERT [155]	PolyOne [156]	DeBERTa	Polymer property prediction	2023
MatInFormer [157]	JARVIS-tools [158]	RoBERTa	Material property prediction	2023
CatBERTa [41]	OC20 [21]	RoBERTa	Catalyst energy prediction	2023
SELFormer [42]	ChEMBL [159]	RoBERTa	Bioactivity, toxicity, physicochemical property prediction	2023
SolvBERT [43]	CombiSolv-QM [160], Solubility dataset [161]	BERT	Solvation free energy and solubility prediction	2023
LLM-Prop [162]	Materials Project [24]	T5	Crystal property prediction	2023
ChemBERTa-2 [163]	PubChem [164]	RoBERTa	Bioactivity, toxicity, physicochemical, pharmacokinetic property prediction	2022
SPT [118]	Brouwer et al. [165]	Transformer Encoder	Activity coefficient prediction	2022
MolFormer [44]	Zinc [153], PubChem [164]	BERT	Bioactivity, toxicity, physicochemical, quantum mechanical property prediction	2022
ChemFormer [166]	ZINC 15 [153, 167]	BART	Solubility, solvation free energy, lipophilicity prediction	2022
Mol-BERT [45]	ZINC [153], ChEMBL [159]	BERT	Bioactivity, toxicity prediction	2021
MolBERT [168]	ChemBench [169]	BERT	Bioactivity, toxicity, physicochemical property prediction	2020
ChemBERTa [40]	PubChem [164]	RoBERTa	Bioactivity, toxicity prediction	2020
SMILES Transformer [103]	ChEMBL24 [170]	Transformer	Bioactivity, toxicity, physicochemical property prediction	2019
SMILES-BERT [171]	ZINC [153]	BERT	Bioactivity, lipophilicity prediction	2019

training from scratch. Pretraining the model can take several days or weeks, even with relatively advanced graphics processing units (GPUs). Despite large-scale pretraining and fine-tuning, the reported performances were not always significantly higher than those of simple descriptor-based models [42,103,117,155]. In some cases, classical machine learning models outperformed transformers [76,103,155]. Therefore, a trade-off between computational cost, predictive accuracy, and data availability must be taken into consideration when choosing between classical machine learning models and transformer-based approaches. Although transformers are proven to be powerful in a wide range of sequence modelling tasks, their effectiveness in molecular property prediction is ultimately limited by the expressive capacity of the input representation.

While string-based representations encode molecular structure

indirectly through the sequence of characters, they do not explicitly express the full connectivity between atoms [27]. Instead, the connectivity is embedded implicitly in the syntax and order of the tokens. Sequence models are capable of learning patterns from these sequences and can infer structural information to some extent. However, these models are not inherently designed to prioritize or directly learn from explicit connectivity information, which may limit their effectiveness in fully capturing the relational topology of molecular graphs.

5. Graph machine learning

Graphs are mathematical structures composed of nodes connected with edges, used to model pairwise relations between objects [174]. Graph representation of molecules, input preparations and graph-based machine learning models are discussed in this section.

5.1. 2D graph representation

Molecules can be represented as graphs, where atoms are treated as nodes and bonds between them as edges. In two-dimensional space, graphs are mathematically defined as a tuple $G = (V, E)$ where V represents the set of vertices (or nodes), and E represents the set of edges. Typically, the connectivity information is stored in an adjacency matrix A of size $N \times N$, where $A_{ij} = 1$ if an edge is present between the nodes i and j and $A_{ij} = 0$ otherwise. Weighted adjacency matrix is also used in some applications, where $A_{ij} > 0$ if the edge is present between the nodes i and j , $A_{ij} = 0$ if the edge is absent. Additionally, specific characteristics of atoms, bonds and the molecule itself can be represented using node, edge and graph-level feature vectors, respectively. Commonly used features for atoms, bonds and molecule-level and their encoding types are listed in Table 4 [71,175,176].

5.2. Input preparation

The molecular graph is represented using node features, connectivity information and optionally, edge features. Node features can include the type of the atom (eg, C, H, O) and other important features depending on the atom's position in the graph. Some of the commonly used atomic features are listed in Table 4. Different types of atomic features for molecular graphs can be found in Pocha et al. [175]. Typically, the type of the atom is represented by one-hot encoding, and other atomic features can be concatenated to the one-hot encoding to form the node feature vector. For example, let's consider a molecule with elements C, H and O. For simplicity, if we manually choose the atomic number, atomic

Table 4
Atom, bond and molecule-level features commonly used in molecular graphs [71,175,176].

level	Attributes	Description
atom	atom-type	type of atoms (one-hot)
	Aromatic	whether the atom is within an aromatic ring (0 or 1)
	Donor	whether the atom donates electron (0 or 1)
	Acceptor	whether the atom accepts electron (0 or 1)
	Chirality	R or S (one-hot)
	formal charge	electric charge (integer)
	Degree	number of covalent bonds (one-hot)
bond	Radical	number of radical electrons (integer)
	hybridization	SP, SP ² , SP ³ , etc. (one-hot)
	bond-type	single, double, triple, etc. (one-hot)
molecule	conjugation	whether the bond is conjugated (0 or 1)
	Ring	whether the bond is in a ring (0 or 1)
	Stereo	Z, E, Any or None (one-hot)
molecule	average atomic weight	molecular weight divided by number of atoms (float)
	bonds per atom	number of bonds divided by number of atoms (float)

mass (assuming integer) and number of valence electrons as atomic features, the node feature vectors of C, H and O will be,

$$C = [1, 0, 0, 6, 12, 4]$$

$$H = [0, 1, 0, 1, 1, 1]$$

$$O = [0, 0, 1, 8, 16, 6]$$

The first three elements correspond to the one-hot encoding, where the first, second, and third elements represent carbon (C), hydrogen (H), and oxygen (O), respectively. The remaining three elements are the selected atomic features in the given order. The node features are highly important in graph-based molecular property prediction as they serve as the initial point of the iterative update sequence [177]. For the edge feature vector, consider the double bond between nodes 7 and 10 in Isobutyric acid as shown in Fig. 10 a. If we select the type of bond, conjugation, and ring as the bond features (Table 4), the edge feature vector will be $E_{7-10} = [0, 1, 0, 0, 0]$. The first three elements represent the one-hot encoding of bond type (single, double and triple), and the second element is 1 since it's a double bond. The 4th and 5th elements are zero as the bond is neither conjugated nor is it in a ring. Similarly, for the single bond between nodes 9 and 8 (Fig. 10 a), feature vector will be $E_{8-9} = [1, 0, 0, 0, 0]$.

The connectivity information can be encoded in several ways; two of the commonly used methods for 2D graphs are adjacency matrix and edge list. In the adjacency matrix, if a bond exists between two atoms i and j , the corresponding matrix elements (A_{ij}) are assigned a value of 1, whereas the absence of a bond is represented by 0. Since molecular graphs are typically undirected (a bond does not have direction), the adjacency matrix is symmetric, meaning that if a bond exists between atoms 2 and 3, then both A_{23} and A_{32} are 1. For example, consider a simple molecule with five atoms (Fig. 9),

The adjacency matrix will be:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Each row and column corresponds to an atom, and the entries indicate the presence of a bond between the respective atoms. In contrast, the edge list representation encodes molecular connectivity as a collection of ordered pairs. Since molecular graphs are undirected, both (i, j) and (j, i) must be included for each bond. For the graph in Fig. 9, the edge list representation would be: $\{(1, 2), (1, 3), (2, 1), (2, 3), (2, 4), (3, 1), (3, 2), (3, 4), (4, 2), (4, 3), (4, 5), (5, 4)\}$. While both methods provide essential structural information about the molecule, the edge list is the default method in PyTorch-Geometric [178] library in Python.

5.3. Graph-based models

5.3.1. Graph neural networks (GNN)

Graph neural network (GNN) are used in machine learning for predicting the properties of graph data. The types of regression or

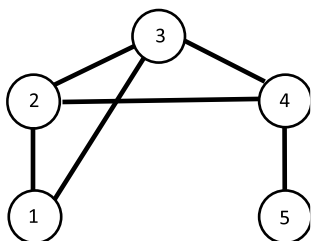


Fig. 9. An example graph structure of demonstrating adjacency matrix and edge list.

classification tasks that can be done using GNNs are typically categorized into node-level, edge-level and graph-level property prediction [38]. In chemistry applications, molecular property prediction typically falls into node-level and graph-level tasks: graph-level prediction focuses on properties of the entire molecule (whole graph), while node-level prediction targets atom-specific quantities such as per-atom energies. A wide variety of GNNs are proposed in the literature for molecular property prediction [74,104,105,179–182]. Although the exact architecture of the GNNs may vary, the vast majority of these architectures can be categorized into three general "flavors" of GNN layers: convolutional, attentional, and message passing [183]. Convolutional GNNs can be viewed as a special case of attentional GNNs [184], where the attention mechanism reduces to fixed or uniform weights. Furthermore, both convolutional and attentional GNNs – along with most GNN variants used for molecular property prediction – can be unified under the framework of message passing neural networks (MPNNs) [182,183].

5.3.2. Message passing neural network (MPNN)

Message Passing Neural Networks (MPNNs) operate by updating node features based on information from their neighboring nodes in the graph. In each layer of the MPNN, node features are iteratively updated, and the final node representations are subsequently used for property prediction (Fig. 10 d). Mathematically, MPNNs mainly consist of two components: a message function and an update function [182]. The message function determines the method by which a given node collects information from its neighboring nodes, and it varies depending on the model architecture. The update function determines how the aggregated neighboring messages should be combined with the existing node features. Aggregation operation should be a permutation invariant operation [183], meaning the output from aggregation should not depend on the ordering of the neighboring nodes. Therefore, operations such as sum, average, maximum, and other alternatives [185–187] are used because they are permutation invariant. In the first layer of the MPNN, the message function collects information (or messages) from a node's neighboring nodes, while the update function updates the current node's feature vector by collecting the incoming messages and combining them with the node's existing features [182]. In the second layer of the MPNN, the nodes that were updated in the first layer are further updated using the message and update functions. This process is continued for each layer of the MPNN. A key distinction between MPNNs and transformers is that, in an MPNN's first layer, information is gathered only from a node's local neighborhood. However, in transformers, each substructure or token attends to all other substructures, and all long-range interactions are captured in a single layer. In MPNN, Long-range interactions gradually accumulate over multiple message-passing layers. This strong inductive bias makes MPNNs particularly effective for predicting short-range, localized properties.

The message and node update functions are given in Eqs. (9) and (10). m_i^{t+1} is the combined message. Once the node features are updated, a pooling mechanism is used to aggregate all node features into a single embedding vector that represents the entire graph (Fig. 10 d). Pooling can be a sum, average, max, MLP or attention-based operation on the updated nodes. The final graph embedding is used for graph-level prediction of properties. In Fig. 10 d, a feedforward dense layer is followed after the embedding vector for property prediction. The pooled graph embedding acts as a learned molecular fingerprint, which can also be passed to descriptor-based ML models such as SVM, RF, DT, or GPR instead of using a neural-network dense layer for prediction [188–190]. Fig. 10 d shows a schematic diagram of MPNN for molecular property prediction with a single layer of node update.

$$m_i^{t+1} = \sum_{j \in \mathcal{N}^-(i)} M_t(h_i^t, h_j^t, e_{ij}) \quad (9)$$

$$h_i^{t+1} = U_t(h_i^t, m_i^{t+1}) \quad (10)$$

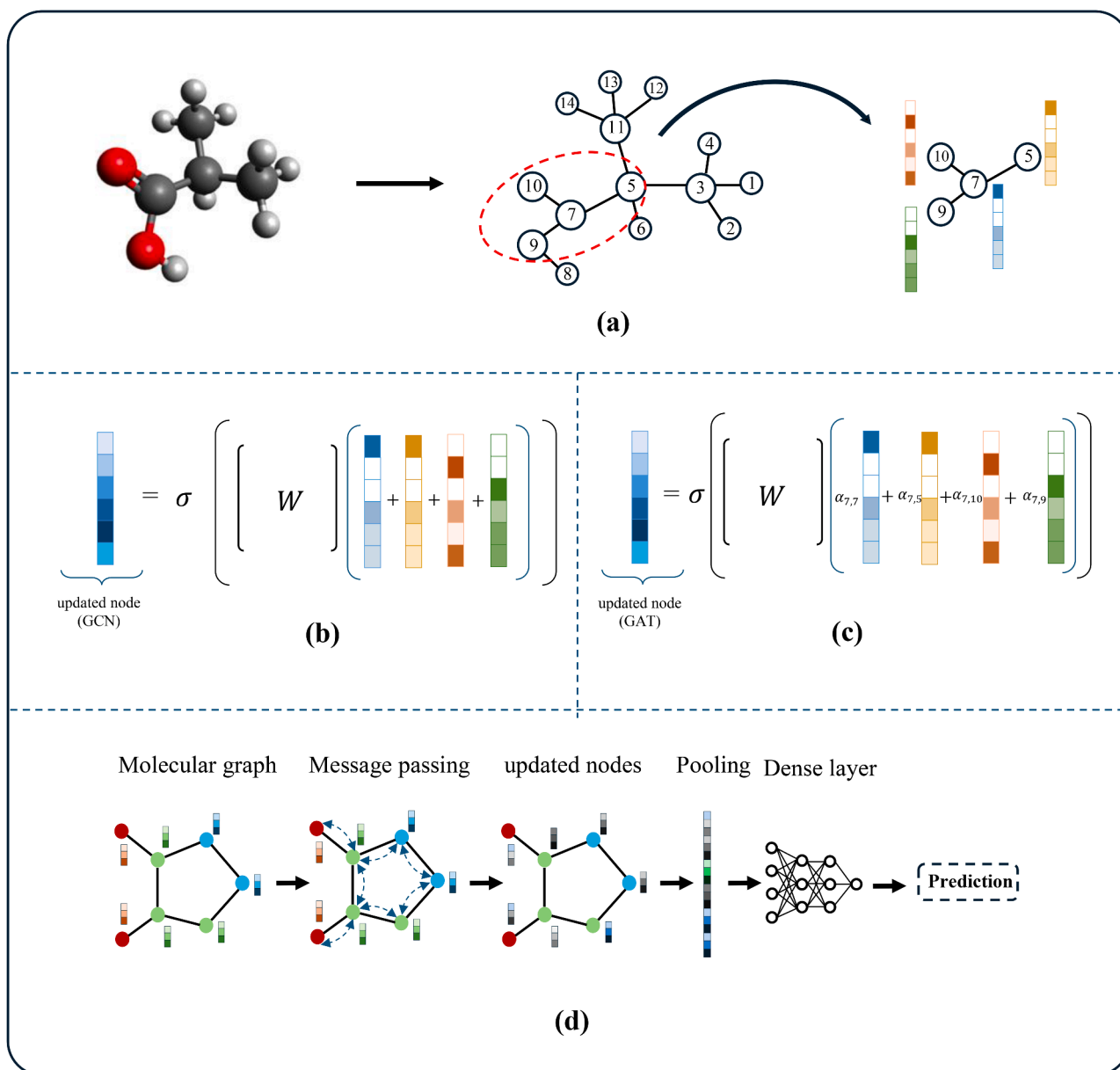


Fig. 10. (a) The neighborhood of the node 7 in Isobutyric acid is focused for demonstrating graph convolution and attention (b) Node update of node 7 using graph convolution with sum aggregation (c) Node update of node 7 using graph attention (d) schematic diagram of property prediction using MPNN.

The Eqs. (9), (10) are a simplistic representation message aggregation and update in MPNNs [182], where m_i^{t+1} is the aggregated messages from the node i 's neighborhood and h_i^{t+1} is updated node i feature after $(t+1)^{\text{th}}$ update. h_i^t , h_j^t , e_{ij} represent feature vectors of node i , node j , and edge $i-j$, respectively, after t updates. U_t represents the update function and $\mathcal{N}(i)$ is the set of nodes in the neighborhood of i .

5.3.3. Graph convolution network (GCN)

Graph convolution networks (GCN), introduced by Kipf et al [191], is a form of MPNN and are widely used for molecule property prediction tasks. Graph convolution of node features mainly involves two steps: aggregation and transformation [191]. For a given node, its neighboring node features are combined using a suitable aggregation function, then they are linearly transformed and combined with the current node feature. Fig. 10 b demonstrates the node update in a GCN model. After the aggregation, a linear transformation is applied, followed by a

non-linear activation function. In Fig. 10 b, W is a learnable weight matrix (linear transformations) and σ denotes the non-linear activation function (ReLU). Comparing with MPNN Eqs. (9) and (10), the message and update functions in Fig. 10 b could be written as $M_t(h_i^t, h_j^t, e_{ij}) = W^t h_j^t$ and $U_t = \sigma \left(W^t h_i^t + \sum_{j \in \mathcal{N}(i)} W^t h_j^t \right)$. Once the nodes are updated, a pooling function is used to aggregate all node embeddings to a single vector, which is then used for property prediction as shown in Fig. 10 d

5.3.4. Graph attention network (GAT)

Graph attention networks (GAT) use attention mechanism in the aggregation and transformation function for updating node features [184]. In the traditional GCN model [191], the aggregation operation combines the information from all neighboring nodes without giving special attention to any of the nodes. However, the GAT model incorporates attention-based message passing that allows the model to provide more or less importance to certain neighboring nodes over

others during aggregation. An additional attention coefficient α is used for weighing the importance of each neighboring node to the current node embedding. Fig. 10 c shows the node update in GAT, where W is the learnable weight matrix and σ is the non-linear activation (ReLU). The parameter α_{ij} indicates the importance of node j to the node i , considering all other neighbors of node i , and is given by Eq. (11),

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_k]))} \quad (11)$$

where LeakyReLU is a non-linear activation function, W and $\mathbf{a}^T$ are learnable parameters, and $\mathcal{N}(i)$ refers to the neighbors of node, and $\mathbf{h}_i$ is the embedding of node i .

5.3.5. GraphSAGE

Building on the original GCN by Kipf et al. [191], custom GNN frameworks were later introduced in models like GraphSAGE [192]. GraphSAGE [192] used three different aggregation functions: mean, max, and LSTM. Initially, a linear transformation followed by non-linear activation is applied to the neighboring node features. The aggregation functions are then applied to the transformed neighborhood features (Fig. 11 b). The aggregated message is then concatenated with the updating node feature, and another linear transformation and non-linear activation are applied (Fig. 11 c). Although LSTM is a sequence model, GraphSAGE results illustrated that it showed competitive performance compared to other aggregation functions. Unlike GCN and GAT shown in Fig. 10, GraphSAGE preserves the node's own feature separately via concatenation before applying the final node update (Fig. 11 c).

5.3.6. Edge-aware GNN

The GNN models described above only use node features and connectivity information. In practice, incorporating edge features into the

message passing function is crucial for accurate property prediction. A simple example of incorporating edge features is shown in Fig. 13 c. This specific message passing scheme is taken from the TransformerConv model in PyTorch Geometric [178]. In this model, after the attention-based node feature summation, edge features are also summed with attention, and a separate learnable weights and non-linearity is applied to the edge features. This architecture can greatly improve the accuracy of properties such as atomic potential energy as it directly depends on interatomic distances.

6. Geometric graph neural networks

Molecular machine learning has been used for a variety of prediction tasks, ranging from physical-level properties such as toxicity and permeability to molecular-level properties such as potential energy and atomic forces. While physical-level property predictions are valuable in their respective domain, the prediction of energy and atomic forces is particularly important as they can be used for molecular dynamics simulations, which is crucial for studying catalysis [193], batteries [194], drug-target interactions [195], and so on. Common benchmark datasets for energy and force prediction consist of data obtained from DFT calculations. DFT calculates energy based on atomic positions in 3D space [1], and atomic forces are obtained as the negative gradient of energy with respect to coordinates. GNN models that incorporate 3D coordinates and geometric information, commonly referred to as geometric graph neural networks, have been shown to achieve state-of-the-art performance across various energy and force prediction benchmarks [10,21–23,196]

6.1. Model requirements for energy and force prediction

For accurate prediction of DFT-calculated energy and forces, the

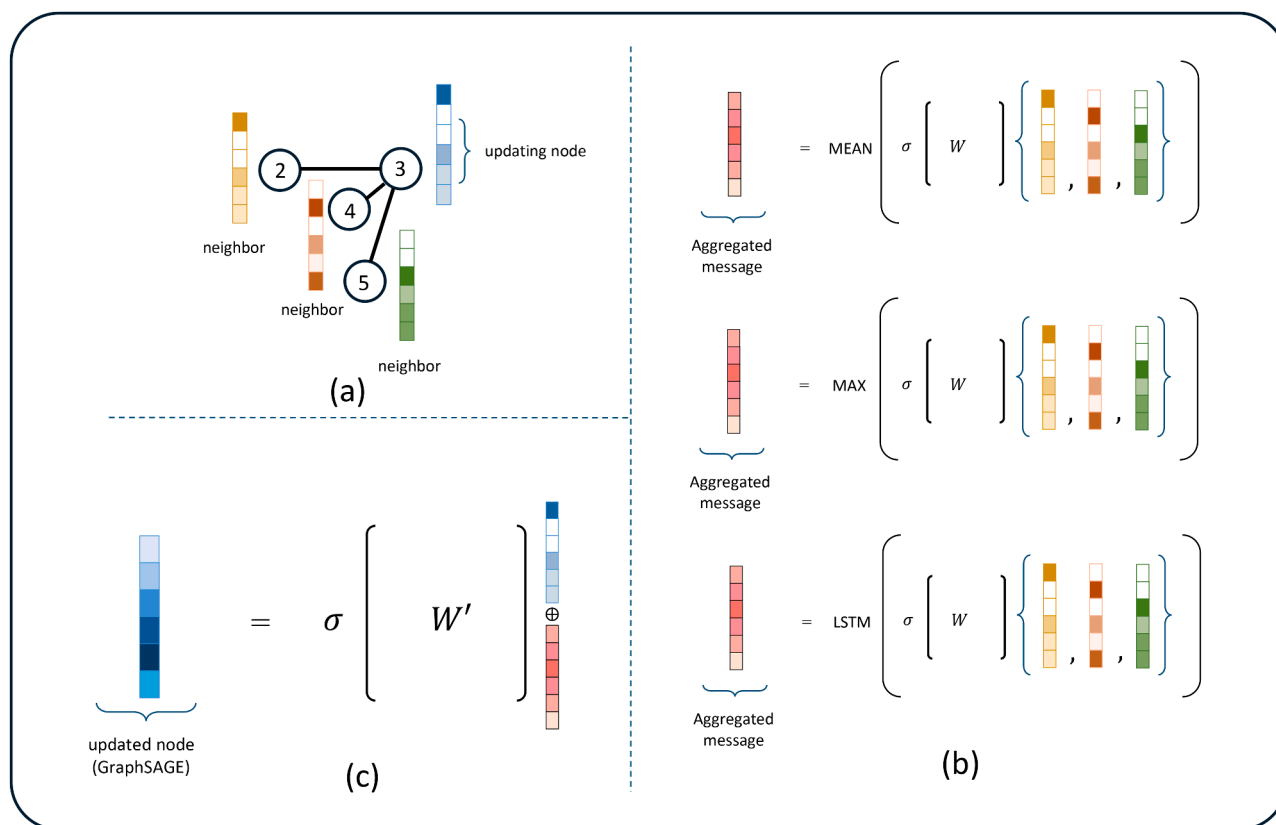


Fig. 11. GraphSAGE message passing (a) Neighborhood in focus (b) Aggregation functions in GraphSAGE (c) Node update with aggregated message and current node features. Bias terms are excluded for simplicity.

model must follow basic laws of physics, either by directly incorporating physics into the model or by indirect learning [72]. For example, if we rotate a molecule, its total energy must remain the same and the atomic forces must rotate in accordance with the molecule's rotation. In addition, the predicted force field should ideally satisfy the potential energy conservation, i.e. $F_i = -\frac{\partial E}{\partial r_i}$, where F_i is the force acting on atom i . It implies that the predicted energy must be differentiable across the coordinate space, which means the potential energy surface should not have any discontinuity or sharp edges. Moreover, for stable molecular dynamic simulations, atomic forces must be differentiable at all points [72]. Therefore, the model for energy prediction should be double differentiable i.e. first derivative ensures smooth potential energy surface and conservative force field, and the second derivative ensures smooth force fields for stable molecular simulations. In addition, for predicting physical properties such as thermal conductivity, dynamic stability, thermal stability, and optoelectronic behavior, precise prediction of phonon band structures and vibration modes is required [197]. For this task, more fine-grained variations in potential energy surfaces must be accurately captured. Therefore, when phonon band structures are to be calculated, higher-order derivatives of the learned potential energy surfaces should exist [197].

6.1.1. Geometric invariance

Geometrically invariant features are the characteristics of a geometric object that do not change based on geometric transformation such as rotation, translation, scaling or reflection. A neural network model $f(x)$ is described as invariant to the transformation $\mathcal{T}$, if,

$$f(\mathcal{T}(x)) = f(x) \quad (12)$$

Meaning, despite transforming the coordinates of the input x by the transformation $\mathcal{T}$, the output of the function remains as $f(x)$. Such a model is termed an invariant model. In an atomic system, the overall energy of the system is an invariant property, so if the coordinate system is rotated or translated, the overall energy should remain the same. Geometric invariance can be indirectly incorporated into the model through data augmentation, in which several transformations of the input data is added into the existing dataset, without varying the target data of transformed samples. However, this is computationally expensive as it requires massive datasets consisting of numerous geometric transformations for effective training. Another method is by directly incorporating invariant geometric features such as interatomic distances [74], bond angles [72], dihedral angles [73], etc. into the model. For example, if the model only uses interatomic distance as the geometric feature for energy prediction (e.g. SchNet [74]), rotating or translating the molecule will not alter the predicted energy, as the interatomic distance is unaltered by such transformations.

6.1.2. Geometric equivariance

Geometrically equivariant features are characteristics of a geometric object that change in a predictable manner under geometric transformations such as rotation, translation, or reflection. A neural network model $f(x)$ is described as equivariant to the input transformation $\mathcal{T}$, if,

$$f(\mathcal{T}(x)) = \mathcal{T}'f(x) \quad (13)$$

Where $\mathcal{T}'$ is the corresponding transformation of the output with respect to the input transformation $\mathcal{T}$. Atomic forces and velocity are equivariant properties, while the total energy is an invariant property. For example, if the coordinate system is rotated, the forces acting on each atom should also rotate accordingly. One common way to preserve equivariance of atomic forces is by calculating it directly from predicted energy using $F_i = -\frac{\partial E}{\partial r_i}$, in which the gradient is calculated using backpropagation [72–74,198]. Using the gradient methods ensures equivariance and energy conservation; however, the accuracy of the predicted forces is inherently limited by the accuracy of the energy prediction. An alternative approach is to calculate force directly as the

model output. However, as a result of decoupling energy and force predictions, models that predict force directly from the input may produce force fields inconsistent with energy conservation. Still, direct force prediction is used in some modern geometric GNNs as it bypasses the need for backpropagation for force prediction and significantly improves model training efficiency [197].

6.1.3. Basis functions

Geometric GNNs use basis functions to encode geometric features such as interatomic distances or angles as high-dimensional representations. Basis functions convert scalar features into more expressive representations. For example, SchNet [74] and DTNN [198] used interatomic distance to predict energy. Instead of representing the interatomic distances as scalars, they expanded each distance into high-dimensional vectors using Gaussian radial basis functions (RBF). Fig. 13 b shows the distance d_{ij} converted into a high-dimensional signal in SchNet [74]. In SchNet [74], a single interatomic distance is encoded using 300 gaussian radial basis functions of the form, $RBF = \exp(-\gamma \|d_{ij} - \mu_k\|)$, where $\gamma = 10 \text{ \AA}^{-1}$ and μ_k varies from $0 \text{ \AA}$ to $30 \text{ \AA}$ with an increment of $0.1 \text{ \AA}$ for each of the 300 Gaussian functions. This output signal representing the distance will be continuous and differentiable with respect to r , since it is the sum of Gaussian functions. An illustration of 5 Gaussian RBF functions is shown in Fig. 12 b DimeNet [72] and GemNet [73] used Bessel functions for distance expansion (Fig. 12 a) and spherical Fourier-Bessel basis for joint expansion of both distance and angular information (Fig. 12 c). In recent equivariant geometric GNNs, spherical harmonics are used as basis functions of irreducible representations to construct steerable features that transform equivariantly under 3D rotation [108] (Fig. 12 d).

6.2. Geometric graphs

Geometric graphs are represented using adjacency matrix, node scalar features, 3D coordinates and optionally, node vector features. It is defined as $G = (A, S, X, \vec{V})$, where, A , S , X and $\vec{V}$ represent adjacency matrix, scalar node features, 3D coordinates, and the optional vector features, respectively [12]. For N number of nodes, and 'd' dimensional scalar and vector features, $A \in [0, 1]^{N \times N}$, $S \in \mathbb{R}^{N \times d}$, $X \in \mathbb{R}^{N \times 3}$ and $\vec{V} \in \mathbb{R}^{N \times d}$.

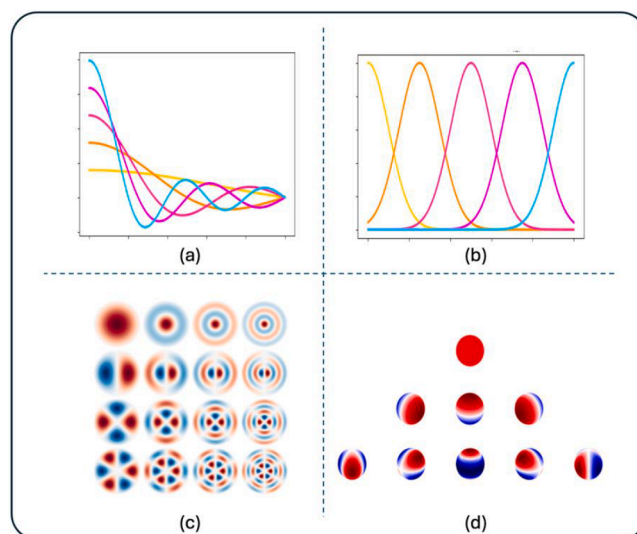


Fig. 12. Basis functions for capturing geometric information. (a) Bessel radial basis functions (b) Gaussian radial basis functions (c) 2D Fourier-Bessel basis functions (d) Spherical harmonics.

6.3. Input preparation

Inputs to the geometric GNN models typically are a set of atomic numbers $z = \{z_1, z_2, \dots, z_n\}$ and coordinates $X = \{x_1, x_2, \dots, x_n\}$ [177]. In addition to this, more hand-crafted node and edge information (Table 4) can be added as input features. However, discrete features that cause discontinuity in the potential energy surface are typically avoided [74] in machine learning interatomic potential (MLIP) models. Unlike 2D graphs that define connectivity information based on bonds in the molecule, geometric graphs use a cut-off radius and all nodes within the cut-off are defined to be connected. In the adjacency matrix, $A_{ij} = 1$ if the distance between the nodes is less than or equal to the cutoff distance; otherwise $A_{ij} = 0$ [12]. However, using a sudden cutoff can cause basis functions to be discontinuous at the cutoff radius [72]. This can be avoided by wrapping discontinuous functions with differentiable (e.g. polynomial) envelopes [72] or by using a gradual cutoff [199] to ensure double differentiability (for force prediction) at all distances. If the model requires data augmentation, the existing input data needs to be modified with additional input transformations for learning invariance or equivariance.

6.4. Embedding, interaction and output layers

Typical geometric GNN architectures [72–74] consist of an embedding layer, a series of interaction layers, and output layers. The embedding layer converts the atomic numbers of each node into learned embedding vectors of specified dimension, similar to NLP models (Fig. 6 b). In the interaction layers, the node features are updated based on the geometry of the molecule. The geometric information encoded using basis functions is used to form message functions, and each node feature is updated using the update function. Invariant GNNs update only scalar features and equivariant GNNs can update both scalar and vector features based on the geometric information. Once the features are updated, a readout function such as sum or average is used to create a single embedding for graph-level prediction. The output layer converts the graph-level embedding into the target prediction. In the output layers, a series of MLPs are used for invariant property prediction [72–74], and learnable, equivariance-preserving operations are used for equivariant property prediction [200–207].

6.5. Geometric invariant models

Invariant models ensure that the output (e.g. energy) remains the same when the same input is rotated or translated. This is achieved by using invariant geometric features such as interatomic distance [74] or bond angles [72,73] to form internal representations of the model. When the input is transformed (e.g. rotated), the internal representation will not change since the interatomic distance or angles do not change with the rotation of the molecule. As a result, the model will predict the same output for different transformed versions of the same molecule.

6.5.1. Distance-based models

6.5.1.2. Deep tensor neural network (DTNN). DTNN [198] is an early work in geometric GNNs for energy and force prediction based on interatomic distances. The distances were expanded using gaussian radial basis functions. Each node was updated based on the neighboring node features and the distances to the neighbors. The total energy was learned from individual node level energies, and atomic forces were calculated based on gradient method, $F_i = -\frac{\partial E}{\partial r_i}$. Using the gradient approach for force calculation ensures equivariance; however, since the forces are calculated from the predicted energy, the accuracy of the force prediction can only be as good as the energy prediction.

6.5.1.3. Crystal graph convolution neural networks (CGCNN). CGCNN

was designed for predicting the properties of crystalline materials [208]. The node features were generated using the properties of atom and its position in the periodic table. As CGCNN was one of the earliest works that used 3D geometric information of atomic systems, its architecture differs significantly from other advanced invariant models. The edge feature vectors in CGCNN were generated by computing the interatomic distances in 3D space and then creating a one-hot encoding by categorizing these distances into 10 categories. The model was used for a wide range of property prediction, including overall potential energy.

6.5.1.4. SchNet. The model utilized interatomic distance as the invariant feature and used Gaussian radial basis expansion for smooth high-dimensional representation of distance. SchNet [74] introduced continuous convolution in the node update function. The learned continuous filter derived from the expanded Gaussian functions was used for continuous convolution with the node embeddings [74]. SchNet computed atomic forces using the gradient method. The core interaction layer in SchNet is visualized in Fig. 13d. Initially, the neighboring node features and RBF-expanded edge features are passed through a neural network. After transformation, each neighboring node and the corresponding edge features are multiplied elementwise. This operation constitutes the interaction part. The current node features and interaction terms are then summed to update the node as shown in Fig. 13 d.

6.5.1.5. PhysNet. PhysNet [199] also used interatomic distances as the invariant geometric feature. Similar to SchNet [74], PhysNet used Gaussian RBF for distance expansion and continuous filters were obtained based on interatomic distances. In addition, both models used gradient-based approach for computing the atomic forces after predicting the energy. The major difference between PhysNet and SchNet is that the latter incorporates long-range interactions such as electrostatic forces for predicting the energy.

6.5.1.6. MegNet. MegNet [71] used Gaussian expanded interatomic distance along with other auxiliary information. In addition to node-level features, edge and graph-level features were initialized and updated. MegNet outperformed SchNet [74] for several properties in the QM9 dataset [10].

6.5.2. Distance and angle-based models

6.5.2.7. DimeNet. Models that primarily rely on pairwise interatomic distances overlook important geometric features, such as bond angles, that could be essential for accurate property prediction. For instance, models like SchNet are unable to differentiate between atomic systems that have identical atoms and interatomic distances but different bond angles. To address this issue, DimeNet [72] introduced a message-passing framework that incorporates angular information along with interatomic distances. DimeNet [72] used Bessel basis functions for distance expansion and 2D spherical Fourier-Bessel functions for jointly expanding both distance and triplet angles. The potential energy was predicted directly from input and atomic forces were calculated using the gradient method; it achieved higher accuracy than distance-based models for both energy and force predictions. Fig. 13 e shows node update complexity in DimeNet [72]. For a given triplet $i-j-k$, message calculation requires the distance $i-j$, $j-k$ and the triplet angle $\angle ijk$. Similarly, three triplet information is required to calculate the message m_{ij} in Fig. 13 e. For a given node i , similar messages need to be calculated for all neighboring nodes $j \in \mathcal{N}(i)$ and are aggregated to update node embeddings.

6.5.2.8. GemNet. GemNet [73] introduced a message-passing scheme that incorporates interactions between quadruplets of atoms, extending beyond the triplet-based interactions used in DimeNet [72] and the pairwise interactions used in SchNet [74]. Since DimeNet only

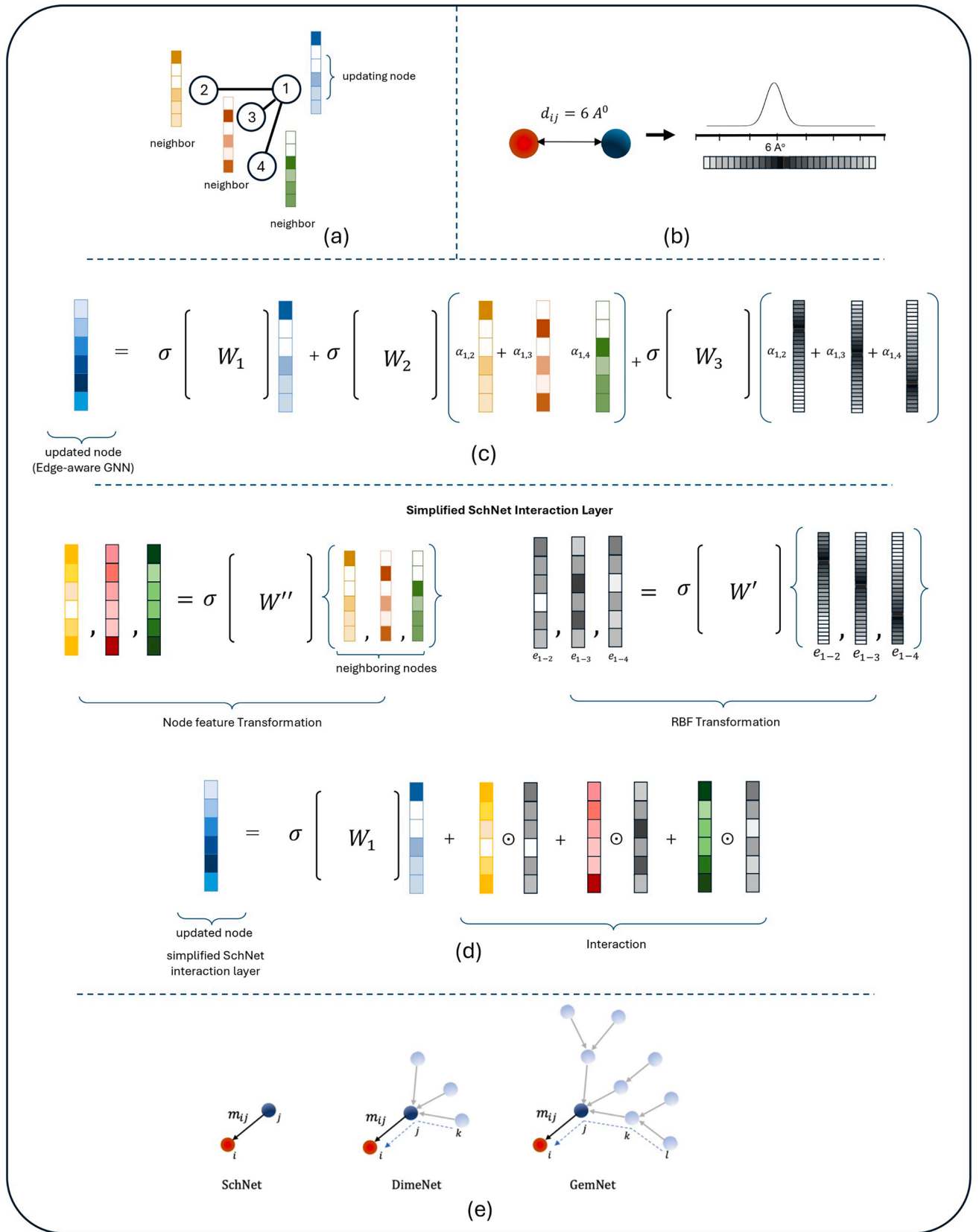


Fig. 13. Illustrations of edge-aware GNN, SchNet and message passing complexity in invariant models (a) atomic neighborhood in focus (b) RBF-expanded inter-atomic distance (c) node update function in TransformerConv (d) Simplified node update function in SchNet interaction layer (e) message passing complexity in SchNet, DimeNet and GemNet.

incorporates triplet angles, it cannot distinguish between molecules having the same interatomic distance and triplet angles, but different dihedral angles. To address this issue, GemNet considers a quadruplet of atoms, and calculates messages based on the interatomic distances, triplet angles and the dihedral angle formed by the quadruplet. GemNet outperformed distance-based models and DimeNet for several datasets. Fig. 13 e shows the information required to calculate a single message in GemNet. For a given quadruplet $i-j-k-l$, message calculation requires the distances $i-j$, $j-k$, $k-l$ and the triplet angles $\angle ijk$, $\angle jkl$ and the dihedral angle $\angle ijkl$. Similarly, 5 quadruplets are present in Fig. 13 e, and information from all 5 quadruplets are required to calculate the message m_{ij} . For a given node i , similar messages are calculated for all neighboring nodes $j \in N(i)$ and are aggregated to update node embeddings. As a result, GemNet [73] requires high computational cost compared to other models based on distance and triplet angles.

6.5.2.9. SphereNet. SphereNet [209] introduced spherical message passing to circumvent the higher computational cost associated with computing torsion. SphereNet demonstrated that the distance, angular and torsional information can be effectively encoded using spherical coordinates. This approach was able to capture torsion angle with reduced computational cost compared to GemNet's quadruplet method [73]. SphereNet achieved competitive performance with GemNet and outperformed DimeNet and previous distance-based models on several datasets.

6.5.2.10. ComENet. ComENet [210] incorporated rotation angles in addition to the two structural angles used in SphereNet. The rotation angle information τ_{ij} for a given edge $i-j$ was calculated using two reference planes. The model was able to identify molecular conformers using the additional rotation angle parameter with very high computational efficiency.

6.6. Geometric equivariant models

Geometric invariant GNNs use fixed parameters such as interatomic distance or angles that are not sensitive to rotation or translation of the molecule [72–74,198,209]. In contrast, equivariant graph neural networks (EGNNs) incorporate internal representations that transform consistently with the input. The concept of equivariance and steerability in neural networks was originally introduced by Cohen and Welling [211,212] in their works on G-CNN and Steerable CNNs. The concept was extended to 3D GNNs by Thomas et al. [108] through the introduction of Tensor Field Networks [108], which utilized spherical harmonics and tensor products to preserve equivariance. A similar approach was followed by models such as Cormorant [213], NequIP [180], MACE [214], SEGNN [200], Allegro [215], Equiformer [203], and they are often collectively referred to as *e3nn* models [201,216]. The major goal of EGNNs is to create learnable internal representations that vary predictably under rotation or, more generally, under Euclidean transformations.

6.6.1. Equivariant GNNs using spherical harmonics

In atomic systems, if the molecule is rotated, translated, or undergoes any Euclidean transformation, the atomic forces must transform accordingly. Equivariance to translation can be easily incorporated into the model by using the relative displacement vector $\vec{r}_i - \vec{r}_j$ instead of absolute position vectors [108]. 3D GNN models for atomic systems mainly focus on making the model equivariant to rotation, i.e. SO(3) equivariance [201]. SO(3) refers to the group of all rotations in 3D space preserving Euclidean symmetry. Commonly used Euclidean symmetry groups are mentioned in supporting information. Several geometric GNNs [108,201,213,215,217,218] use spherical tensors for creating internal representations that transform predictably under the rotation of the input. The spherical tensors transform under rotation in the same

way as a function defined on a sphere, and any spherical function can be represented by a linear combination of spherical harmonics basis functions (Fig. 14 a). For a given rotation $\mathbf{R}$, there exists a unique Wigner-D-matrix [219] ($\mathcal{D}(\mathbf{R})$) that transforms the spherical harmonics basis functions, and by extension, the spherical function (Fig. 14 b). As a result, spherical tensors also transform according to the Wigner-D-matrix (Fig. 15 a). For a spherical tensor of degree l , the $\mathcal{D}(\mathbf{R})$ matrix only depends on the rotation $\mathbf{R}$ and the chosen basis for representation space (e.g. spherical harmonics) [19], and is independent of the specific values inside the spherical tensor.

A spherical tensor can contain different types of components, and each type transforms independently of one another under rotation. These different types of components are called the irreducible representations or *irreps*. The irreducible representations (*irreps*) are indexed by non-negative integers $l = 0, 1, 2, \dots$ and each *irrep* of rank l includes $2l + 1$ elements numbered from $-l$ to l , termed as m . For example, if the *irreps* of a spherical tensor include the types $l = 0, 1, 2$, then each type will contain 1, 3, 5 elements, respectively (Fig. 15 a). The transformation behavior of each type in a spherical tensor under rotation is different. For example, for a 3D rotation of $\mathbf{R}$, the $l = 0$ component is a scalar and it doesn't change i.e. $\mathcal{D}^0(\mathbf{R}) = 1$. The type $l = 1$ represents a 3D vector; it will rotate like any vector in 3D space, so, $\mathcal{D}^1(\mathbf{R}) = \mathbf{R}$. However, the rotational behavior of the type $l = 2$ is not similar to that of a scalar or a 3D vector, and it is given by $\mathcal{D}^2(\mathbf{R})$ in the Wigner D-Matrix (Fig. 15 a). The inputs to the 3D GNN models are typically the atom types and their coordinates [202]. Fig. 15 b shows how scalar features like atom type is converted into spherical tensors with 6 channels for each type. The embedding layer is a linear transformation, and the new types formed ($l = 1, 2$) at the end are typically initialized at 0 [201,213,218]. The channels and types in node embedding tensors are a design choice, and it need not contain equal channels for each type [216]. For example, it can be 3 scalar channels ($l = 0$), 5 vector channels ($l = 1$), and 3 channels of type $l = 2$. However, types $l > 2$ were avoided in earlier models to reduce computational cost [19]. Fig. 15 c shows the relative direction vector $\vec{r}_{ij}$ encoded as spherical tensors using the real-valued spherical harmonics basis functions Y_l^m of degree $l = 2$. Since spherical harmonics (SH) are defined only on the unit sphere, the unit vector $\frac{\vec{r}_{ij}}{\|\vec{r}_{ij}\|}$ is used in real

spherical harmonics equations to get SH basis values. The real SH basis functions based on cartesian coordinates are given as Equation S.1 in supporting information. Now, the node embeddings and directional embeddings are constructed as spherical tensors. The core operation in equivariant GNN models is the message calculation step that uses tensor product for combining node and directional information [201], without breaking equivariance. A full tensor product between two spherical tensors couples each *irrep* l_1 present in the first tensor with each *irrep* l_2 present in the second tensor. The output from the tensor product may contain reducible and irreducible types of tensors. Since the Wigner D-Matrix is defined only for irreducible representations of SO(3), the tensor product output is converted into irreducible representations using Clebsch-Gordan (CG) coefficients [220]. The CG decomposition is given by,

$$l_1 \otimes l_2 = |l_1 - l_2| \oplus |l_1 - l_2| + 1 \oplus |l_1 - l_2| + 2 \oplus \dots \oplus (l_1 + l_2 - 1) \oplus (l_1 + l_2) \quad (14)$$

For example, tensor product between the *irreps* $l_1 = 1$ and $l_2 = 2$ is decomposed as $1 \otimes 2 = 1 \oplus 2 \oplus 3$. Different examples of CG decomposition are shown in Fig. 16 a. Using the CG rule, the coupling of $0 \otimes l$, $1 \otimes l$ and $2 \otimes l$ during the full tensor product is shown in Fig. 16 b-d. The output of the full tensor product, with the resulting *irreps* grouped by type, is shown in Fig. 16 e. As shown in Fig. 16 e, the output can contain higher-order *irreps* $l > 2$, however, these components are typically omitted in most *e3nn* models. A simple example of tensor product calculation and parametrization is given in Duval et al. [19] The node update used in *e3nn* library [221] is defined as follows,

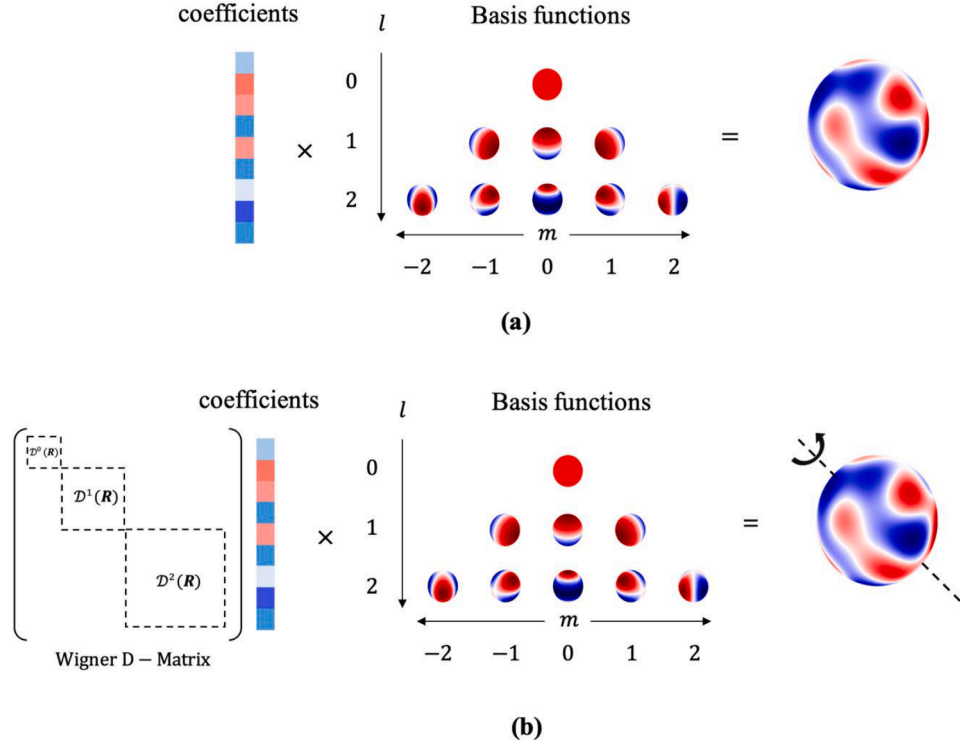


Fig. 14. (a) Spherical function approximated as sum product of coefficients and spherical harmonics basis functions (b) The transformation of spherical functions under rotation is governed by Wigner D-Matrix $\mathcal{D}^l(R)$ when the coordinate system is transformed with R. The rotation axis on the sphere is for illustration purposes only. This figure is adapted from an explanatory video by Zitnick et al [218], which showcases the architecture of the spherical channels network model.

$$f_i = \frac{1}{\sqrt{z}} \sum_{j \in \mathcal{N}(i)} f_j \otimes \left(h \left(\left\| \vec{r}_{ij} \right\| \right) \right) Y \left(\frac{\vec{r}_{ij}}{\left\| \vec{r}_{ij} \right\|} \right) \quad (15)$$

- f_j, f_i are the nodes input and output
- z is the average degree of the nodes
- $\mathcal{N}(i)$ is the set of neighbors of the node i
- $\vec{r}_{ij}$ is the relative vector
- h is a multi layer perceptron
- Y is the spherical harmonics
- $x \otimes (w)y$ is a tensor product of x with y parametrized by weights w

$h \left(\left\| \vec{r}_{ij} \right\| \right)$ is the learned embeddings linearly transformed from the

RBF expanded interatomic distance. $h \left(\left\| \vec{r}_{ij} \right\| \right)$ embeddings are used as weights in the tensor product for parametrizing the model. The transformation behavior of the node's spherical tensor, the directional spherical tensor, and the output of their tensor product under a rotation R is governed by a single Wigner D-matrix, as long as all tensors are represented by irreducible representations. After the tensor product, another key operation is channel mixing and gated activation. Channel mixing is done by applying linear transformation separately for each irrep type as shown in Fig. 16 f. However, non-linear activation cannot be applied to each irreps as it would break equivariance. To overcome this, gated activation is used. As shown in Fig. 16 f, non-linear activation is applied to scalar channels, and each element in an irrep type is multiplied by the same gate value. After multiple channel mixing, gated activation and other equivariant operations, the final potential energy of the atom is predicted by applying regression head on the scalar ($l=0$) channels (Fig. 16 g). The atomic forces are typically calculated using the gradient method [108,180,202,213], or it can also be computed directly

as the model output [202]. The above outlined model construction is a high-level overview of strictly equivariant EGNN models based on spherical harmonics, and the actual architecture differs in each model [108,200,202,205,213,215,217]. For example, in Allegro [215], spherical tensors are initialized from two-body embeddings derived from both atom types and directional vectors, and the tensor product is carried out between spherical tensors formed from two-body embeddings. In addition to the tensor update, the scalar embeddings were updated in a separate channel. In cormorant [213], two-body interaction is captured by forming spherical tensors from directional vectors only, and one-body interaction by forming spherical tensors from embeddings of atom types (Fig. 15 b, 15 c). Two types of tensor products are used in cormorant, one between one-body spherical tensors of the same atom for propagating information from the previous layer to the current, and the other between two-body spherical tensors for capturing interactions between pairs of atoms [213]. In SCN [218], tensor product is entirely avoided, but the model achieved higher performance in OC20 dataset [21] than all models before SCN. SCN used spherical harmonics for constructing spherical channels (functions), and 1×1 convolutions were performed after projecting spherical channels to 2D grid [218]. However, SCN does not strictly enforce equivariance but learns it indirectly through spherical channels [218]. In Equiformer [203], spherical tensors were initialized from two-body embeddings, and scalar ($l=0$) channels of each neighboring spherical tensors were utilized in a separate part to incorporate attention mechanism in message passing. In eSCN [201], the tensor product (SO(3) convolutions) was reduced to mathematically equivalent SO(2) convolutions, significantly decreasing computational cost. The SO(2) convolution approach was subsequently used in EquiformerV2 [107] and recently in eSEN [197]. EquiformerV2 [107] and eSEN [197] achieved state-of-the-art performance on several benchmark datasets [21,22,24].

6.6.2. Equivariant GNNs using cartesian tensors

EGNNs using irreducible representations and tensor products come

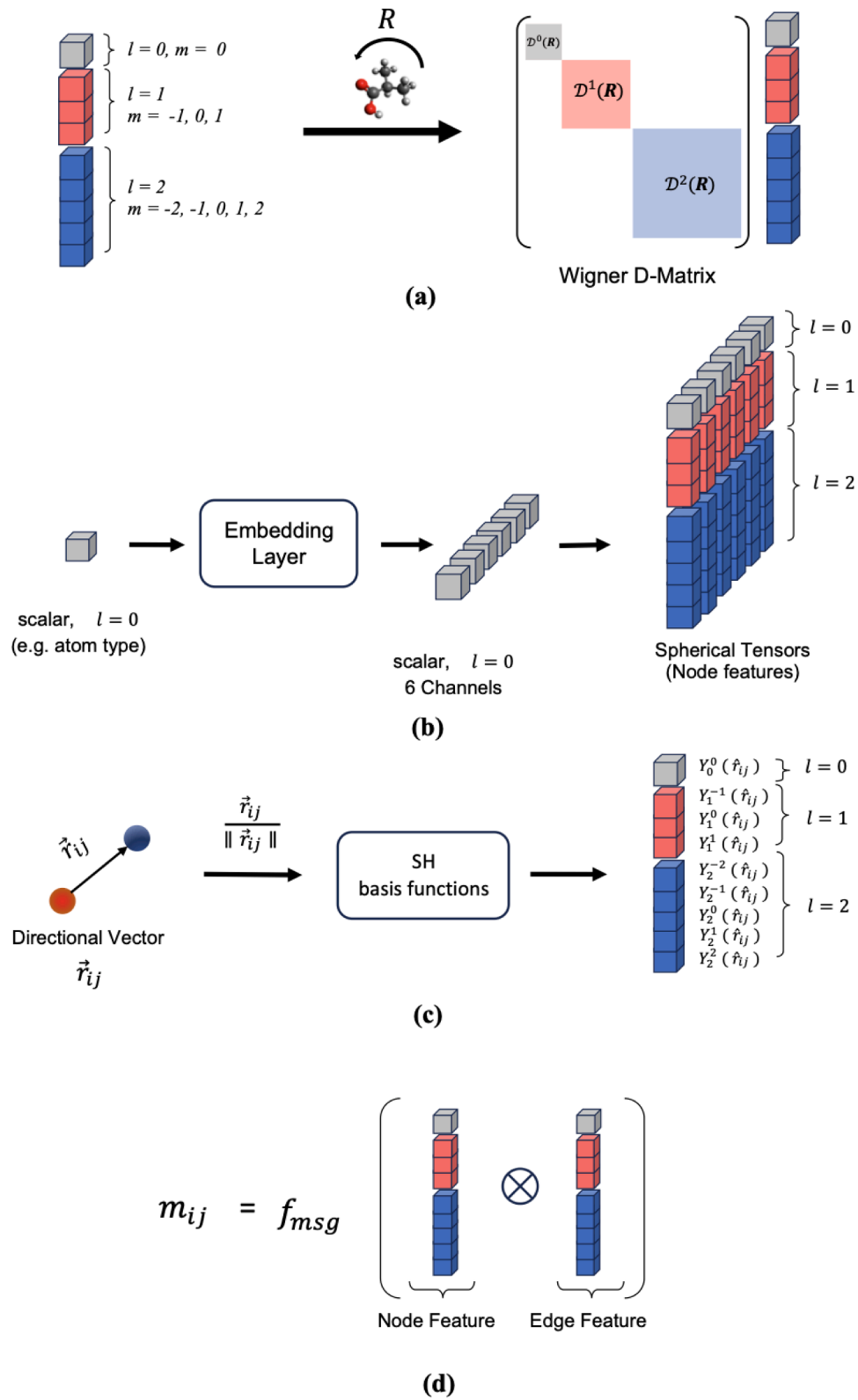


Fig. 15. (a) Transformation of a spherical tensor of degree $l = 2$ under rotation R given by the Wigner D-Matrix (b) Scalar features converted into spherical tensors. Embedding layers are linear transformations and elements in $l = 1, 2, \dots$ are typically initialized to 0 (c) Relative position vectors encoded as spherical tensors using spherical harmonics basis functions (d) Tensor product operation between node and edge spherical vectors for calculating message m_{ij} between nodes i and j . The tensor product operation varies depending on the architecture and need not necessarily be between node and edge features.

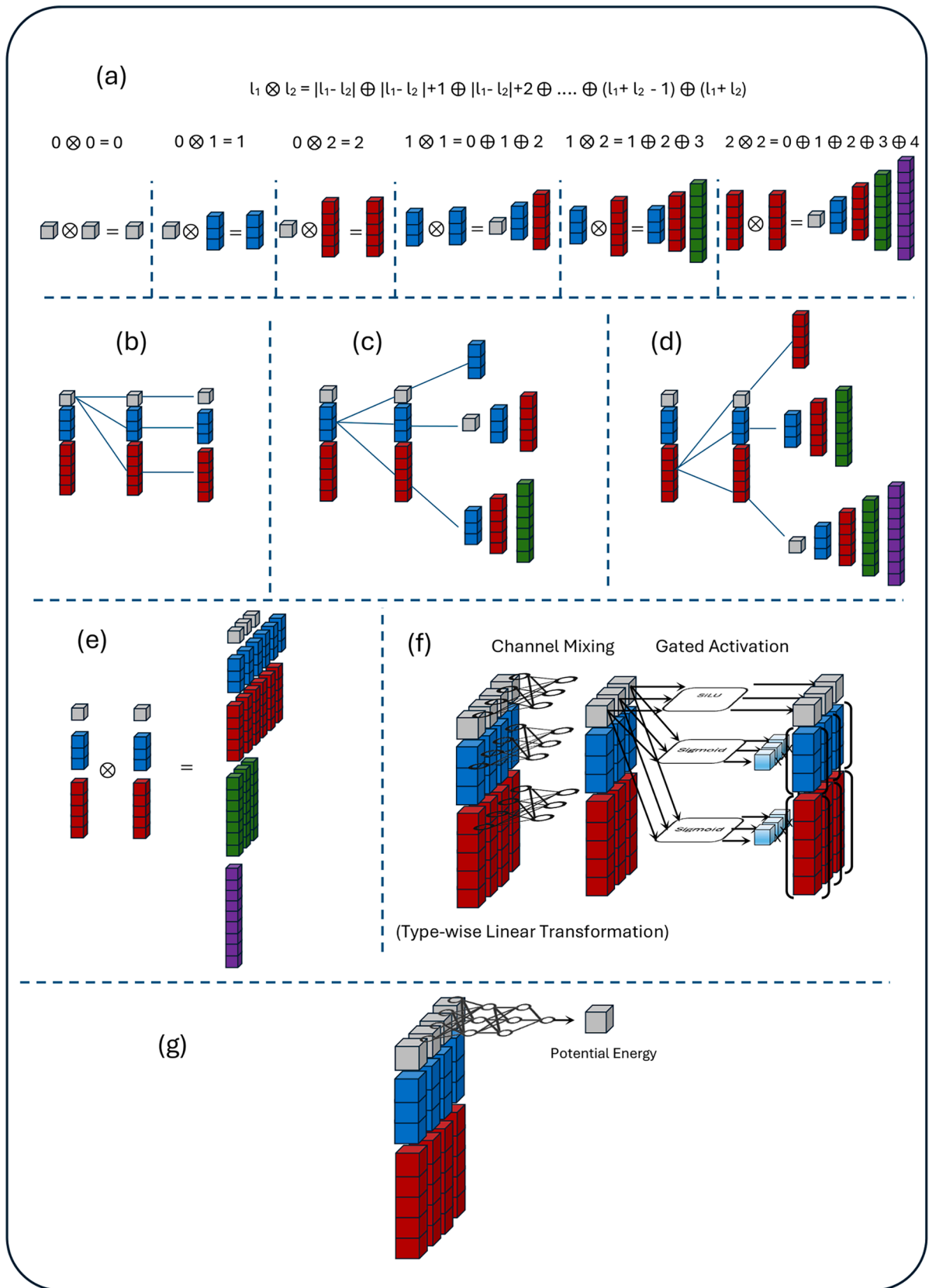


Fig. 16. Tensor operations in equivariant GNNs (a) Clebsch–Gordan tensor product decomposition rule (b) Tensor product decomposition of $0 \otimes l$ (c) Tensor product decomposition of $1 \otimes l$ (d) Tensor product decomposition of $2 \otimes l$ (e) full tensor product decomposition of $\oplus_{l_1=0}^2 \oplus_{l_2=0}^2 (l_1 \otimes l_2)$ (f) channel mixing with type-wise linear transformation and gated activation. SiLU and Sigmoid functions are selected based on Equiformer [203] implementation (g) Regression head applied on scalar ($l=0$) channels for final potential energy prediction.

at the cost of high computational expense. To address this, cartesian tensors such as scalars (rank 0), vectors (rank 1) or higher-order tensors (rank ≥ 2) can also be used as internal representations in equivariant models since their transformations are predictable. Under a rotation R , a scalar quantity such as atomic mass remains unchanged; a vector, such as atomic force $\mathbf{v}$, transforms as $\mathbf{v} \mapsto R\mathbf{v}$; a rank 2 tensor such as the shear stress acting on a cube, transforms as $\mathbf{M} \mapsto R\mathbf{M}R^T$. Similarly, the transformation behavior of higher order cartesian tensors are known for a given transformation R . Building on this, Schutt et al. introduced PaiNN, an equivariant message passing model using only scalars and vectors [207]. In addition to the atom types $Z_i \in \mathbb{N}$ and coordinates of atoms $\vec{r}_i \in \mathbb{R}^3$, PaiNN initializes a vector feature $\vec{v}_i^0 = \vec{0} \in \mathbb{R}^{F \times 3}$, where F denotes the number of nodes. The model uses only those operations that transform equivariantly with respect to the input. For example, scaling of a vector ($s \cdot \mathbf{v}$) can be used since the scaled vector transforms predictably with respect to the transformation of $\mathbf{v}$. Moreover, direct non-linearities can only be applied to scalar features and only linear operations or gated nonlinearities are allowed on a vector [207]. A few general examples of allowed operations are listed in Schutt et al. [207]. Similar approach was followed in TorchMD-NET [206], where they initialized a vector feature along with other scalars and the node coordinates. In addition, an attention mechanism was used for processing scalar node features and interatomic distance. Similarly, models such as E(n) GNN [222] and NewtonNet [223] uses only scalars and vector operations. The equivariant models using only scalar and vectors are relatively simple and more computationally efficient. However, higher order tensors of rank ≥ 2 can also be used for encoding more expressive internal representations. Examples of such models include TeaNet [224], TensorNet [225] and HotPP [204]. Summary of invariant and equivariant geometric GNNs is given in Table 5.

6.7. Key insights and limitations

Geometric graph neural networks achieved state-of-the-art performance across several benchmarks on quantum-mechanical property prediction. Their key advantage is that they are designed to capture a physically faithful representation of atomistic systems, which naturally exist as point clouds embedded in 3D space. This enables the model to be used as surrogates for density functional theory (DFT) calculations, molecular dynamics and phonon vibration analysis. While these models sit at the top of common MLIP benchmarks, finding the right balance between expressiveness and scalability is still being actively explored. Earlier geometric GNNs that relied on spherical harmonics with full SO (3) convolutions provided strong angular expressiveness but suffered from poor scalability. Storing highly expressive representations require the usage of higher order tensors ($L \geq 2$). In practice, most e3nn-based networks were limited to computing only low angular orders, typically $L \leq 2$ or 3, as the cost of full tensor product scales at the order of $O(L^6)$. While the use of cartesian tensors maintains equivariance and provides faster performance, their expressiveness is not par with models utilizing spherical tensors. In contrast, the SO(2) convolution approach introduced in Passaro et al. [201] retains richer angular information while reducing the tensor-product complexity to $O(L^3)$. As a result, models based on SO(2) convolution currently find the right balance of expressiveness and scalability. A summary of expressiveness, accuracy and scalability of geometric GNNs is provided in Table 6.

Apart from scalability issues, the reliance on 3D atomic coordinates can also be a concern when dealing with completely unknown molecules or materials whose stable structures have not yet been determined. Although several geometric GNNs are designed for structural relaxation to obtain a ground-state configuration, the process is highly sensitive to the initial geometry. If the starting coordinates are far from the true minimum-energy arrangement, the relaxation may fail to converge or may settle into an unphysical structure. To mitigate this issue, a preliminary relaxation using faster methods, such as xTB [228], may be

Table 5

Geometric graph neural network for quantum mechanical property prediction.

Model	Datasets	Message Passing	Year
DTNN [198]	GDB-9, GDB-7, MD trajectories	Invariant – Distance	2017
SchNet [74]	QM9, MD17, ISO17	Invariant – Distance	2017
CGCNN [208]	Materials Project	Invariant – Distance	2018
PhysNet [199]	QM9, MD17, ISO17, S_N2 reactions	Invariant – Distance	2019
MEGNet [71]	QM9, materials project crystals	Invariant – Distance	2019
DimeNet [72]	QM9, MD17	Invariant – Distance, triplet angle	2020
GemNet [73]	COLL, MD17, OC20	Invariant – Distance, angles, torsion	2021
SphereNet [209]	OC20, QM9, MD17	Invariant – Distance, angles, torsion	2022
ComENet [210]	OC20, Molecule3D, QM9	Invariant – Distance, angles, torsion	2022
TFN [108]	QM9, MD17, ModelNet40	Equivariant – Spherical Harmonics	2018
Cormorant [213]	QM9, MD17	Equivariant – Spherical Harmonics	2019
SE(3) Transformer [205]	ScanObjectNN, QM9, N-body system	Equivariant – Spherical Harmonics	2020
E(n) GNN [222]	QM9, charged N-body dataset, etc.	Equivariant – Scalars and vectors	2021
PaiNN [207]	QM9, MD17	Equivariant – Scalars and vectors	2021
NewtonNet [223]	MD17, CCSD(T), ANI-1, etc.	Equivariant – Scalars and vectors	2022
SCN [218]	OC20	Not strictly Equivariant – Spherical Harmonics	2022
NequIP [180]	MD17, CCSD(T), AIMD simulations (water and ice, $\text{Li}_4\text{P}_2\text{O}_7$, etc.)	Equivariant – Spherical Harmonics	2022
SEGNN [202]	QM9, OC20, N-body system	Equivariant – Spherical Harmonics	2022
MACE [214]	rMD17, 3BPA, and AcAc	Equivariant – Spherical Harmonics	2022
TorchMD-NET [206]	MD17, ANI-1, QM9	Equivariant – Scalars and vectors	2022
TeaNet [224]	Custom dataset, QM9, Materials project	Equivariant – Higher order cartesian tensors	2022
Equiformer [203]	QM9, MD17, OC20	Equivariant – Spherical Harmonics	2023
eSCN [201]	OC20, OC22	Equivariant – Spherical Harmonics	2023
Allegro [215]	rMD-17, 3BPA, QM9, AIMD trajectories (Li_3PO_4 and Ag)	Equivariant – Spherical Harmonics	2023
EquiformerV2 [107]	QM9, OC20, OC22	Equivariant – Spherical Harmonics	2023
TensorNet [225]	QM9, rMD17, SPICE, ANI1x, COMP6, etc.	Equivariant – Higher order cartesian tensors	2023
HotPP [204]	ANI-1x, GAP17/20, Water systems, COMP6	Equivariant – Higher order cartesian tensors	2024
VisNet [226]	MD17, MD22, Molecule3D, QM9, etc.	Equivariant – Scalars and vectors	2024
SevenNet [227]	M3GNet dataset	Equivariant – Spherical Harmonics	2024
eSEN [197]	MPTrj, Matbench discovery, MDR phonon, SPICE-MACE-OFF	Equivariant – Spherical Harmonics	2025
UMA [243]	OMat24, OMol25, OC20, OMC25, and ODAC25	Equivariant – Spherical Harmonics	2025

required before applying MLIP-based structural relaxation.

7. Engineering perspectives and deployment considerations

The ultimate aim of computational material and molecular design is

Table 6
Advantages and limitations of geometric GNNs.

Model class	Features used	Advantages	Limitations
Invariant	Distance (e.g. SchNet)	Fastest; highly scalable	Low expressivity; cannot capture angles or conformers
	Distance + triplet angle (DimeNet)	Captures 3-body geometry; higher accuracy than distance-only models	Cannot capture torsion
	Distance + triplet + dihedral angle (GemNet)	Captures 4-body geometry; higher accuracy than distance + triplet angle models	Poor scalability, each message requires edge-based 2-hop information
	Distances + spherical coordinates (polar & azimuthal) (SphereNet)	Captures torsion with more efficiency	Less expressive than models explicitly handling 4-body geometry
	Distances + local reference plane rotation angle (ComENet)	Highest efficiency in capturing torsion	Less expressive than models explicitly handling 4-body geometry
Equivariant	Scalars, vectors, tensors in Cartesian space	Simple & scalable implementation of equivariance	Lower angular expressivity than spherical-harmonic models
	Spherical tensors + full SO(3) convolution	High expressivity	Poor scalability, full tensor product scales as $O(L^6)$ with rank L
	Spherical tensors + SO(2) convolution	High practical expressivity by enabling higher L; good scalability (Tensor product scales as $O(L^3)$)	Requires careful local-frame handling
	Spherical tensors + full SO(3) convolution with FlashTP [229]	Uses CG sparsity to cut cost while preserving full SO(3) expressivity	Still more expensive than low-rank cartesian equivariant models

to synthesis the right molecule or material that exhibit desired properties. The design space for this task is vast and synthesizing each of them is infeasible, as there are innumerable possible candidates. From the engineering perspective, we identify the following areas where predictive AI models can provide significant practical value.

1. Accelerating DFT + Experiment pipeline

Simulation tools such as density functional theory (DFT) are used to provide theoretical estimates and guide experiments toward more suitable candidates. However, DFT is computationally expensive and practically infeasible for screening millions of molecules. Here, a reliable machine learning model can be used as a faster alternative, as it can predict and screen millions of candidates in a matter of minutes or hours. A complete ML + DFT + Experiment pipeline is shown in Fig. 17 a. Initially, millions of hypothetical candidates are virtually constructed. The trained ML model predicts their properties and screen only those with desired properties. This ML-backed screening allows rapid prioritization and cuts the cost and accelerates the decision-making process. The small screened subset needs to be validated with DFT for stability and synthesizability checks, as ML models can potentially identify molecules or materials that may not be realizable in the real world. This stage can also be combined with expert analysis to check the safety and practical feasibility of the DFT-identified candidates. Finally, the selected candidates need to be synthesized. Based on the DFT calculations, expert analysis, and experimental results, the AI model can be further improved to produce increasingly accurate predictions. Overall,

this ML-backed approach is significantly more efficient and faster than traditional trial-and-error methods or simulation-driven workflows combined with experiments.

2. AI-driven navigation of the design space

A trained neural network creates a continuous function that links the input to the target property (Fig. 17 b). By optimizing the output with respect to the input, we can identify the set of descriptors or molecular structures that best match the optimal target. For example, Sekine et al. [244] predicted electrode properties such as discharge capacity and discharge voltage of sodium-ion cathodes using composition-based descriptors, and subsequently optimized the energy density (capacity $\times$ voltage) by varying the composition. This approach enabled the identification of compositions that maximize energy density. The continuous function generated by the model serves as an efficient tool to navigate the design space and helps to quickly identify the candidates with optimum properties. This function also serves as the foundation for generative AI models for drug design and material discovery, however, a detailed discussion on generative models is beyond the scope of this study.

3. Self-supervised learning decodes properties of unlabeled data

Real-world data often come with little or no information about the properties of the materials or molecules being studied. While characterization techniques can reveal information to a good extent, highly noisy data remain difficult to decode. In this context, unsupervised or self-supervised learning provides powerful tools to extract meaningful information from completely unknown drug compounds or materials. In unsupervised training, each candidate is projected into a high-dimensional embedding space using only its input representation, such that similar molecules are mapped close together and dissimilar molecules farther apart. In Fig. 17 c, 2D visualization of the embedding space shows that molecule B is more likely to be similar to molecule A than molecule C based on proximity. This property is particularly useful for analyzing unknown molecules; for example, a compound located close to aspirin in the embedding space is more likely to exhibit properties similar to aspirin. A significant example leveraging this capability was introduced by Roman et al. [245] in the DreaMS model. DreaMS is trained on unlabeled tandem mass spectra mined from the MassIVE GNPS repository using a self-supervised transformer-based architecture. Although more than 90 % of the input spectra lack explicit annotations, the model learns embeddings that cluster spectra according to underlying molecular structure, chemical composition, and sample origin. It revealed molecular-level groupings as well as higher-level chemical groupings, such as plant-based, animal-based, and beverage-related compounds.

7.1. Model selection

The selection of molecular representations and model architectures is highly problem-specific and is often determined through trial-and-error experiments using different input types and model designs. However, several high-level considerations can help streamline this process. An overview of practical guidelines is provided in Fig. 18. As shown in Fig. 18, classical ML models are generally well suited for scenarios with limited data availability and computational resources. Sequence models are effective for predicting global properties from string or sequential representations. When large-scale datasets and GPU resources are available, pretraining–finetuning frameworks based on transformer architectures are particularly powerful for unlabeled data clustering and global property prediction. In contrast, when target properties are strongly influenced by local neighborhood interactions and molecular geometry, GNNs are typically the most suitable choice. In particular, when atomic forces and energies are the targets, geometric GNNs, and

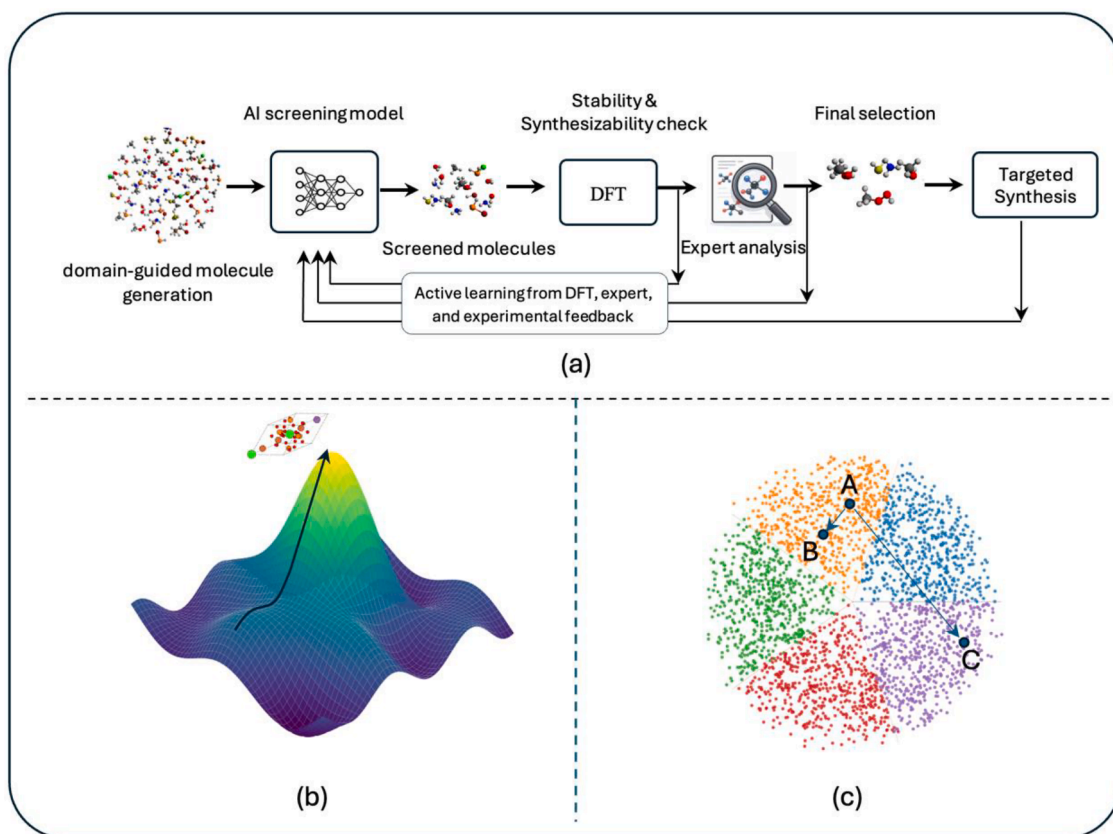


Fig. 17. Practical roles of AI models in molecular and materials design workflows. (a) AI-based screening accelerates the DFT + experiment workflow (b) Optimizing the neural network function for navigating the design space. (c) Self-supervised learning decodes information from unknown molecules.

more specifically equivariant GNNs, are the most appropriate models.

While there is no strict definition for low, medium, and large data regimes in molecular machine learning, datasets with fewer than 1000 samples are typically considered small, those in the range of 10^3 – 10^4 as medium, and those exceeding 10^4 as large. However, for large-scale transformer pretraining, datasets on the order of $\sim 10^4$ samples are still relatively small, as many modern models are trained on datasets comprising millions of data points (Table 3). In addition to the practices shown in Fig. 18, multi-modal approaches that incorporate multiple input types and model classes are gaining increasing attention. A brief discussion of these approaches is provided in the future outlook section.

8. Conclusion & outlook

Machine learning approaches are rapidly emerging as powerful tools for accelerating the discovery of new materials and drugs. The ability of machine learning models to extract information from molecular structure can be used as an effective tool to scan the chemical space in search of molecules and materials with desirable properties. The key findings of our study are summarized as follows,

- Descriptor-based models remain highly effective, especially for small or well-characterized datasets, and can outperform deep models when high-quality descriptors are available. However, their broader applicability is limited by poor generalizability and the lack of reliable descriptors for novel or previously uncharacterized molecules and materials.
- String-based representations (e.g., SMILES) enable modeling with sequential architectures such as RNNs and Transformers. By taking advantage of the scalability of Transformers, highly accurate models have been built using simple molecular representations. However, while transformers are powerful, their ultimate performance is

constrained by the limited expressivity of compact string representations, which do not capture the spatial arrangement of atoms in 3D space.

- Graph-based and geometric representations address these limitations by encoding structural connectivity directly. Message-passing neural networks, particularly 3D geometric GNNs, achieve state-of-the-art accuracy for quantum-mechanical properties by enforcing physical symmetries such as rotational invariance and equivariance.
- Despite their accuracy, geometric GNNs are computationally demanding, especially those relying on higher-order tensors or expensive equivariant operations, underscoring the need for models that balance physical fidelity with improved scalability and efficiency.
- From a modeling and practical deployment perspective, classical ML models are generally well suited for applications with limited data and computational resources. Sequence models are effective for predicting global properties from string/sequential representations. When large datasets and GPU resources are available, pre-training–finetuning pipelines using transformer architectures are particularly effective for unlabeled data clustering and global property prediction. In contrast, when the target property depends strongly on local neighborhood structure and molecular geometry, GNNs are typically the most appropriate choice.
- In modern molecular ML, different representations, architectures, and training methods are used interchangeably across different modelling paradigms. The examples include, usage of classical ML models to neural embeddings generated by sequence models or GNNs [76]. Transformer encoder for processing molecular graph, as demonstrated in Graphormer [230]. Masked Language Modeling (MLM) was adapted in molecular graphs to enable self-supervised pretraining, as shown in GROVER [231]. Sequence models such as LSTM [120] have been used as aggregation functions within GNNs,

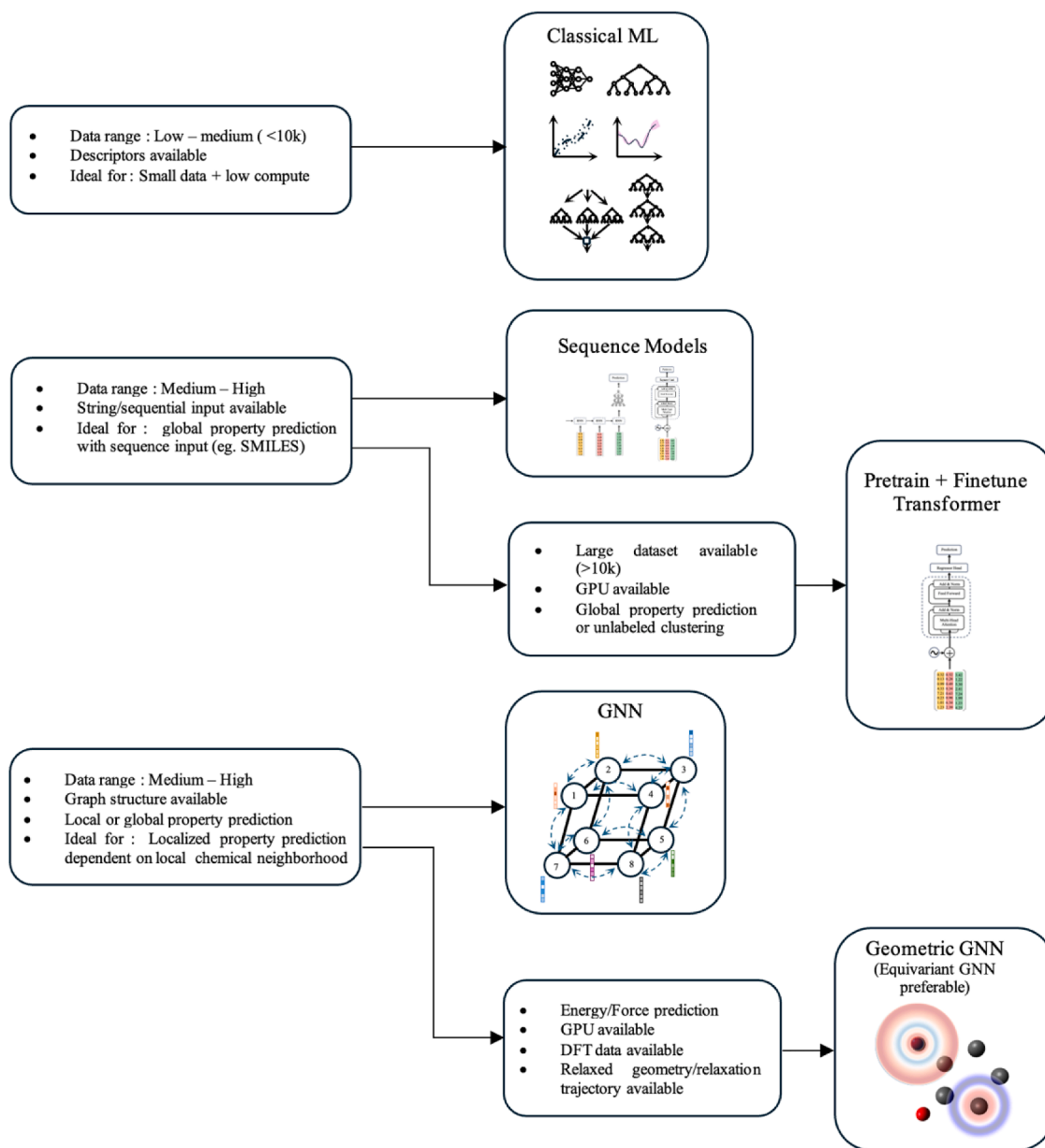


Fig. 18. High-level guidelines for selecting molecular representations and machine learning architectures based on data availability, computational resources, and target property characteristics.

as shown in GraphSAGE [192]. In models like CatBERTa [41] and LLM-Prop [162], textual descriptions of materials are used instead of compact string-based representations as inputs to transformer-based models

- From an engineering perspective, molecular machine learning primarily serves to accelerate molecular/materials discovery pipelines. Predictive AI models enable rapid screening of millions of candidates, and significantly reduces decision-making time and cost while prioritizing promising systems for downstream DFT calculations and experimental validation. Moreover, the continuous functions learned by neural networks enable efficient optimization over large design spaces and help identify optimal molecular or material candidates. In addition, for unlabeled real-world data, self-supervised learning organizes molecules or materials in high-dimensional embedding spaces and enables the discovery of new candidates based on their proximity to known high-performing drugs or materials.

Based on the current progress, we identify the following future

directions

1. 3D atomistic modelling using Transformers without graph priors

The well-established scalability and generalizability of Transformers motivate exploring their use in 3D molecular systems and MLIPs. Recent work by Kreiman et al. [232] shows that tokenizing discretized atomic positions, energies, and forces—paired with continuous coordinate embeddings allows Transformers to operate directly on 3D structures without predefined graphs, achieving performance competitive with equivariant geometric GNNs. This direction will enable MLIPs to make use of off-the-shelf transformer architectures, advanced efficiency algorithms such as latent or sparse attention, and to fully exploit modern GPU hardware optimized for dense matrix operations. However, more systematic benchmarking across datasets is needed to assess force accuracy, long-time MD stability, and the practical trade-offs relative to established equivariant GNN-based MLIPs.

2. Computational Efficiency of Equivariant Tensor Operations

The key bottleneck for scalability in equivariant geometric GNNs is the Clebsch-Gordan Tensor Product (CGTP). Recently, the FlashTP [229] algorithm showed a significant reduction in tensor product computational cost by taking advantage of the inherent sparsity in Clebsch-Gordan coefficient matrices. Currently, FlashTP [229] has only been applied to the SevenNet [227] model and it showed 4.2x and 3.5x speedup for inference and training, respectively. However, this method can be extended to models operating on SO(2) convolution, such as eSEN or equiformerV2, and it could lead to significant efficiency improvement. Another CGTP acceleration method, Fused Tensor Product (FTP) improves efficiency by fusing all irreducible representations (irreps) into a single dense tensor that can be handled using standard matrix multiplications [233]. Other than the CGTP, the Gaunt Tensor Product (GTP) offers a different strategy by transforming the representation into the frequency domain and using Fast Fourier Transforms to accelerate tensor-product computations [234]. While GTP is computationally attractive, it also sacrifices some expressivity compared to full CGTP.

3. Hybrid models

Models combining multiple representations and encoders are emerging as strong alternatives to single-representation models. Hybrid models incorporating graph+strings [235,236], descriptors+strings [237], descriptors+graphs [238] or descriptors+graphs+strings [239] are increasingly getting attention as they take advantage of expressivity from each molecular representation. In this direction, the adoption of equivariant GNNs combined with other representations remains relatively unexplored. While higher-order representations in equivariant GNNs are difficult to handle in such hybrid models, the scalar ($l = 0$) channels can be combined with other representations without breaking equivariance of the Geometric GNNs.

4. Physics-enhanced Equivariant GNNs

Equivariant geometric GNNs are predominantly used as MLIPs. Their adoption in broader application areas such as energy storage, thermo-physical property prediction or drug discovery, is still emerging. Based on the area of application, physics-based information can be added to scalar ($l = 0$) channels of spherical tensors. Although traditional equivariant models rely only on atomic numbers for initializing node features, incorporating domain-specific information to scalar channels can potentially improve performance without breaking equivariance.

5. Foundation Models Built on Modern Large-Scale Atomistic Datasets

Recent datasets such as OMol25 [240], OMat24 [241], and LeMat-Traj [242] provide unprecedented access to large, diverse, and high-fidelity quantum-chemical data for both molecules and materials. Their scale and breadth significantly lower the barrier to developing truly large pretraining datasets and open the possibility of learning universal atomic representations that transfer across chemical domains. Despite this progress, foundation models based on Transformers, GNNs, or hybrid architectures that are trained directly on these modern datasets remain scarce. Building such models represents a major opportunity for future work, with potential to create standardized, geometry-aware pretraining backbones that support a wide range of downstream tasks in molecular property prediction, materials discovery, and MLIPs.

Funding sources

This research was supported by Khalifa University of Science and Technology through Grant No. RC2-2018-024

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT for language refinement and Grammarly for grammar and readability improvements. The author reviewed and edited the content and take full responsibility for the final publication.

Abbreviations

AI, artificial intelligence; BART, bidirectional & autoregressive transformer; BERT, bidirectional encoder representations from transformers; BoB, bag of bonds; CGCNN, crystal graph convolution neural network; CL, contrastive learning; CLS, classification token; CM, coulomb matrix; DeBERTa, decoding-enhanced BERT with disentangled attention; DFT, density functional theory; DNA, deoxyribonucleic acid; DT, decision tree; DTNN, deep tensor neural network; ECFP, extended connectivity circular fingerprint; EGNN, equivariant graph neural network; EOS, end of sentence token; GAT, graph attention network; GB, gradient boosted trees; GCN, graph convolution network; GNN, graph neural networks; GPR, gaussian process regression; GPU, graphics processing units; GRU, gated recurrent unit; ID, identifier; IID, independent and identically distributed; InChI, international chemical identifier; IUPAC, international union of pure and applied chemistry; LLaMA, large language model meta AI; LLM, large language model; LSTM, long short-term memory; MACCS, molecular access system keys; MHA, multi-head attention; ML, machine learning; MLM, masked language modelling; MLP, multi-layer perceptron; MPNN, message passing neural network; NLP, natural language processing; PAD, padding token; PLS, partial least square; QM, quantum mechanical; RBF, radial basis function; RF, random forest; RNN, recurrent neural network; RoBERTa, Robustly Optimized BERT Pretraining Approach; SELFIES, self-referencing embedded strings; SH, spherical harmonics; SMILES, simplified molecular input line entry system; SOS, start of sentence token; SVM, support vector machines; UMA, universal model for atoms; UNK, unknown token; XGBoost, extreme gradient boosting

CRediT authorship contribution statement

Muhammed Thameem: Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Obaid AlHmoudi:** Writing – review & editing, Validation, Methodology, Conceptualization. **Ahmad Al Salloum:** Validation, Methodology. **Naeema Al Darmaki:** Validation, Supervision. **Ali Elkamel:** Writing – review & editing, Validation, Supervision, Conceptualization. **Ali A. AlHammadi:** Writing – review & editing, Validation, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors acknowledge the financial support from Khalifa University of Science and Technology.

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at [doi:10.1016/j.rineng.2026.109241](https://doi.org/10.1016/j.rineng.2026.109241).

Data availability

No primary research results, software or code have been included

and no new data were generated or analysed as part of this review.

References

- [1] W. Kohn, A.D. Becke, R.G. Parr, Density functional theory of electronic structure, *J. Phys. Chem.* 100 (1996) 12974–12980, <https://doi.org/10.1021/jp960669l>.
- [2] J. Schirmer, A. Dreuw, Critique of the foundations of time-dependent density-functional theory, *Phys. Rev. A (Coll. Park)* 75 (2007) 022513, <https://doi.org/10.1103/PhysRevA.75.022513>.
- [3] B.G. del Rio, B. Phan, R. Ramprasad, A deep learning framework to emulate density functional theory, *NPJ. Comput. Mater.* 9 (2023) 158, <https://doi.org/10.1038/s41524-023-01115-3>.
- [4] J. Vamathevan, D. Clark, P. Czodrowski, I. Dunham, E. Ferran, G. Lee, B. Li, A. Madabhushi, P. Shah, M. Spitzer, S. Zhao, Applications of machine learning in drug discovery and development, *Nat. Rev. Drug Discov.* 18 (2019) 463–477, <https://doi.org/10.1038/s41573-019-0024-5>.
- [5] C. Ling, A review of the recent progress in battery informatics, *NPJ. Comput. Mater.* 8 (2022) 33, <https://doi.org/10.1038/s41524-022-00713-x>.
- [6] W. Yang, T.T. Fidelis, W.-H. Sun, Machine learning in catalysis, from proposal to practicing, *ACS Omega (Westport)* 5 (2020) 83–88, <https://doi.org/10.1021/acsomega.9b03673>.
- [7] Y. Liu, T. Zhao, W. Ju, S. Shi, Materials discovery and design using machine learning, *J. Materiomics* 3 (2017) 159–177, <https://doi.org/10.1016/j.jmat.2017.08.002>.
- [8] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2 (1989) 359–366, [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- [9] L. David, A. Thakkar, R. Mercado, O. Engkvist, Molecular representations in AI-driven drug discovery: a review and practical guide, *J. Cheminform.* 12 (2020) 56, <https://doi.org/10.1186/s13321-020-00460-5>.
- [10] R. Ramakrishnan, P.O. Dral, M. Rupp, O.A. von Lilienfeld, Quantum chemistry structures and properties of 134 kilo molecules, *Sci. Data* 1 (2014) 140022, <https://doi.org/10.1038/sdata.2014.22>.
- [11] Z. Wu, B. Ramsundar, E.N. Feinberg, J. Gomes, C. Geniesse, A.S. Pappu, K. Leswing, V. Pande, MoleculeNet: a benchmark for molecular machine learning, *Chem. Sci.* 9 (2018) 513–530, <https://doi.org/10.1039/C7SC02664A>.
- [12] C.K. Joshi, C. Bodnar, S.V. Mathis, T. Cohen, P. Liò, On the expressive power of geometric graph neural networks, in: *Proceedings of the 40 Th International Conference on Machine Learning*, 2023.
- [13] B.M. Abraham, M.V. Jyothirmay, P. Sinha, F. Viñes, J.K. Singh, F. Illas, Catalysis in the digital age: unlocking the power of data with machine learning, *WIREs Comput. Mol. Sci.* 14 (2024), <https://doi.org/10.1002/wcms.1730>.
- [14] S. Dara, S. Dhamecherla, S.S. Jada, C.M. Babu, M.J. Ahsan, Machine learning in drug discovery: a review, *Artif. Intell. Rev.* 55 (2022) 1947–1999, <https://doi.org/10.1007/s10462-021-10058-4>.
- [15] Z. Shen, H. Liu, Y. Shen, J. Hu, L. Chen, C. Nan, Machine learning in energy storage materials, *Interdiscip. Materials (Basel)* 1 (2022) 175–195, <https://doi.org/10.1002/idm2.12020>.
- [16] L. David, A. Thakkar, R. Mercado, O. Engkvist, Molecular representations in AI-driven drug discovery: a review and practical guide, *J. Cheminform.* 12 (2020) 56, <https://doi.org/10.1186/s13321-020-00460-5>.
- [17] T. Gao, W. Lu, Machine learning toward advanced energy storage devices and systems, *iScience* 24 (2021) 101936, <https://doi.org/10.1016/j.isci.2020.101936>.
- [18] M.C. Ramos, C.J. Collison, A.D. White, A review of large language models and autonomous agents in chemistry, *Chem. Sci.* 16 (2025) 2514–2572, <https://doi.org/10.1039/D4SC03921A>.
- [19] A. Duval, S.V. Mathis, C.K. Joshi, V. Schmidt, S. Miret, F.D. Malliaros, T. Cohen, P. Liò, Y. Bengio, M. Bronstein, A Hitchhiker's guide to geometric GNNs for 3D atomic systems, (2023).
- [20] T. Liyaqat, T. Ahmad, C. Saxena, Advancements in molecular property prediction: a survey of single and multimodal approaches, *Arch. Comput. Methods Eng.* (2025), <https://doi.org/10.1007/s11831-025-10317-5>.
- [21] L. Chanussot, A. Das, S. Goyal, T. Lavril, M. Shuaibi, M. Riviere, K. Tran, J. Heras-Domingo, C. Ho, W. Hu, A. Palizhati, A. Sriram, B. Wood, J. Yoon, D. Parikh, C. L. Zitnick, Z. Ulissi, Open Catalyst 2020 (OC20) dataset and community challenges, *ACS. Catal.* 11 (2021) 6059–6072, <https://doi.org/10.1021/acscatal.0c04525>.
- [22] R. Tran, J. Lan, M. Shuaibi, B.M. Wood, S. Goyal, A. Das, J. Heras-Domingo, A. Kolluru, A. Rizvi, N. Shoghi, A. Sriram, F. Therrien, J. Abed, O. Voznyy, E. H. Sargent, Z. Ulissi, C.L. Zitnick, The open catalyst 2022 (OC22) dataset and challenges for oxide electrocatalysts, *ACS. Catal.* 13 (2023) 3066–3084, <https://doi.org/10.1021/acscatal.2c05426>.
- [23] S. Chmiela, A. Tkatchenko, H.E. Sauceda, I. Poltavsky, K.T. Schütt, K.-R. Müller, Machine learning of accurate energy-conserving molecular force fields, *Sci. Adv.* 3 (2017), <https://doi.org/10.1126/sciadv.1603015>.
- [24] A. Jain, S.P. Ong, G. Hautier, W. Chen, W.D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, K.A. Persson, Commentary: the materials project: a materials genome approach to accelerating materials innovation, *APL. Mater.* (2013) 1, <https://doi.org/10.1063/1.4812323>.
- [25] Z. Li, M. Jiang, S. Wang, S. Zhang, Deep learning methods for molecular representation and property prediction, *Drug Discov. Today* 27 (2022) 103373, <https://doi.org/10.1016/j.drudis.2022.103373>.
- [26] K. Atz, F. Grisoni, G. Schneider, Geometric deep learning on molecular representations, *Nat. Mach. Intell.* 3 (2021) 1023–1032, <https://doi.org/10.1038/s42256-021-00418-8>.
- [27] Z. Guo, K. Guo, B. Nan, Y. Tian, R.G. Iyer, Y. Ma, O. Wiest, X. Zhang, W. Wang, C. Zhang, N.V. Chawla, Graph-based molecular representation learning, (2022).
- [28] K. Hansen, F. Biegler, R. Ramakrishnan, W. Pronobis, O.A. von Lilienfeld, K.-R. Müller, A. Tkatchenko, Machine learning predictions of molecular properties: accurate many-body potentials and nonlocality in chemical space, *J. Phys. Chem. Lett.* 6 (2015) 2326–2331, <https://doi.org/10.1021/acs.jpclett.5b00831>.
- [29] M. Rupp, A. Tkatchenko, K.-R. Müller, O.A. von Lilienfeld, Fast and accurate modeling of molecular atomization energies with machine learning, *Phys. Rev. Lett.* 108 (2012) 058301, <https://doi.org/10.1103/PhysRevLett.108.058301>.
- [30] J. Schmidt, M.R.G. Marques, S. Botti, M.A.L. Marques, Recent advances and applications of machine learning in solid-state materials science, *NPJ. Comput. Mater.* 5 (2019) 83, <https://doi.org/10.1038/s41524-019-0221-0>.
- [31] D. Rogers, M. Hahn, Extended-connectivity fingerprints, *J. Chem. Inf. Model.* 50 (2010) 742–754, <https://doi.org/10.1021/ci100050t>.
- [32] R. Nilakantan, N. Bauman, J.S. Dixon, R. Venkataraghavan, Topological torsion: a new molecular descriptor for SAR applications. Comparison with other descriptors, *J. Chem. Inf. Comput. Sci.* 27 (1987) 82–85, <https://doi.org/10.1021/ci00054a008>.
- [33] J.L. Durant, B.A. Leland, D.R. Henry, J.G. Nourse, Reoptimization of MDL keys for use in drug discovery, *J. Chem. Inf. Comput. Sci.* 42 (2002) 1273–1280, <https://doi.org/10.1021/ci010132r>.
- [34] I.P. Malashin, V.S. Tynchenko, V.A. Nelyub, A.S. Borodulin, A.P. Gantimurov, Estimation and prediction of the polymers' Physical characteristics using the machine learning models, *Polymers (Basel)* 16 (2023) 115, <https://doi.org/10.3390/polym16010115>.
- [35] K.C. Felton, L. Raßpe-Lange, J.G. Rittig, K. Leonhard, A. Mitsos, J. Meyer-Kirschner, C. Knösche, A.A. Lapkin, ML-SAFT: a machine learning framework for PCP-SAFT parameter prediction, *Chem. Eng. J.* 492 (2024) 151999, <https://doi.org/10.1016/j.cej.2024.151999>.
- [36] A.F. Zahrt, J.J. Henle, B.T. Rose, Y. Wang, W.T. Darrow, S.E. Denmark, Prediction of higher-selectivity catalysts by computer-driven workflow and machine learning, *Science* (1979) 363, <https://doi.org/10.1126/science.aau5631>, 2019.
- [37] S. Choi, Y. Kim, J.W. Kim, Z. Kim, W.Y. Kim, Feasibility of activation energy prediction of gas-phase reactions by machine learning, *Chemistry – a Eur. J.* 24 (2018) 12354–12358, <https://doi.org/10.1002/chem.201800345>.
- [38] P. Reiser, M. Neubert, A. Eberhard, L. Torresi, C. Zhou, C. Shao, H. Metni, C. van Hoessel, H. Schopmans, T. Sommer, P. Friederich, Graph neural networks for materials science and chemistry, *Commun. Mater.* 3 (2022) 93, <https://doi.org/10.1038/s43246-022-00315-6>.
- [39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, (2017).
- [40] S. Chithrananda, G. Grand, B. Ramsundar, ChemBERTa: large-scale self-supervised pretraining for molecular property prediction, (2020).
- [41] J. Ock, C. Guntuboina, A. Barati Farimani, Catalyst energy prediction with CatBERTa: unveiling feature exploration strategies through large language models, *ACS. Catal.* 13 (2023) 16032–16044, <https://doi.org/10.1021/acscatal.3c04956>.
- [42] A. Yüksel, E. Ulusoy, A. Ünlü, T. Doğan, SELFFormer: molecular representation learning via SELFIES language models, (2023).
- [43] J. Yu, C. Zhang, Y. Cheng, Y.-F. Yang, Y.-B. She, F. Liu, W. Su, A. Su, SolvBERT for solvation free energy and solubility prediction: a demonstration of an NLP model for predicting the properties of molecular complexes, *Digit. Discov.* 2 (2023) 409–421, <https://doi.org/10.1039/D2DD00107A>.
- [44] J. Ross, B. Belgodere, V. Chenthamarakshan, I. Padhi, Y. Mroueh, P. Das, Large-scale chemical language representations capture molecular structure and properties, *Nat. Mach. Intell.* 4 (2022) 1256–1264, <https://doi.org/10.1038/s42256-022-00580-7>.
- [45] J. Li, X. Jiang, Mol-BERT: an effective molecular representation with BERT for molecular property prediction, *Wirel. Commun. Mob. Comput.* (2021) 2021, <https://doi.org/10.1155/2021/171815>.
- [46] S. Wang, Y. Guo, Y. Wang, H. Sun, J. Huang, SMILES-BERT, in: *Proceedings of the 10th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics*, New York, NY, USA, ACM, 2019, pp. 429–436, <https://doi.org/10.1145/3307339.3342186>.
- [47] A. Seko, A. Togo, I. Tanaka, Descriptors For Machine Learning of Materials Data, *Nanoinformatics*, Springer Singapore, Singapore, 2018, pp. 3–23, https://doi.org/10.1007/978-981-10-7617-6_1.
- [48] S. Raghunathan, U.D. Priyakumar, Molecular representations for machine learning applications in chemistry, *Int. J. Quantum. Chem.* 122 (2022), <https://doi.org/10.1002/qua.26870>.
- [49] H.L. Morgan, The generation of a unique machine description for chemical structures—a technique developed at chemical abstracts service, *J. Chem. Doc.* 5 (1965) 107–113, <https://doi.org/10.1021/c160017a018>.
- [50] J. Behler, Atom-centered symmetry functions for constructing high-dimensional neural network potentials, *J. Chem. Phys.* 134 (2011), <https://doi.org/10.1063/1.3553717>.
- [51] K.T. Schütt, H. Glawe, F. Brockherde, A. Sanna, K.R. Müller, E.K.U. Gross, How to represent crystal structures for machine learning: towards fast prediction of electronic properties, *Phys. Rev. B* 89 (2014) 205118, <https://doi.org/10.1103/PhysRevB.89.205118>.
- [52] A.P. Bartók, R. Kondor, G. Csányi, On representing chemical environments, *Phys. Rev. B* 87 (2013) 184115, <https://doi.org/10.1103/PhysRevB.87.184115>.

- [53] A.S. Christensen, L.A. Bratholm, F.A. Faber, O. Anatole von Lilienfeld, FCHL revisited: faster and more accurate quantum machine learning, *J. Chem. Phys.* 152 (2020), <https://doi.org/10.1063/1.5126701>.
- [54] B. Huang, O.A. von Lilienfeld, Quantum machine learning using atom-in-molecule-based fragments selected on the fly, *Nat. Chem.* 12 (2020) 945–951, <https://doi.org/10.1038/s41557-020-0527-z>.
- [55] G. Landrum, RDKit: open-source cheminformatics, (2006).
- [56] David Weininger, SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules, *J. Chem. Inf. Comput. Sci.* (1988) 31–36.
- [57] Fingerprinting and molecular similarity - getting started In Python. <https://www.rdkit.org/docs/GettingStartedInPython.html#fingerprinting-and-molecular-similarity>, (n.d.).
- [58] D. Theng, K.K. Bhojar, Feature selection techniques for machine learning: a survey of more than two decades of research, *Knowl. Inf. Syst.* 66 (2024) 1575–1637, <https://doi.org/10.1007/s10115-023-02010-5>.
- [59] M.Ebrahimi Warkiani, M.H. Moattar, A comprehensive survey on recent feature selection methods for mixed data: challenges, solutions and future directions, *Neurocomputing*. 623 (2025) 129372, <https://doi.org/10.1016/j.neucom.2025.129372>.
- [60] I.T. Jolliffe, J. Cadima, Principal component analysis: a review and recent developments, *Philos. Trans. R. Soc. A* 374 (2016) 20150202, <https://doi.org/10.1098/rsta.2015.0202>.
- [61] Z.-P. Liu, Linear discriminant analysis, in: *Encyclopedia of Systems Biology*, Springer New York, New York, NY, 2013, pp. 1132–1133, https://doi.org/10.1007/978-1-4419-9863-7_395.
- [62] C.E. Rasmussen, C.K.I. Williams, *Gaussian Processes for Machine Learning*, The MIT Press, 2005, <https://doi.org/10.7551/mitpress/3206.001.0001>.
- [63] J.R. Quinlan, Induction of decision trees, *Mach. Learn.* 1 (1986) 81–106, <https://doi.org/10.1007/BF00116251>.
- [64] L. Breiman, Random forests, *Mach. Learn.* 45 (2001) 5–32, <https://doi.org/10.1023/A:1010933404324>.
- [65] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: a highly efficient gradient boosting decision tree, in: *31st Conference on Neural Information Processing Systems (NIPS, Long Beach, CA, USA, 2017 n.d.*
- [66] T. Chen, C. Guestrin, XGBoost: a scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, New York, NY, USA, ACM, 2016, pp. 785–794, <https://doi.org/10.1145/2939672.2939785>.
- [67] R. Tibshirani, Regression shrinkage and selection via the lasso, *J. R. Stat. Soc. Ser. B (Methodological)* (1996).
- [68] R. Rosipal, N. Krämer, Overview and recent advances in partial least squares, in: 2006: pp. 34–51, https://doi.org/10.1007/11752790_2.
- [69] C. Cortes, V. Vapnik, Support-Vector Networks, *Mach. Learn.* 1995, pp. 273–297.
- [70] L. Grinsztajn, E. Oyallon, G. Varoquaux, Why do tree-based models still outperform deep learning on tabular data?, in: *36th Conference on Neural Information Processing Systems*, 2022.
- [71] C. Chen, W. Ye, Y. Zuo, C. Zheng, S.P. Ong, Graph networks as a universal machine learning framework for molecules and crystals, *Chem. Mater.* 31 (2019) 3564–3572, <https://doi.org/10.1021/acs.chemmater.9b01294>.
- [72] J. Gastegger, J. Groß, S. Günnemann, Directional message passing for molecular graphs, in: 2020.
- [73] J. Gastegger, F. Becker, S. Günnemann, GemNet: universal directional graph neural networks for molecules, (2021).
- [74] K.T. Schütt, P.-J. Kindermans, H.E. Sauceda, S. Chmiela, A. Tkatchenko, K.-R. Müller, SchNet: a continuous-filter convolutional neural network for modeling quantum interactions, in: *31st Conference on Neural Information Processing Systems*, 2017.
- [75] P. Xu, X. Ji, M. Li, W. Lu, Small data machine learning in materials science, *NPJ. Comput. Mater.* 9 (2023) 42, <https://doi.org/10.1038/s41524-023-01000-z>.
- [76] J. Deng, Z. Yang, H. Wang, I. Ojima, D. Samaras, F. Wang, A systematic study of key elements underlying molecular property prediction, *Nat. Commun.* 14 (2023) 6395, <https://doi.org/10.1038/s41467-023-41948-6>.
- [77] D.H. Mok, H. Li, G. Zhang, C. Lee, K. Jiang, S. Back, Data-driven discovery of electrocatalysts for CO₂ reduction using active motifs-based machine learning, *Nat. Commun.* 14 (2023) 7033, <https://doi.org/10.1038/s41467-023-43118-0>.
- [78] D. Morgan, R. Jacobs, Data for “machine learning design of perovskite catalytic properties,” DOI: 10.6084/M9.Figshare.24450445.V2 (2023).
- [79] R. Jacobs, J. Liu, H. Abernathy, D. Morgan, Machine learning design of Perovskite catalytic properties, *Adv. Energy Mater.* 14 (2024), <https://doi.org/10.1002/aenm.202303684>.
- [80] Z. Li, S. Wang, W.S. Chin, L.E. Achenie, H. Xin, High-throughput screening of bimetallic catalysts enabled by machine learning, *J. Mater. Chem. A Mater.* 5 (2017) 24131–24138, <https://doi.org/10.1039/C7TA01812F>.
- [81] T.D. Huan, A. Mannodi-Kanakkithodi, C. Kim, V. Sharma, G. Piliandia, R. Ramprasada, A polymer dataset for accelerated property prediction and design, *Sci. Data* 3 (2016) 160012, <https://doi.org/10.1038/sdata.2016.12>.
- [82] Polymer genome platform, www.polymergenome.org, (n.d.).
- [83] C. Kim, A. Chandrasekaran, T.D. Huan, D. Das, R. Ramprasada, Polymer genome: a data-powered Polymer informatics platform for property predictions, *J. Phys. Chem. C* 122 (2018) 17575–17585, <https://doi.org/10.1021/acs.jpcc.8b02913>.
- [84] K. Wu, N. Sukumar, N.A. Lanzillo, C. Wang, R. “Rampi” Ramprasada, R. Ma, A. F. Baldwin, G. Sotzing, C. Breneman, Prediction of polymer properties using infinite chain descriptors (ICD) and machine learning: toward optimized dielectric polymeric materials, *J. Polym. Sci. B Polym. Phys.* 54 (2016) 2082–2091, <https://doi.org/10.1002/polb.24117>.
- [85] S. Otsuka, I. Kuwajima, J. Hosoya, Y. Xu, M. Yamazaki, PolyInfo: polymer database for Polymeric materials design, in: *2011 International Conference on Emerging Intelligent Data and Web Technologies, IEEE*, 2011, pp. 22–29, <https://doi.org/10.1109/EIDWT.2011.13>.
- [86] L. Tao, V. Varshney, Y. Li, Benchmarking machine learning models for polymer informatics: an example of glass transition temperature, *J. Chem. Inf. Model.* 61 (2021) 5395–5413, <https://doi.org/10.1021/acs.jcim.1c01031>.
- [87] Dortmund Datenbank. <https://www.ddbst.com/>, 2022.
- [88] J. Ferraz-Caetano, F. Teixeira, M.N.D.S. Cordeiro, Explainable supervised machine learning model to predict solvation gibbs energy, *J. Chem. Inf. Model.* 64 (2024) 2250–2262, <https://doi.org/10.1021/acs.jcim.3c00544>.
- [89] C.L. Yaws, *The Yaws handbook of thermodynamic Properties for hydrocarbons and chemicals*, 2006.
- [90] M.M. Ghahremanpour, P.J. van Maaren, J.C. Ditz, R. Lindh, D. van der Spoel, Large-scale calculations of gas phase thermochemistry: enthalpy of formation, standard entropy, and heat capacity, *J. Chem. Phys.* 145 (2016), <https://doi.org/10.1063/1.4962627>.
- [91] M.N. Aldosari, K.K. Yalamanchi, X. Gao, S.M. Sarathy, Predicting entropy and heat capacity of hydrocarbons using machine learning, *Energy AI* 4 (2021) 100054, <https://doi.org/10.1016/j.egyai.2021.100054>.
- [92] K.K. Yalamanchi, V.C.O. van Oudenhoven, F. Tutino, M. Monge-Palacios, A. Alshehri, X. Gao, S.M. Sarathy, Machine learning to predict standard enthalpy of formation of hydrocarbons, *J. Phys. Chem. A* 123 (2019) 8305–8313, <https://doi.org/10.1021/acs.jpca.9b04771>.
- [93] B. Oral, B. Tekin, D. Eroglu, R. Yildirim, Performance analysis of Na-ion batteries by machine learning, *J. Power. Sources*. 549 (2022) 232126, <https://doi.org/10.1016/j.jpowsour.2022.232126>.
- [94] G. Wang, T. Fearn, T. Wang, K.-L. Choy, Machine-learning approach for predicting the discharging capacities of doped lithium nickel–Cobalt–Manganese cathode materials in Li-ion batteries, *ACS. Cent. Sci.* 7 (2021) 1551–1560, <https://doi.org/10.1021/acscentsci.1c00611>.
- [95] K. Min, B. Choi, K. Park, E. Cho, Machine learning assisted optimization of electrochemical properties for Ni-rich cathode materials, *Sci. Rep.* 8 (2018) 15778, <https://doi.org/10.1038/s41598-018-34201-4>.
- [96] M. Shariq, S. Marimuthu, A.R. Dixit, S. Chattopadhyaya, S. Pandiaraj, M. Muthuramamoorthy, A.N. Alodhyab, M. Khaja Nazeeruddin, A.N. Grace, Machine learning models for prediction of electrochemical properties in supercapacitor electrodes using MXene and graphene nanoplatelets, *Chem. Eng. J.* 484 (2024) 149502, <https://doi.org/10.1016/j.cej.2024.149502>.
- [97] J. Lu, H. Zhang, J. Yu, D. Shan, J. Qi, J. Chen, H. Song, M. Yang, Predicting rate constants of hydroxyl radical reactions with Alkanes using machine learning, *J. Chem. Inf. Model.* 61 (2021) 4259–4265, <https://doi.org/10.1021/acs.jcim.1c00809>.
- [98] C.W. Gao, J.W. Allen, W.H. Green, R.H. West, Reaction mechanism Generator: automatic construction of chemical kinetic mechanisms, *Comput. Phys. Commun.* 203 (2016) 212–225, <https://doi.org/10.1016/j.cpc.2016.02.013>.
- [99] D.T. Ahneman, J.G. Estrada, S. Lin, S.D. Dreher, A.G. Doyle, Predicting reaction performance in C–N cross-coupling using machine learning, *Science* 360 (2018) 186–190, <https://doi.org/10.1126/science.aar5169>.
- [100] J. Hoja, L. Medrano Sandonas, B.G. Ernst, A. Vazquez-Mayagoitia, R.A. DiStasio, A. Tkatchenko, QM7-X, a comprehensive dataset of quantum-mechanical properties spanning the chemical space of small organic molecules, *Sci. Data* 8 (2021) 43, <https://doi.org/10.1038/s41597-021-00812-2>.
- [101] D.L. Mobley, J.P. Guthrie, FreeSolv: a database of experimental and calculated hydration free energies, with input files, *J. Comput. Aided. Mol. Des.* 28 (2014) 711–720, <https://doi.org/10.1007/s10822-014-9747-x>.
- [102] Q. Yang, Y. Liu, J. Cheng, Y. Li, S. Liu, Y. Duan, L. Zhang, S. Luo, An ensemble structure and physicochemical (SPOC) descriptor for machine-learning prediction of chemical reaction and molecular properties, *Chemphyschem.* 23 (2022), <https://doi.org/10.1002/cphc.202200255>.
- [103] S. Honda, S. Shi, H.R. Ueda, SMILES transformer: pre-trained molecular fingerprint for low data drug discovery, (2019).
- [104] D. Duvenaud, D. Maclaurin, J. Aguilera-Iparraguirre, R. Gómez-Bombarelli, T. Hirzel, A. Aspuru-Guzik, R.P. Adams, Convolutional networks on graphs for learning molecular fingerprints, (2015).
- [105] S. Kearnes, K. McCloskey, M. Berndl, V. Pande, P. Riley, Molecular graph convolutions: moving beyond fingerprints, *J. Comput. Aided. Mol. Des.* 30 (2016) 595–608, <https://doi.org/10.1007/s10822-016-9938-8>.
- [106] K. Jacobs, *Independent identically distributed (IID) random variables. Discrete Stochastics*, Birkhäuser Basel, Basel, 1992, pp. 65–101, https://doi.org/10.1007/978-3-0348-8645-1_4.
- [107] Y.-L. Liao, B. Wood, A. Das, T. Smidt, EquiformerV2: Improved equivariant transformer for scaling to higher-degree representations, (2023).
- [108] N. Thomas, T. Smidt, S. Kearnes, L. Yang, L. Li, K. Kohlhoff, P. Riley, Tensor field networks: rotation- and translation-equivariant neural networks for 3D point clouds, (2018).
- [109] M. Krenn, F. Häse, A. Nigam, P. Friederich, A. Aspuru-Guzik, Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation, *Mach. Learn. Sci. Technol.* 1 (2020) 045024, <https://doi.org/10.1088/2632-2153/aba947>.
- [110] S.R. Heller, A. McNaught, I. Pletnev, S. Stein, D. Tchekhovskoi, InChI, the IUPAC International Chemical Identifier, *J. Cheminform.* 7 (2015) 23, <https://doi.org/10.1186/s13321-015-0068-4>.
- [111] T.-S. Lin, C.W. Coley, H. Mochigase, H.K. Beech, W. Wang, Z. Wang, E. Woods, S. L. Craig, J.A. Johnson, J.A. Kalow, K.F. Jensen, B.D. Olsen, BigSMILES: A

- structurally-based line notation for describing macromolecules, *ACS. Cent. Sci.* 5 (2019) 1523–1531, <https://doi.org/10.1021/acscentsci.9b00476>.
- [112] David Weininger, SMARTS - a language for describing molecular patterns, <https://www.daylight.com/Dayhtml/Doc/Theory/Theory.Smarts.html> (n.d.).
- [113] M. Abbasi, B.P. Santos, T.C. Pereira, R. Sofia, N.R.C. Monteiro, C.J.V. Simões, R. M.M. Brito, B. Ribeiro, J.L. Oliveira, J.P. Arrais, Designing optimized drug candidates with Generative Adversarial Network, *J. Cheminform.* 14 (2022) 40, <https://doi.org/10.1186/s13321-022-00623-6>.
- [114] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, (2015).
- [115] T. Kudo, Subword regularization: improving neural network translation models with multiple Subword candidates, (2018).
- [116] X. Song, A. Salcianu, Y. Song, D. Dopson, D. Zhou, Fast WordPiece Tokenization, (2020).
- [117] C. Xu, Y. Wang, A. Barati Farimani, TransPolymer: a transformer-based language model for polymer property predictions, *NPJ. Comput. Mater.* 9 (2023) 64, <https://doi.org/10.1038/s41524-023-01016-5>.
- [118] B. Winter, C. Winter, J. Schilling, A. Bardow, A smile is all you need: predicting limiting activity coefficients from SMILES with natural language processing, *Digit. Discov.* 1 (2022) 859–869, <https://doi.org/10.1039/D2DD00058J>.
- [119] R.M. Schmidt, Recurrent Neural Networks (RNNs): a gentle introduction and overview, (2019).
- [120] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (1997) 1735–1780, <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [121] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Gated feedback recurrent neural networks, in: F. Bach, D. Blei (Eds.), *Proceedings of the 32nd International Conference on Machine Learning*, Lille, France, PMLR, 2015, pp. 2067–2075, in: <https://proceedings.mlr.press/v37/chung15.html>.
- [122] S. Goswami, R. Ghosh, A. Neog, B. Das, Deep learning based approach for prediction of glass transition temperature in polymers, *Mater. Today Proc.* 46 (2021) 5838–5843, <https://doi.org/10.1016/j.matpr.2021.02.730>.
- [123] Y. Wang, M. Huang, Xiaoyan Zhu, L. Zhao, Attention-based LSTM for aspect-level sentiment classification, in: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, Stroudsburg, PA, USA, Association for Computational Linguistics, 2016, pp. 606–615, <https://doi.org/10.18653/v1/D16-1058>.
- [124] P.J. Canatalay, O.N. Ucan, A bidirectional LSTM-RNN and GRU method to exon prediction using splice-site mapping, *Appl. Sci.* 12 (2022) 4390, <https://doi.org/10.3390/app12094390>.
- [125] J. Qian, X. Qiu, X. Tan, Q. Li, J. Chen, X. Jiang, An attentive LSTM based approach for adverse drug reactions prediction, *Appl. Intell.* (2022), <https://doi.org/10.1007/s10489-022-03721-y>.
- [126] D. Bhattacharya, H.J. Cassidy, M.A. Hickner, W.F. Reinhart, Large language models as molecular design engines, *J. Chem. Inf. Model.* 64 (2024) 7086–7096, <https://doi.org/10.1021/acs.jcim.4c01396>.
- [127] P. Schwaller, T. Laino, T. Gaudin, P. Bolgar, C.A. Hunter, C. Bekas, A.A. Lee, Molecular transformer: A model for uncertainty-calibrated chemical reaction prediction, *ACS. Cent. Sci.* 5 (2019) 1572–1583, <https://doi.org/10.1021/acscentsci.9b00576>.
- [128] B. Winter, C. Winter, T. Esper, J. Schilling, A. Bardow, SPT-NRTL: A physics-guided machine learning model to predict thermodynamically consistent activity coefficients, *Fluid. Phase Equilib.* 568 (2023) 113731, <https://doi.org/10.1016/j.fluid.2023.113731>.
- [129] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, (2018).
- [130] M.A.K. Raiaan, Md.S.H. Mukta, K. Fatema, N.M. Fahad, S. Sakib, M.M.J. Mim, J. Ahmad, M.E. Ali, S. Azam, A review on large language models: architectures, applications, taxonomies, open issues and challenges, *IEEe Access.* 12 (2024) 26839–26874, <https://doi.org/10.1109/ACCESS.2024.3365742>.
- [131] Y. Gu, R. Tinn, H. Cheng, M. Lucas, N. Usuyama, X. Liu, T. Naumann, J. Gao, H. Poon, Domain-specific language model pretraining for biomedical natural language processing, *ACM. Trans. Comput. Healthc.* 3 (2022) 1–23, <https://doi.org/10.1145/3458754>.
- [132] M.G. Quiles, P.A.L. Ribeiro, G.A. Pinheiro, R.C. Prati, J.L.F. da Silva, Enhancing low-cost molecular property prediction with contrastive learning on SMILES representations, in: 2024: pp. 387–401. https://doi.org/10.1007/978-3-031-65329-2_26.
- [133] J. Jiang, Y. Li, R. Zhang, Y. Liu, INTransformer: data augmentation-based contrastive learning by injecting noise into transformer for molecular property prediction, *J. Mol. Graph. Model.* 128 (2024) 108703, <https://doi.org/10.1016/j.jmgm.2024.108703>.
- [134] S. Liu, W. Nie, C. Wang, J. Lu, Z. Qiao, L. Liu, J. Tang, C. Xiao, A. Anandkumar, Multi-modal molecule structure–text model for text-based retrieval and editing, *Nat. Mach. Intell.* 5 (2023) 1447–1457, <https://doi.org/10.1038/s42256-023-00759-6>.
- [135] F. Schroff, D. Kalenichenko, J. Philbin, FaceNet: A unified embedding for face recognition and clustering, (2015). <https://doi.org/10.1109/CVPR.2015.7298682>.
- [136] T. Chen, S. Kornblith, M. Norouzi, G. Hinton, A simple framework for contrastive learning of visual representations, (2020).
- [137] X. Wang, R. Zhang, C. Shen, T. Kong, L. Li, Dense contrastive learning for self-supervised visual pre-training, (2020).
- [138] Y. Wang, J. Wang, Z. Cao, A. Barati Farimani, Molecular contrastive learning of representations via graph neural networks, *Nat. Mach. Intell.* 4 (2022) 279–287, <https://doi.org/10.1038/s42256-022-00447-x>.
- [139] Z. He, L. Chen, H. Lv, R. Zhou, J. Xu, Y. Chen, J. Hu, Y. Gao, A novel descriptor and molecular graph-based bimodal contrastive learning framework for drug molecular property prediction, in: 2023: pp. 700–715. https://doi.org/10.1007/978-981-99-4749-2_60.
- [140] X. Jiang, L. Tan, Q. Zou, DGCL: dual-graph neural networks contrastive learning for molecular property prediction, *Brief. Bioinform.* 25 (2024), <https://doi.org/10.1093/bib/bbae474>.
- [141] T. Kuang, Y. Ren, Z. Ren, 3D-Mol: a novel contrastive learning framework for molecular property prediction with 3D information, *Pattern Anal. Appl.* 27 (2024) 71, <https://doi.org/10.1007/s10044-024-01287-8>.
- [142] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: A robustly optimized BERT pretraining approach, (2019).
- [143] P. He, X. Liu, J. Gao, W. Chen, DeBERTa: decoding-enhanced BERT with disentangled attention, (2020).
- [144] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: open and efficient foundation language models, (2023).
- [145] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, (2019).
- [146] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, L. Zettlemoyer, BART: Denoising Sequence-To-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension, 2019.
- [147] O.K. Alhmoudi, M. Aboushanab, M. Thameem, A. Elkamel, A.A. AlHammadi, Domain adaptation of a SMILES chemical transformer to SELFIES with limited computational resources, *Sci. Rep.* 15 (2025) 23627, <https://doi.org/10.1038/s41598-025-05017-w>.
- [148] O.K. Alhmoudi, M. Thameem, A. Elkamel, A.A. AlHammadi, Low-compute repurposing of an English transformer into a molecular encoder: A two-stage SELFIES adaptation of RoBERTa, *Res. Eng.* 28 (2025) 108390, <https://doi.org/10.1016/j.rineng.2025.108390>.
- [149] F. Cai, K. Hanna, T. Zhu, T.-R. Tzeng, Y. Duan, L. Liu, S. Pilla, G. Li, F. Luo, A foundation model for chemical design and property prediction, (2024).
- [150] J. Chambers, M. Davies, A. Gaulton, A. Hersey, S. Velankar, R. Petryszak, J. Hastings, L. Bellis, S. McGlinchey, J.P. Overington, UniChem: a unified chemical structure cross-referencing and identifier tracking system, *J. Cheminform.* 5 (2013) 3, <https://doi.org/10.1186/1758-2946-5-3>.
- [151] B. Winter, P. Rehner, T. Esper, J. Schilling, A. Bardow, Understanding the language of molecules: predicting pure component parameters for the PC-SAFT equation of state from SMILES, *Digit. Discov.* (2025), <https://doi.org/10.1039/D4DD00077C>.
- [152] Zhuoran Li, Xu Sun, Wanyu Lin, Jiannong Cao, Unveiling Molecular Secrets: An LLM-Augmented Linear Model For Explainable and Calibratable Molecular Property Prediction, *ArXiv*, 2024.
- [153] J.J. Irwin, T. Sterling, M.M. Mysinger, E.S. Bolstad, R.G. Coleman, ZINC: A free tool to discover chemistry for biology, *J. Chem. Inf. Model.* 52 (2012) 1757–1768, <https://doi.org/10.1021/ci3001277>.
- [154] R. Ma, T. Luo, PIM: a benchmark database for polymer informatics, *J. Chem. Inf. Model.* 60 (2020) 4684–4690, <https://doi.org/10.1021/acs.jcim.0c00726>.
- [155] C. Kuenneth, R. Ramprasad, polyBERT: a chemical language model to enable fully machine-driven ultrafast polymer informatics, *Nat. Commun.* 14 (2023) 4099, <https://doi.org/10.1038/s41467-023-39868-6>.
- [156] C. Kuenneth, Rampi Ramprasad, polyOne Data Set - 100 million hypothetical polymers including 29 properties, (2022).
- [157] H. Huang, R. Magar, C. Xu, A.B. Farimani, Materials informatics transformer: a language model for interpretable Materials properties prediction, (2023).
- [158] K. Choudhary, K.F. Garrity, A.C.E. Reid, B. DeCost, A.J. Bicchieri, A.R. Hight Walker, Z. Trautt, J. Hatrick-Simpers, A.G. Kusne, A. Centrone, A. Davydov, J. Jiang, R. Pachter, G. Cheon, E. Reed, A. Agrawal, X. Qian, V. Sharma, H. Zhuang, S.V. Kalinin, B.G. Sumpter, G. Plania, P. Acar, S. Mandal, K. Haule, D. Vanderbilt, K. Rabe, F. Tavazza, The joint automated repository for various integrated simulations (JARVIS) for data-driven materials design, *NPJ. Comput. Mater.* 6 (2020) 173, <https://doi.org/10.1038/s41524-020-00440-1>.
- [159] D. Mendez, A. Gaulton, A.P. Bento, J. Chambers, M. De Veij, E. Félix, M. P. Magariños, J.F. Mosquera, P. Mutowo, M. Nowotka, M. Gordillo-Marañón, F. Hunter, L. Junco, G. Mugumbate, M. Rodriguez-Lopez, F. Atkinson, N. Bosc, C. J. Radoux, A. Segura-Cabrera, A. Hersey, A.R. Leach, ChEMBL: towards direct deposition of bioassay data, *Nucleic. Acids. Res.* 47 (2019) D930–D940, <https://doi.org/10.1093/nar/gky1075>.
- [160] F.H. Vermeire, W.H. Green, Transfer learning for solvation free energies: from quantum chemistry to experiments, *Chem. Eng. J.* 418 (2021) 129307, <https://doi.org/10.1016/j.cej.2021.129307>.
- [161] S. Boobier, D.R.J. Hose, A.J. Blacker, B.N. Nguyen, Machine learning with physicochemical relationships: solubility prediction in organic solvents and water, *Nat. Commun.* 11 (2020) 5753, <https://doi.org/10.1038/s41467-020-19594-z>.
- [162] A.N. Rubungo, C. Arnold, B.P. Rand, A.B. Dieng, LLM-prop: predicting physical and electronic properties of crystalline solids from their text descriptions, (2023).
- [163] W. Ahmad, E. Simon, S. Chithrananda, G. Grand, B. Ramsundar, ChemBERTa-2: towards chemical foundation models, (2022).
- [164] S. Kim, J. Chen, T. Cheng, A. Gindulyte, J. He, S. He, Q. Li, B.A. Shoemaker, P. A. Thiessen, B. Yu, L. Zaslavsky, J. Zhang, E.E. Bolton, PubChem 2025 update, *Nucleic. Acids. Res.* 53 (2025) D1516–D1525, <https://doi.org/10.1093/nar/gkae1059>.

- [165] T. Brouwer, S.R.A. Kersten, G. Bargeman, B. Schuur, trends in solvent impact on infinite dilution activity coefficients of solutes reviewed and visualized using an algorithm to support selection of solvents for greener fluid separations, *Sep. Purif. Technol.* 272 (2021) 118727, <https://doi.org/10.1016/j.seppur.2021.118727>.
- [166] R. Irwin, S. Dimitriadis, J. He, E.J. Bjerrum, Chemformer: a pre-trained transformer for computational chemistry, *Mach. Learn. Sci. Technol.* 3 (2022) 015022, <https://doi.org/10.1088/2632-2153/ac3fbf>.
- [167] T. Sterling, J.J. Irwin, ZINC 15 – Ligand discovery for everyone, *J. Chem. Inf. Model.* 55 (2015) 2324–2337, <https://doi.org/10.1021/acs.jcim.5b00559>.
- [168] B. Fabian, T. Edlich, H. Gaspar, M. Segler, J. Meyers, M. Fiscato, M. Ahmed, Molecular representation learning with language models and domain-relevant auxiliary tasks, (2020).
- [169] S.J. Capuzzi, I.S.-J. Kim, W.I. Lam, T.E. Thornton, E.N. Muratov, D. Pozefsky, A. Tropsha, ChEMBL: a publicly accessible, integrated cheminformatics portal, *J. Chem. Inf. Model.* 57 (2017) 105–108, <https://doi.org/10.1021/acs.jcim.6b00462>.
- [170] A. Gaulton, A. Hersey, M. Nowotka, A.P. Bento, J. Chambers, D. Mendez, P. Mutowo, F. Atkinson, L.J. Bellis, E. Cibrián-Uhalte, M. Davies, N. Dedman, A. Karlsson, M.P. Magariños, J.P. Overington, G. Papadatos, I. Smit, A.R. Leach, The ChEMBL database in 2017, *Nucleic. Acids. Res.* 45 (2017) D945–D954, <https://doi.org/10.1093/nar/gkw1074>.
- [171] S. Wang, Y. Guo, Y. Wang, H. Sun, J. Huang, in: *Proceedings of the 10th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics*, New York, NY, USA, ACM, 2019, pp. 429–436, <https://doi.org/10.1145/3307339.3342186>.
- [172] Tokenizers, n.d. <https://huggingface.co/docs/tokenizers/en/index>, 2025. accessed April 20
- [173] Models (n.d.), https://huggingface.co/docs/transformers/en/main_classes/model, 2025. accessed April 20.
- [174] Graph Theory, n.d. www.wikipedia.org/wiki/Graph_theory, 2025. accessed April 20
- [175] A. Pocha, T. Danel, S. Podlowska, J. Tabor, L. Maziarka, Comparison of atom representations in graph neural networks for molecular property prediction, in: 2021 International Joint Conference on Neural Networks (IJCNN), IEEE, 2021, pp. 1–8, <https://doi.org/10.1109/IJCNN52387.2021.9533698>.
- [176] Z. Xiong, D. Wang, X. Liu, F. Zhong, X. Wan, X. Li, Z. Li, X. Luo, K. Chen, H. Jiang, M. Zheng, Pushing the boundaries of molecular representation for drug discovery with the graph attention mechanism, *J. Med. Chem.* 63 (2020) 8749–8760, <https://doi.org/10.1021/acs.jmedchem.9b00959>.
- [177] Johannes Gasteiger, On the Convergence of Structure and Geometry in Graph Neural Networks, PhD, Technical University of Munich, 2022.
- [178] M. Fey, J.E. Lenssen, Fast graph representation learning with PyTorch geometric, (2019).
- [179] X. Zang, X. Zhao, B. Tang, Hierarchical molecular graph self-supervised learning for property prediction, *Commun. Chem.* 6 (2023) 34, <https://doi.org/10.1038/s42004-023-00825-5>.
- [180] S. Batzner, A. Musaelian, L. Sun, M. Geiger, J.P. Mailoa, M. Kornbluth, N. Molinari, T.E. Smidt, B. Kozinsky, E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials, *Nat. Commun.* 13 (2022) 2453, <https://doi.org/10.1038/s41467-022-29939-5>.
- [181] H. Ma, Y. Bian, Y. Rong, W. Huang, T. Xu, W. Xie, G. Ye, J. Huang, Cross-dependent graph neural networks for molecular property prediction, *Bioinformatics.* 38 (2022) 2003–2009, <https://doi.org/10.1093/bioinformatics/btac039>.
- [182] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, (2017).
- [183] M.M. Bronstein, J. Bruna, T. Cohen, P. Velicković, Geometric deep learning: grids, groups, graphs, geodesics, and gauges, (2021).
- [184] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, in: *International Conference on Learning Representations (ICLR)*, 2018.
- [185] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Velickovic, Principal neighbourhood aggregation for graph nets, in: 34th Conference on Neural Information Processing Systems, NeurIPS, 2020 n.d.
- [186] N. Dehmamy, A.-L. Barabási, R. Yu, Understanding the representation power of graph neural networks in learning graph topology, in: 33rd Conference on Neural Information Processing Systems, NeurIPS, 2019 n.d.
- [187] S. Geisler, D. Zügner, S. Günnemann, Reliable graph neural networks via robust aggregation, in: 34th Conference on Neural Information Processing Systems, NeurIPS, 2020 n.d.
- [188] D. Deng, X. Chen, R. Zhang, Z. Lei, X. Wang, F. Zhou, XGraphBoost: extracting graph neural network-based features for a better prediction of molecular properties, *J. Chem. Inf. Model.* 61 (2021) 2697–2705, <https://doi.org/10.1021/acs.jcim.0c01489>.
- [189] D. Buterez, J.P. Janet, S.J. Kiddle, D. Oglic, P. Lió, Transfer learning with graph neural networks for improved molecular property prediction in the multi-fidelity setting, *Nat. Commun.* 15 (2024) 1517, <https://doi.org/10.1038/s41467-024-45566-8>.
- [190] B. Tang, S.T. Kramer, M. Fang, Y. Qiu, Z. Wu, D. Xu, A self-attention based message passing neural network for predicting molecular lipophilicity and aqueous solubility, *J. Cheminform.* 12 (2020) 15, <https://doi.org/10.1186/s13321-020-0414-z>.
- [191] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, (2016).
- [192] W.L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: 31st Conference on Neural Information Processing Systems, 2017.
- [193] G. Piccini, M.-S. Lee, S.F. Yuk, D. Zhang, G. Collinge, L. Kollias, M.-T. Nguyen, V.-A. Glezakou, R. Rousseau, *Ab initio* molecular dynamics with enhanced sampling in heterogeneous catalysis, *Catal. Sci. Technol.* 12 (2022) 12–37, <https://doi.org/10.1039/D1CY01329G>.
- [194] N. Yao, X. Chen, Z.-H. Fu, Q. Zhang, Applying classical, *Ab initio*, and machine-learning molecular dynamics simulations to the liquid electrolyte for rechargeable batteries, *Chem. Rev.* 122 (2022) 10970–11021, <https://doi.org/10.1021/acs.chemrev.1c00904>.
- [195] J.D. Durrant, J.A. McCammon, Molecular dynamics simulations and drug discovery, *BMC. Biol.* 9 (2011) 71, <https://doi.org/10.1186/1741-7007-9-71>.
- [196] L.C. Blum, J.-L. Reymond, 970 Million druglike small molecules for virtual screening in the chemical Universe database GDB-13, *J. Am. Chem. Soc.* 131 (2009) 8732–8733, <https://doi.org/10.1021/ja902302h>.
- [197] X. Fu, B.M. Wood, L. Barroso-Luque, D.S. Levine, M. Gao, M. Dzamba, C.L. Zitnick, Learning smooth and expressive interatomic potentials for physical property prediction, (2025).
- [198] K.T. Schütt, F. Arbabzadah, S. Chmiela, K.R. Müller, A. Tkatchenko, Quantum-chemical insights from deep tensor neural networks, *Nat. Commun.* 8 (2017) 13890, <https://doi.org/10.1038/ncomms13890>.
- [199] O.T. Unke, M. Meuwly, PhysNet: a neural network for predicting energies, forces, dipole moments, and partial charges, *J. Chem. Theory. Comput.* 15 (2019) 3678–3693, <https://doi.org/10.1021/acs.jctc.9b00181>.
- [200] J. Brandstetter, R. Hesselink, E. van der Pol, E.J. Bekkers, M. Welling, Geometric and physical quantities improve E(3) equivariant message passing, (2021).
- [201] S. Passaro, C.L. Zitnick, Reducing SO(3) convolutions to SO(2) for efficient equivariant GNNs, (2023).
- [202] J. Brandstetter, R. Hesselink, E. van der Pol, E.J. Bekkers, M. Welling, Geometric and physical quantities improve E(3) equivariant message passing, (2021).
- [203] Y.-L. Liao, T. Smidt, Equiformer: equivariant graph attention transformer for 3D atomistic graphs, (2022).
- [204] J. Wang, Y. Wang, H. Zhang, Z. Yang, Z. Liang, J. Shi, H.-T. Wang, D. Xing, J. Sun, E(n)-equivariant cartesian tensor message passing interatomic potential, *Nat. Commun.* 15 (2024) 7607, <https://doi.org/10.1038/s41467-024-51886-6>.
- [205] F.B. Fuchs, D.E. Worrall, V. Fischer, M. Welling, SE(3)-transformers: 3D Roto-translation equivariant attention networks, (2020).
- [206] P. Thölke, G. De Fabritiis, TorchMD-NET: equivariant transformers for neural network based molecular potentials, (2022).
- [207] K.T. Schütt, O.T. Unke, M. Gastegger, Equivariant message passing for the prediction of tensorial properties and molecular spectra, (2021).
- [208] T. Xie, J.C. Grossman, Crystal graph convolutional neural networks for an accurate and interpretable prediction of material properties, (2017). <https://doi.org/10.1103/PhysRevLett.120.145301>.
- [209] Y. Liu, L. Wang, M. Liu, X. Zhang, B. Oztekin, S. Ji, Spherical message passing for 3D graph networks, (2021).
- [210] L. Wang, Y. Liu, Y. Lin, H. Liu, S. Ji, ComENet: towards complete and efficient message passing for 3D molecular graphs, (2022).
- [211] T.S. Cohen, M. Welling, Group equivariant convolutional networks, (2016).
- [212] T.S. Cohen, M. Welling, Steerable CNNs, (2016).
- [213] B. Anderson, T.-S. Hy, R. Kondor, Cormorant: covariant molecular neural networks, (2019).
- [214] I. Batatia, D.P. Kovács, G.N.C. Simm, C. Ortner, G. Csányi, MACE: higher order equivariant message passing neural networks for fast and accurate force fields, (2022).
- [215] A. Musaelian, S. Batzner, A. Johansson, L. Sun, C.J. Owen, M. Kornbluth, B. Kozinsky, Learning local equivariant representations for large-scale atomistic dynamics, *Nat. Commun.* 14 (2023) 579, <https://doi.org/10.1038/s41467-023-36329-y>.
- [216] M. Geiger, T. Smidt, e3nn: Euclidean neural networks, (n.d.).
- [217] S. Batzner, A. Musaelian, L. Sun, M. Geiger, J.P. Mailoa, M. Kornbluth, N. Molinari, T.E. Smidt, B. Kozinsky, E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials, *Nat. Commun.* 13 (2022) 2453, <https://doi.org/10.1038/s41467-022-29939-5>.
- [218] C.L. Zitnick, A. Das, A. Kolluru, J. Lan, M. Shuaibi, A. Sriram, Z. Ulissi, B. Wood, Spherical channels for modeling atomic interactions, in: 2022.
- [219] R. Goodman, Representations and Invariants of the Classical Groups, Wallach, N. R., 2000.
- [220] D.J. Griffiths, D.F. Schroeter, *Introduction to Quantum Mechanics*, Cambridge university press, 2018 n.d.
- [221] M. Geiger, T. Smidt, e3nn: Euclidean neural networks, (2022).
- [222] V.G. Satorras, E. Hoogeboom, M. Welling, E(n) equivariant graph neural networks, (2021).
- [223] M. Haghighatlari, J. Li, X. Guan, O. Zhang, A. Das, C.J. Stein, F. Heidar-Zadeh, M. Liu, M. Head-Gordon, L. Bertels, H. Hao, I. Leven, T. Head-Gordon, NewtonNet: a Newtonian message passing network for deep learning of interatomic potentials and forces, *Digit. Discov.* 1 (2022) 333–343, <https://doi.org/10.1039/D2DD00008C>.
- [224] S. Takamoto, S. Izumi, J. Li, TeaNet: universal neural network interatomic potential inspired by iterative electronic relaxations, *Comput. Mater. Sci.* 207 (2022) 111280, <https://doi.org/10.1016/j.commatsci.2022.111280>.
- [225] G. Simeon, G. de Fabritiis, TensorNet: cartesian tensor representations for efficient learning of molecular potentials, (2023).
- [226] Y. Wang, T. Wang, S. Li, X. He, M. Li, Z. Wang, N. Zheng, B. Shao, T.-Y. Liu, Enhancing geometric representations for molecules with equivariant vector-scalar interactive message passing, *Nat. Commun.* 15 (2024) 313, <https://doi.org/10.1038/s41467-023-43720-2>.

- [227] Y. Park, J. Kim, S. Hwang, S. Han, Scalable parallel algorithm for graph neural network interatomic potentials in molecular dynamics simulations, *J. Chem. Theory. Comput.* 20 (2024) 4857–4868, <https://doi.org/10.1021/acs.jctc.4c00190>.
- [228] C. Bannwarth, S. Ehlert, S. Grimme, GFN2-xTB—An accurate and broadly parametrized self-consistent tight-binding quantum chemical method with multipole electrostatics and density-dependent dispersion contributions, *J. Chem. Theory. Comput.* 15 (2019) 1652–1671, <https://doi.org/10.1021/acs.jctc.8b01176>.
- [229] S.Y. Lee, H. Kim, Y. Park, D. Jeong, S. Han, Y. Park, J.W. Lee, FlashTP: fused, sparsity-aware tensor product for machine learning interatomic potentials, in: *Proceedings of the 42nd International Conference on Machine Learning*, Vancouver, Canada, 2025.
- [230] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, T.-Y. Liu, Do transformers really perform bad for graph representation?, in: *35th Conference on Neural Information Processing Systems*, 2021.
- [231] Y. Rong, Y. Bian, T. Xu, W. Xie, Y. Wei, W. Huang, J. Huang, Self-supervised graph transformer on large-scale molecular data, in: *34th Conference on Neural Information Processing Systems*, 2020.
- [232] T. Kreiman, Y. Bai, F. Atieh, E. Weaver, E. Qu, A.S. Krishnapriyan, Transformers discover molecular structure without graph priors, (2025).
- [233] O.T. Unke, H. Maennel, E3x: E(3)-equivariant deep learning made easy, (2024).
- [234] Shengjie Luo, Tianlang Chen, Aditi S. Krishnapriyan, Enabling efficient equivariant operations in the fourier basis via Gaunt tensor products, in: *International Conference on Learning Representations*, Vienna, Austria, 2024.
- [235] Y. Li, V. Gupta, M.N.T. Kilic, K. Choudhary, D. Wines, W. Liao, A. Choudhary, A. Agrawal, Hybrid-LLM-GNN: integrating large language models and graph neural networks for enhanced materials property prediction, *Digit. Discov.* 4 (2025) 376–383, <https://doi.org/10.1039/D4DD00199K>.
- [236] L. Xie, Y. Jin, L. Xu, S. Chang, X. Xu, Fusing domain knowledge with a fine-tuned large language model for enhanced molecular property prediction, *J. Chem. Theory. Comput.* 21 (2025) 6743–6758, <https://doi.org/10.1021/acs.jctc.5c00605>.
- [237] J.-H. Chen, Y.J. Tseng, A general optimization protocol for molecular property prediction using a deep learning network, *Brief. Bioinform.* 23 (2022), <https://doi.org/10.1093/bib/bbab367>.
- [238] T. Wang, J. Sun, Q. Zhao, Investigating cardiotoxicity related with hERG channel blockers using molecular fingerprints and graph attention mechanism, *Comput. Biol. Med.* 153 (2023) 106464, <https://doi.org/10.1016/j.combiomed.2022.106464>.
- [239] X. Lu, L. Xie, L. Xu, R. Mao, X. Xu, S. Chang, Multimodal fused deep learning for drug property prediction: integrating chemical language and molecular graph, *Comput. Struct. Biotechnol. J.* 23 (2024) 1666–1679, <https://doi.org/10.1016/j.csbj.2024.04.030>.
- [240] D.S. Levine, M. Shuaibi, E.W.C. Spotte-Smith, M.G. Taylor, M.R. Hasyim, K. Michel, I. Batatia, G. Csányi, M. Dzamba, P. Eastman, N.C. Frey, X. Fu, V. Gharakhanyan, A.S. Krishnapriyan, J.A. Rackers, S. Raja, A. Rizvi, A.S. Rosen, Z. Ulissi, S. Vargas, C.L. Zitnick, S.M. Blau, B.M. Wood, The Open Molecules 2025 (OMol25) Dataset, Evaluations, and Models, 2025.
- [241] L. Barroso-Luque, M. Shuaibi, X. Fu, B.M. Wood, M. Dzamba, M. Gao, A. Rizvi, C. L. Zitnick, Z.W. Ulissi, Open Materials 2024 (OMat24) inorganic materials dataset and models, (2024).
- [242] A. Ramlaoui, M. Siron, I. Djafar, J. Musielewicz, A. Rossello, V. Schmidt, A. Duval, LeMat-Traj: a scalable and unified dataset of materials trajectories for atomistic modeling, (2025).
- [243] Brandon M. Wood, Misko Dzamba, Xiang Fu, Meng Gao, Muhammed Shuaibi, Luis Barroso-Luque, Kareem Abdelmaqsoud, Vahe Gharakhanyan, John R. Kitchin, Daniel S. Levine, Kyle Michel, Anuroop Sriram, Taco Cohen, Abhishek Das, Ammar Rizvi, Sushree Jagriti Sahoo, Zachary W. Ulissi, C. Lawrence Zitnick, UMA: A Family of Universal Models for Atoms (2025).
- [244] S. Sekine, T. Hosaka, H. Maejima, R. Tatara, M. Nakayama, S. Komaba, Na [Mn_{0.36}Ni_{0.44}Ti_{0.15}Fe_{0.05}]O₂ predicted via machine learning for high energy Na-ion batteries, *J. Mater. Chem. A* (2024).
- [245] R. Bushuiev, A. Bushuiev, R. Samusevich, C. Brungs, J. Sivic, T. Pluskal, Self-supervised learning of molecular representations from millions of tandem mass spectra using DreaMS, *Nat. Biotechnol.* (2025).